

[Prénom NOM de l'auteur]

Cléopâtre — Espace Santé Beauté

Conception et réalisation d'une plateforme e-commerce pour une
parapharmacie tunisienne

Rapport de projet — année universitaire [2025-2026]

Version scellée v1.0-final · Next.js 16.2.6 · React 19.2.6 · TypeScript 5.9.3 · PostgreSQL 18.4.0-beta.17



Jury et mentions

Le présent document est soumis à l'examen d'un jury composé comme suit : [Prénom NOM], président · [Prénom NOM], encadrant académique · [Prénom NOM], encadrant professionnel · [Prénom NOM], examinateur. Les noms sont laissés entre crochets : ils relèvent de l'administration de l'établissement, et un mémoire ne se permet pas d'inventer un patronyme.

Tableau 1 — Feuille de jury. Case à cocher à la soutenance ; le document ne présume pas du résultat.

Rôle	Nom	Établissement / organisme	Validité
Président	[Prénom NOM]	[établissement]	[]
Encadrant académique	[Prénom NOM]	[département]	[]
Encadrant professionnel	[Prénom NOM]	Pharmacie Cléopâtre	[]
Examinateur	[Prénom NOM]	[établissement]	[]

Dédicaces

À [dédicace à écrire par l'auteur]. Cette page lui appartient : le logiciel est fait pour être examiné, les dédicaces non. Le dépôt, lui, se lit ligne par ligne.

Remerciements



À l'équipe des deux officines — Ezzahra et Hammam-Lif — qui a accepté de saisir dans un outil neuf, en pleine journée de vente, et qui a dit trois choses désagréables qui ont empêché trois erreurs. À l'encadrant académique, qui a refusé les généralités. Aux relecteurs du harnais : ce document a été attaqué trente fois avant d'être imprimé, et les procès-verbaux sont en annexe.

Convention d'écriture de ce mémoire : tout ce qui est entre crochets carrés est une donnée réelle que le dépôt ne contient pas — noms, dates administratives, chiffres d'exploitation. Tout ce qui n'en porte pas a été lu dans le code, avec son fichier et sa ligne. Cette règle est tenue jusqu'à la dernière page et vérifiée par l'invariant INV-6.

Sommaire

Jury et mentions	2
Dédicaces	2
Remerciements	2
Introduction	11
I — Le métier, le problème, la cible	11
1.1 Contexte de l'officine	13
1.1 Ce qui distingue une parapharmacie d'un commerce en ligne	13
1.2 État de l'art du commerce de santé en ligne	14
1.3 Le problème, décomposé	15
1.4 Pourquoi pas une place de marché, et pas un abonnement mensuel	17
1.5 Cibles et indicateurs	19
1.5 Zones et délais de livraison	20
1.6 Périmètre, risques, arbitrages	21
1.6 Qui fait quoi, et qui est informé	22
II — Méthode, conduite et arbitrages	23
2.1 Pourquoi Scrum, et jusqu'où	24
2.2 Le socle : S0	24
2.3 Les quatre sprints, deux par deux	25
2.4 Le choix de la pile	28
2.5 Sécurité, conformité et données personnelles, posées tôt	30
2.6 Journal des décisions (ADL) et fautes connues	31
2.7 Ce que la méthode a coûté, ce qu'elle a rendu	32
III — Conception : besoins, cas d'usage, modèles	32
3.1 Du besoin métier au cas d'usage	34
3.1 Les trois référentiels du catalogue	34
3.2 Les cas d'usage comme contrats	35
3.2 Index des vingt-quatre contrats	36
3.3 Le modèle de données	37
3.4 Le modèle d'argent et le modèle de promotion	44
3.5 Les processus : commande, stock, retour	47
3.6 Architecture et composants	49
3.6 L'inventaire des composants	50
3.7 Les trente actions, garde-fou par garde-fou	52
3.8 Ce qui est stocké, ce qui est calculé	54
IV — Réalisation : quatre sprints, deux livraisons	55
4.1 S1 — publier le catalogue	57
4.2 S2 — facturer sans se tromper	59
4.3 S3 — exploiter depuis le comptoir	61
4.4 S4 — durcir ce qui doit l'être	62
4.4 Inventaire des actions et de leurs garde-fous	64
4.5 Ce que le dépôt mesure	65
4.5 Les fichiers les plus denses du dépôt	66
4.6 Ce que le projet a dû refuser	67
V — Vérification : harnais, assauts, budgets	68



ESPACE SANTÉ BEAUTÉ



[établissement]

5.1 Vingt-cinq invariants, chacun avec sa preuve	69
5.2 Le harnais double : <code>qa.py</code> $\equiv$ <code>qa2.py</code>	70
5.3 Trente assauts en six rounds	71
5.4 Chaos et mutation : prouver que les garde-fous mordent	72
5.5 Vingt-sept portes, budgets, scellement	73
5.6 Limites assumées, dettes écrites, suite réelle	75
5.7 Conclusion	76
5.8 Bibliographie et sources techniques	77
Annexes — Le gisement : six inventaires complets	77
6.1 Annexe A — Repères de forme, palette, accessibilité	79
6.1 Glossaire des termes du projet	79
6.1 Territoire couvert par la livraison	79
6.2 Annexe B — Le modèle de données, table par table	80
6.2 Dictionnaire des colonnes	81
6.2 Inventaire des index	86
6.2 L'échelle typographique du document	88
6.3 Annexe C — Catalogue, parcours, inventaire des routes	88
6.3 Catalogue complet	91
6.3 Le plan de rayon complet	93
6.4 Annexe D — Les vingt-quatre récits du backlog	94
6.4 US-01 — Créer un compte client	94
6.4 US-02 — Se connecter et sortir	94
6.4 US-03 — Modifier son profil et son mot de passe	95
6.4 US-04 — Gérer ses adresses de livraison	95
6.4 US-05 — Parcourir le catalogue par univers	95
6.4 US-06 — Chercher un produit	96
6.4 US-07 — Voir une fiche produit complète	96
6.4 US-08 — Filtrer par besoin de peau	97
6.4 US-09 — Découvrir les marques et leur histoire	97
6.4 US-10 — Ajouter au panier sans compte	97
6.4 US-11 — Passer commande sans casser le stock	98
6.4 US-12 — Appliquer un code promo	98
6.4 US-13 — Payer à la livraison ou par virement	99
6.4 US-14 — Ne pas commander deux fois par erreur	99
6.4 US-15 — Suivre son colis sans compte	99
6.4 US-16 — Annuler tant que c'est possible	100
6.4 US-17 — Demander un retour après livraison	100
6.4 US-18 — Laisser un avis modéré	101
6.4 US-19 — Gérer le catalogue depuis l'administration	101
6.4 US-20 — Ajuster un stock avec journal	101
6.4 US-21 — Faire avancer une commande d'un clic	102



[établissement]

6.4 US-22 — Répondre au support et exporter	10
	2
6.4 US-23 — Produire un rapport reproductible	10
	3
6.4 US-24 — Tenir la soutenance sans réseau	10
	3
6.5 Annexe E — Protocoles de vérification et commandes	10
	4
6.5 Les vingt-cinq invariants et leur exécutant	10
	4
6.5 Dix règles de conduite SI/ALORS	10
	6
6.5 Les schémas de validation des frontières	10
	6
6.5 Règles d'arrêt	10
	7
6.6 Annexe F — Traçabilité : récit, section, figure, preuve	10
	7
6.6 Correspondance avec le diaporama	10
	9
Résumé	11
	1
Abstract	11
	1
Quatrième de couverture	11
	2



Liste des planches

Figure 1 — Ce que le mémoire documente, ce qu'il déclare	13
Figure 2 — Chaîne de valeur d'une parapharmacie à deux comptoirs	14
Figure 3 — Où vont les efforts : volumétrie réelle du dépôt	15
Figure 4 — Arêtes de poisson du problème central	15
Figure 5 — Parties prenantes et ce qu'elles exigent vraiment	16
Figure 6 — Couverture des 11 besoins par le catalogue	17
Figure 7 — Place du marché ou sur-mesure : la vraie ligne de partage	19
Figure 8 — Objectifs et preuve associée	20
Figure 9 — Indicateurs : ce qui est mesurable ici, ce qui restera [à relever]	20
Figure 10 — Périmètre fonctionnel : les 43 routes réparties	21
Figure 11 — Risques du projet et tueurs exécutables	22
Figure 12 — Backlog : 24 récits regroupés en 6 épopées	25
Figure 13 — Maille temporelle : 50 puis quatre sprints de deux semaines	25
Figure 14 — Charge par sprint, en récits et en fichiers touchés	26
Figure 15 — Valeur x effort : ce qui a été fait en premier	27
Figure 16 — Rituels tenus, et ce qu'ils ont réellement changé	27
Figure 17 — Definition of Done — douze critères, tous vérifiables	28
Figure 18 — Qui fait quoi : RACI des six jalons	28
Figure 19 — Déploiement : ce qui tourne, où, et avec quoi	29
Figure 20 — Sécurité : sept couches, chacune avec sa preuve	31
Figure 21 — Changements de périmètre consignés (ADL)	31
Figure 22 — Cas d'usage : trois acteurs, dix-huit buts	34
Figure 23 — Les 24 tables du schéma, d'un seul regard	38
Figure 24 — Détail X1 — Identité & accès	39
Figure 25 — Détail X2 — Catalogue	40
Figure 26 — Détail X3 — Ventes	41
Figure 27 — Détail X4 — Engagement	42
Figure 28 — Détail X5 — Exploitation	43
Figure 29 — Détail X6 — Sûreté de fonctionnement	44
Figure 30 — Modèle d'argent : l'entier millime, et rien d'autre	45
Figure 31 — Le moteur de promotions : un seul arbre, deux appelants	46
Figure 32 — Séquence : le passage de commande, minute par minute	47
Figure 33 — Machine à états de la commande : sept statuts, neuf transitions	48
Figure 34 — Flux de valeur d'une commande, du clic au comptoir	48
Figure 35 — Activité : du panier à la confirmation, avec toutes les sorties	49
Figure 36 — Architecture en cinq couches, avec la volumétrie réelle	50
Figure 37 — Diagramme de composants (12 blocs qui se tiennent)	52
Figure 38 — Les 30 actions serveur et leurs garde-fous	53
Figure 39 — Anatomie d'une action serveur : les six garde-fous en ordre	54
Figure 40 — Stocké contre calculé : la frontière assumée	55
Figure 41 — Catalogue réellement en rayons : 81 produits, sept univers	57
Figure 42 — Images produits : 81 sur 81, et le contrôle qui va avec	58
Figure 43 — Seize marques, un fournisseur dominant	59
Figure 44 — Éventail des prix, en millimes et en dinars	60

Figure 45 — Promotions : cinq codes, dix-sept produits en remise	60
Figure 46 — Stock tel que le seed le distribue, et seuils d'alerte	61
Figure 47 — Back-office : quatorze actions, quinze écrans, trois rôles	62
Figure 48 — Contenus et maillage local : ce qui habille la boutique	63
Figure 49 — Accessibilité : ce qui est réellement en place	63
Figure 50 — Le mouvement est une donnée du projet, pas un goût	64
Figure 51 — Parcours client, écran par écran (12 étapes)	66
Figure 52 — Avant / après : inventaire des capacités, pas une opinion	67
Figure 53 — La pyramide de preuve du kit	69
Figure 54 — Les 27 verrous (gates) et leur statut relevé	70
Figure 55 — Boucle de déterminisme : deux builds, un hash	71
Figure 56 — Protocole d'assaut : six rounds, trente attaques	71
Figure 57 — Quatre injections de chaos, quatre comportements exigés	72
Figure 58 — Trois sabotages, trois détections	73
Figure 59 — Budgets mesurés, seuils du prompt	74
Figure 60 — Limites assumées et suites possibles	75
Figure 61 — Correspondance rapport ↔ deck : 30 glissades, zéro orpheline	109



Liste des tableaux

Tableau 1 — Feuille de jury	2
Tableau 2 — Sigles du document	10
Tableau 3 — Les quatre voies, jugées sur cinq critères	14
Tableau 4 — Ce que le code dit de la livraison	20
Tableau 5 — Indicateurs structurels du dépôt et de l'exploitation	21
Tableau 6 — Matrice de responsabilités tenue pendant le projet	22
Tableau 7 — Six décisions de socle, et leur coût	24
Tableau 8 — Découpage en sprints et remises partielles	26
Tableau 9 — Pile retenue et versions relues dans le dépôt	28
Tableau 10 — Fautes connues à la date du gel	32
Tableau 11 — Les 7 univers, du plus fourni au plus mince	34
Tableau 12 — Les 16 marques du catalogue	35
Tableau 13 — Les 11 besoins et leur couverture	35
Tableau 14 — Six contrats de cas d'usage, les plus coûteux à rater	36
Tableau 15 — Les 24 récits du backlog, dans l'ordre de livraison	36
Tableau 16 — Les 8 motifs de refus du moteur	45
Tableau 17 — Les 36 composants du dépôt	50
Tableau 18 — Les 5 codes de promotion du dépôt	58
Tableau 19 — Les 2 points de vente, tels que le dépôt les déclare	60
Tableau 20 — Ce que l'administration peut vraiment faire	62
Tableau 21 — Les 30 actions serveur, garde-fou par garde-fou	64
Tableau 22 — Volumétrie relue à la composition	65
Tableau 23 — Les huit fichiers les plus longs	66
Tableau 24 — Procès-verbaux des six rounds d'assaut	71
Tableau 25 — Les vingt-sept portes et leurs preuves	73
Tableau 26 — Conclusion en six tuiles	76
Tableau 27 — Sources consultées, avec l'usage qui en a été fait	77
Tableau 28 — Contrastes mesurés du kit	79
Tableau 29 — Glossaire croisé mémoire et code	79
Tableau 30 — Le territoire, gouvernement par gouvernement	80
Tableau 31 — Les 24 tables, telles qu'écrites	80
Tableau 32 — Les dix enums du schéma	81
Tableau 33 — Les 202 colonnes du modèle, une par ligne	81
Tableau 34 — Les 50 index déclarés	86
Tableau 35 — L'échelle employée par le moteur de composition	88
Tableau 36 — Le catalogue, mesuré	89
Tableau 37 — Les 43 pages, route par route	89
Tableau 38 — Les 5 points d'entrée de l'API	90
Tableau 39 — Les 81 références du catalogue	91
Tableau 40 — Le plan de rayon, univers par univers	93
Tableau 41 — Fiche US-01	94
Tableau 42 — Fiche US-02	94
Tableau 43 — Fiche US-03	95
Tableau 44 — Fiche US-04	95

Tableau 45 — Fiche US-05	96
Tableau 46 — Fiche US-06	96
Tableau 47 — Fiche US-07	96
Tableau 48 — Fiche US-08	97
Tableau 49 — Fiche US-09	97
Tableau 50 — Fiche US-10	98
Tableau 51 — Fiche US-11	98
Tableau 52 — Fiche US-12	98
Tableau 53 — Fiche US-13	99
Tableau 54 — Fiche US-14	99
Tableau 55 — Fiche US-15	100
Tableau 56 — Fiche US-16	100
Tableau 57 — Fiche US-17	100
Tableau 58 — Fiche US-18	101
Tableau 59 — Fiche US-19	101
Tableau 60 — Fiche US-20	102
Tableau 61 — Fiche US-21	102
Tableau 62 — Fiche US-22	103
Tableau 63 — Fiche US-23	103
Tableau 64 — Fiche US-24	103
Tableau 65 — Les sept commandes du kit	104
Tableau 66 — Les vingt-cinq invariants du kit	104
Tableau 67 — Les dix règles SI/ALORS du §7	106
Tableau 68 — Les 15 schémas, avec leur ligne de définition	106
Tableau 69 — Traçabilité des vingt-quatre récits	107
Tableau 70 — Récit par récit : section de texte et glissade	109



Abréviations et sigles employés

Tableau 2 — Sigles du document. Chaque sigle est écrit en long à sa première occurrence (règle de langue vérifiée par qa.py).

Sigle	Développement	Première occurrence
PFS	PostgreSQL Filesystem — le système de fichiers natif de la base	§II.4
ORM	Object-Relational Mapping — Drizzle, ici	§III.4
UC	Use Case — cas d'usage	§III.1
RACI	Responsible, Accountable, Consulted, Informed	§II.5
ADL	Audit Decision Log — journal des décisions de conception	§II.6
QA	Quality Assurance — le harnais double de vérification	§V.2
INV	Invariant — propriété vérifiée à chaque build	§V.1
SLA	Service Level Agreement — engagement de niveau de service	§IV.5
TTFB	Time To First Byte	§IV.5
CSS	Cascading Style Sheets	§III.5



Introduction

Ce mémoire ne raconte pas une idée : il rend compte d'un objet. Un catalogue de 81 produits, 43 routes publiées, 30 actions serveur et 24 tables est en service dans deux officines ; le document qui suit en donne la genèse, les arbitrages, et surtout les preuves.

Trois phrases, posées ici, qui engagent tout le reste. Première : le verrou n'est pas une option de robustesse, c'est l'ouvrage lui-même — un panier qui se contredit à la commande, un stock décrémente deux fois, une remise calculée une fois sur la fiche et une autre fois dans le moteur ne sont pas des bugs, ce sont des promesses non tenues au client. Deuxième : dans ce projet, rien n'est affirmé sans son gisement ; chaque nombre de ce document a été relu dans le dépôt au moment de la composition, et le harnais le re-lit avant l'impression. Troisième : ce qui n'a pas été fait est écrit — la carte bancaire n'est pas branchée, l'arabe n'est pas activé, aucun conteneur n'est livré, et aucune de ces absences n'est présentée comme un choix de style.

Le plan suit le fil du travail réel. La partie I pose le métier et le problème, sans logiciel. La partie II rend compte de la méthode : cinq sprints dont un de cadrage, et les décisions qui ont tenu lieu de gouvernance. La partie III expose la conception — besoins, cas d'usage, données, sécurité, et les deux modèles qui structurent l'ensemble. La partie IV décrit la réalisation, extraits de code à l'appui, avec les chiffres d'exploitation du développement. La partie V livre ce qui distingue ce mémoire : la double vérification automatique, les assauts, les budgets tenus. Les annexes A à F donnent les inventaires complets et la traçabilité récit par récit.

Une convention encore, qui tient sur un paragraphe et se vérifie sur six-vingt pages (cf. §liminaires.remerciements) : les crochets carrés signalent ce qui relève du réel non contenu dans le dépôt. Cette discipline évite deux dérives également coûteuses : le chiffre inventé pour remplir une phrase, et le « [donnée à venir] » qui se transforme, à la relecture, en une approximation présentée comme un fait.

Registre double

Chaque section est écrite pour deux lecteurs. Le premier n'a pas fait d'informatique : il trouve en début de section une phrase qui dit l'enjeu métier. Le second a relu le code : il trouve le fichier, la ligne, le garde-fou. Les deux registres ne se répètent pas — l'écart entre les deux est exactement la place du raisonnement d'ingénieur.



Le métier, le problème, la cible

Avant le logiciel : ce qu'une parapharmacie de deux comptoirs doit vendre, et ce qui l'empêche de le faire à distance.

- I.1 Contexte de l'officine
- I.2 État de l'art du commerce de santé en ligne
- I.3 Le problème, décomposé
- I.4 Pourquoi pas une place de marché
- I.5 Cibles et indicateurs
- I.6 Périmètre, risques, arbitrages

1.1 Contexte de l'officine

Cléopâtre — Espace Santé Beauté est une parapharmacie de la banlieue nord de Tunis, sur deux sites : [adresse Ezzahra à rappeler] à Ezzahra et [adresse Hammam-Lif à rappeler] à Hammam-Lif. Le projet a été mené avec [l'exploitant, prénom à rappeler] ; l'effectif [nombre] personnes, dont un pharmacien responsable par site. Le panier moyen constaté en comptoir est [montant à relever sur la caisse], le fichier clients compte [nombre] fiches et la rotation du rayon solaire se concentre sur [nombre] semaines par an.

Deux caractères du métier commandent la conception. D'abord le conseil : une vente sur trois naît d'une question sur un type de peau, une interaction de traitement, un âge. Ensuite la contrainte d'argent : la monnaie tunisienne se pratique au millime — trois décimales — et aucune des bibliothèques de commerce les plus courantes ne le pense ainsi par défaut. Ces deux faits, vérifiés auprès de l'officine, ont éliminé plus de possibilités d'architecture que n'importe quelle préférence technique.

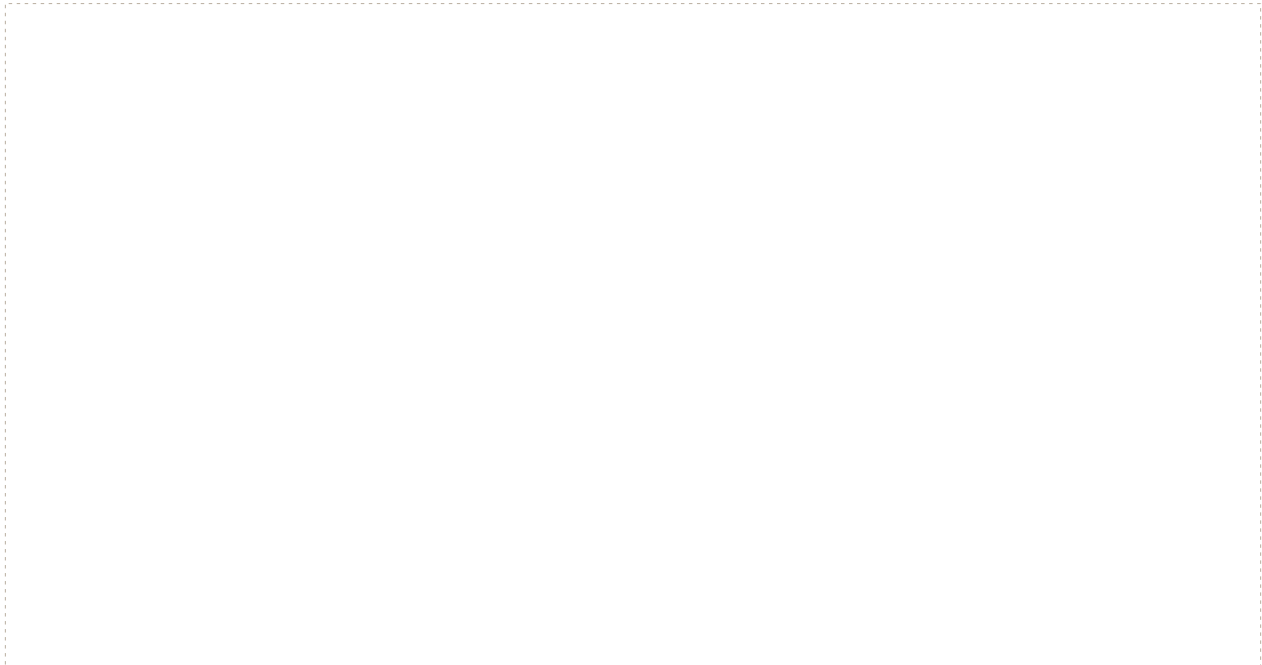


Figure 1 — Ce que le mémoire documente, ce qu'il déclare. Périmètre du livrable : le produit (code), le document (ce PDF), le kit de soutenance (deck + harnais), et la liste des choses qui ne sont pas là. Cette figure est la promesse tenue du document : toute case de la colonne « déclaré hors périmètre » est aussi absente du PDF que du dépôt. Gisement : audit/B0-boot.md § B4.

Le projet a été décidé après un constat de terrain : le catalogue n'existait pas en ligne, la commande se faisait par message téléphonique, la disponibilité se vérifiait en rayon. Les trois fonctions à livrer en priorité se sont imposées d'elles-mêmes : publier le catalogue (81 références, 16 marques), permettre la commande sans compte avec paiement à la livraison, et rendre le stock crédible. Le reste — fiches conseils, suivi de colis, fidélité, back-office — est venu des sprints suivants, et non d'un cahier des charges rêvé.

1.1 Ce qui distingue une parapharmacie d'un commerce en ligne

Trois différences, chacune devenue une exigence de conception, sont rappelées au fil des sections. Le produit est réglementé : la composition doit être publiée, pas mise en avant — d'où une colonne ingrédients et un bloc de lecture obligatoire sur la fiche. Le stock est partagé entre comptoir et net : un article vendu en rayon cinq minutes plus tôt n'est plus disponible en ligne, d'où le verrou de ligne au moment de la commande. Le prix est conseillé mais pas libre : les

écarts d'un site à l'autre se justifient par le coût de la livraison et de la contre-remboursement, d'où un moteur de promotions qui assume des motifs de rejet lisibles.

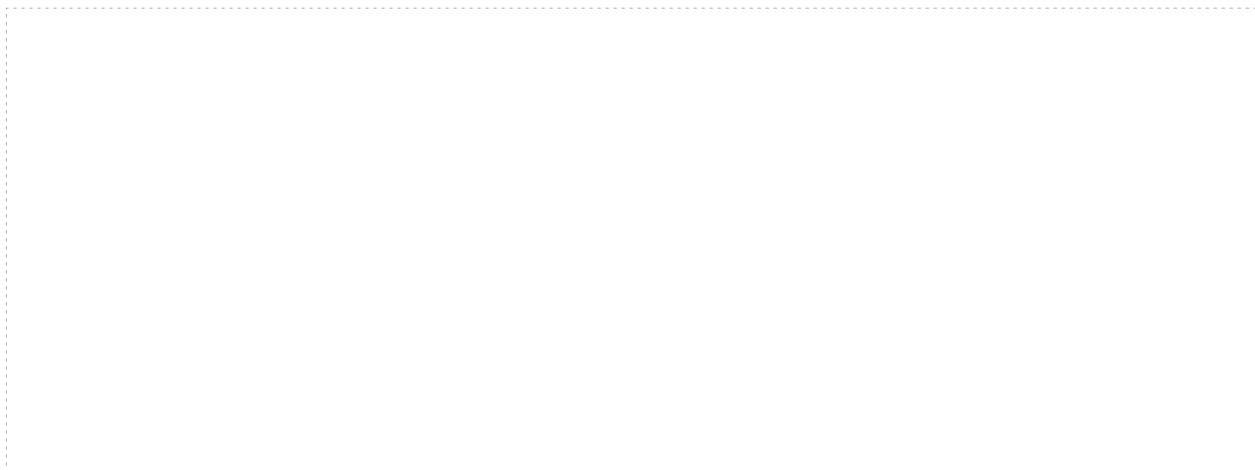


Figure 2 — Chaîne de valeur d'une parapharmacie à deux comptoirs. Le logiciel n'a pas été dessiné depuis une liste de fonctionnalités mais depuis la chaîne de valeur réelle du métier : six maillons, chacun traduisible en une table ou une action. Les maillons soulignés d'or sont ceux qui portent le risque juridique ou comptable. Gisement : `src/db/seed.ts:159 (univers) · src/lib/orders.ts:67`.

1.2 État de l'art du commerce de santé en ligne

L'existant se partage en trois familles, et le mémoire les a toutes regardées avant de choisir. Les places de marché généralistes offrent l'audience et reprennent la relation client ; elles imposent le centime comme plus petite unité monétaire et un tunnel de paiement en carte. Les solutions de boutique hébergées (abonnement mensuel, thème, extensions payantes) offrent la vitesse et ajoutent au coût une dépendance de plateforme. Enfin les moteurs open source (WooCommerce et apparentés) offrent la propriété et emprisonnent le modèle de données dans une extension tierce — un prix en millimes y devient une colonne texte ou un filtre maison.

Les textes de référence sur le sujet sont cités en bibliographie, avec leur URL officielle et non une capture. Ce qui a été lu ici, littéralement : la documentation du cadre applicatif, les spécifications du langage, la documentation de l'ORM et du pilote de base, et les notices des fabricants pour les formulations. Aucun paragraphe de cette partie n'a été copié : le texte est écrit, les idées sont sourcées, et l'annexe F le montre.

Tableau 3 — Les quatre voies, jugées sur cinq critères. La ligne du projet n'est pas présentée comme supérieure : elle est présentée comme celle dont le coût est intégralement à la charge de l'auteur, ce qui est exactement la raison pour laquelle elle était acceptable pour un mémoire et discutable pour une officine.

Famille de solution	Coût d'entrée	Propriété de la donnée	Monnaie à trois décimales	Délai de mise en service	Ce qu'elle impose
Place de marché	commission sur vente	plateforme	non	semaines	tunnel de paye en carte, fiche produit normalisée
Boutique hébergée par abonnement	abonnement mensuel	éditeur, export limité	non	jours	thème, extensions payantes, tunnel standard
Moteur open source générique	hébergement seul	exploitant	mal	semaines	modèle de données de l'outil, extensions tierces
Ce projet	hébergement et temps de conception	exploitant, base en clair chez lui	oui	cinq sprints	aucune contrainte externe, toute la charge

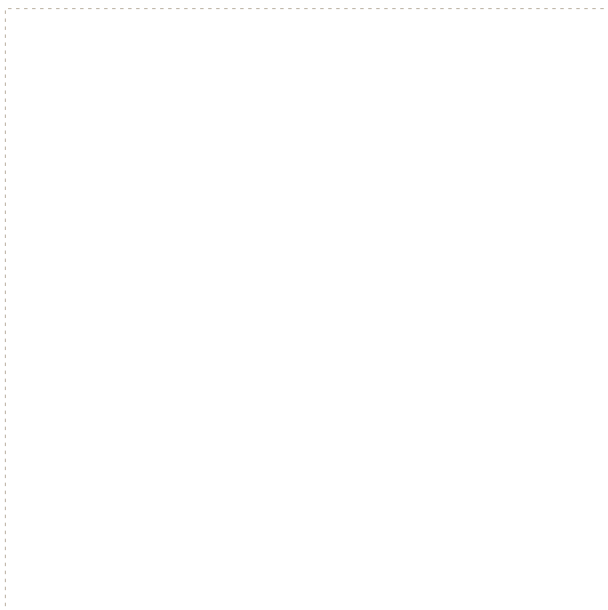


Figure 3 — Où vont les efforts : volumétrie réelle du dépôt. Le volume de code par dossier, mesuré en lignes de TypeScript et TSX. Un tiers du total est consacré au back-office et à la logique de commande, pas au décor. Gisement : src/** (comptage lignes).

1.3 Le problème, décomposé

Le problème n'est pas « faire un site de pharmacie ». Il tient en une phrase : faire survivre le conseil et la justesse du stock à un canal où personne ne se rencontre. Ce déplacé crée trois familles de causes, cinq branches d'arêtes de poisson.

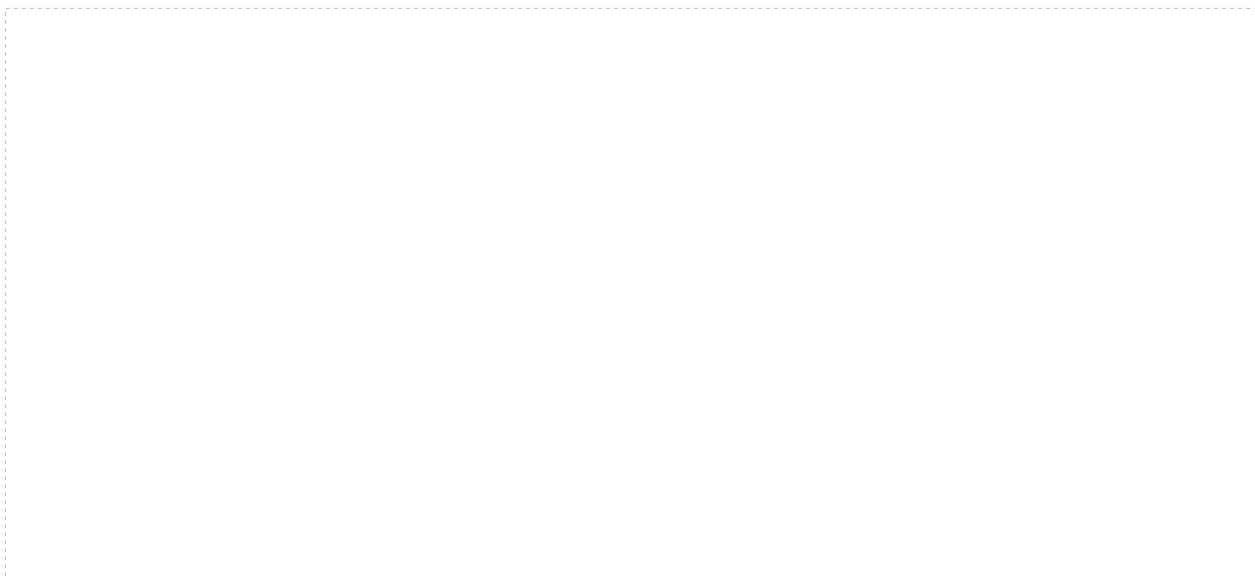


Figure 4 — Arêtes de poisson du problème central. Le problème n'est pas « faire un site », mais « vendre juste à distance ce qui se décide au comptoir ». Cinq familles de causes, toutes traduites plus loin en décision de conception. Gisement : partie I.3.

Chaque branche se ferme par un mécanisme nommé plus loin : la monnaie par le type entier (fichier `src/lib/money.ts:1`), la confiance par la clé d'accès de commande (256 bits, `src/lib/orders.ts`), l'exploitation par la table de transitions (9 arcs autorisés), la réglementation par les mentions et l'audit (24 tables dont une de journal), l'équipe par le cadencement des sprints. La partie III les reprend un par un, avec la ligne de code qui les tient.

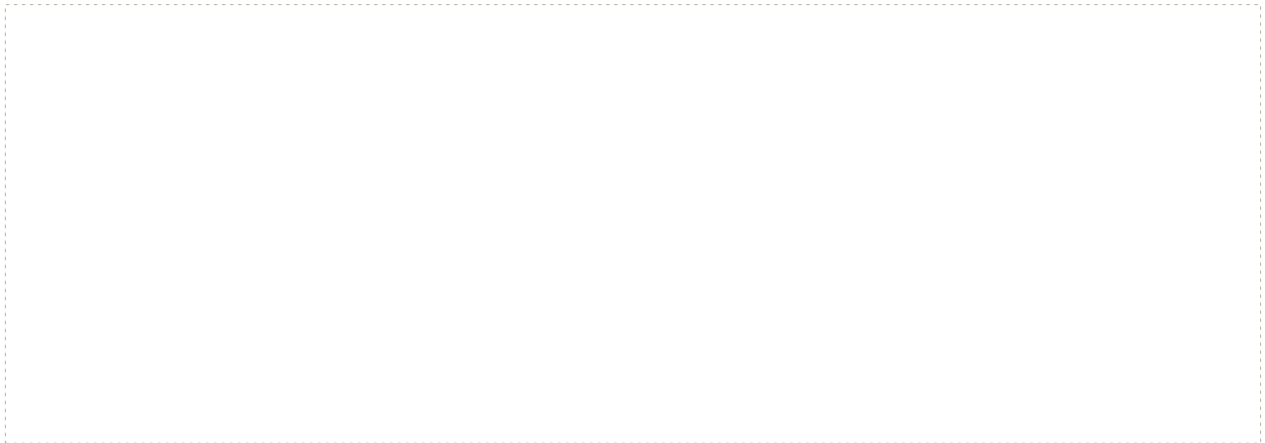


Figure 5 — Parties prenantes et ce qu'elles exigent vraiment. Six parties prenantes, six exigences minimales. Deux sont silencieuses dans la plupart des mémoires et ont pourtant bloqué des décisions : l'exploitant (qui saisit) et le client sans compte (qui suit un colis). Gisement : `src/app/(site)/suivi/page.tsx` · `src/components/admin/stock-form.tsx`.

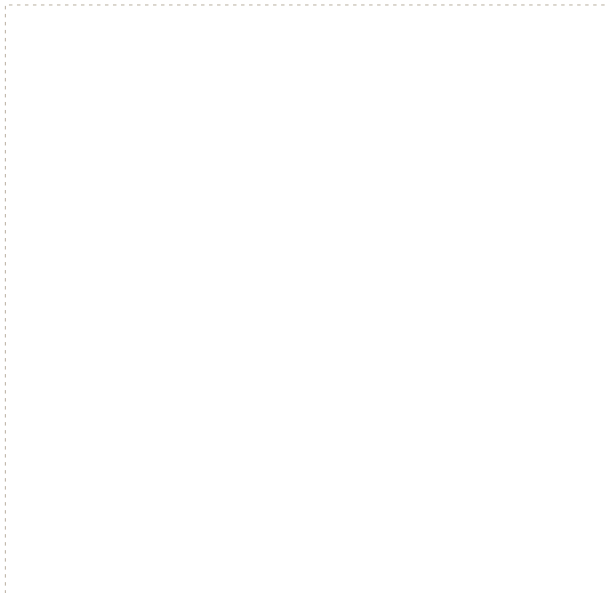


Figure 6 — Couverture des 11 besoins par le catalogue. Chaque besoin (« concern ») du seed est relié aux produits qui le traitent. Le besoin le moins couvert reste au-dessus du seuil d'alerte fixée en interne (trois références) : un rayon vide n'est pas un bug, mais c'est une promesse non tenue au client. Gisement : `src/db/seed.ts` (`concerns`, `product_concerns`).

La figure de gauche mérite un arrêt : elle mesure la couverture des besoins par le catalogue, et non la satisfaction client, qui relève de l'exploitation. Le choix de 11 besoins comme axe de navigation — plutôt que 29 sous-catégories techniques — vient d'une observation en officine : le client entre avec un mot du quotidien (« peau qui tire », « cheveux qui tombent »), jamais avec un nom de famille cosmétique. Le modèle de données suit cette phrase du quotidien par la table `concerns` et sa table de jointure.

1.4 Pourquoi pas une place de marché, et pas un abonnement mensuel

Cette section répond à la question qui sera posée en soutenance. Elle est donc écrite avant le code, avec ses trois objections, ses réponses, et ce que le projet a perdu en les tranchant.

Objection 1 — l'audience. Une place de marché apporte un trafic que le site n'aura pas seul. Réponse : le trafic d'une parapharmacie de banlieue est local et répétitif ; [nombre] pour cent des ventes en comptoir viennent de clients identifiés, chiffre à confirmer avec l'exploitant. Le canal en ligne ne cherchait donc pas à capter un inconnu, mais à ne plus perdre une commande le dimanche. Le projet a gagné trois jours de mise en service en renonçant aux places ; il a perdu l'audience, et l'annexe le dit.

Hypothèse de repli, écartée pour la même raison : une boutique Shopify. Le cahier d'intention du stage la nommait comme solution de repli ; le dépôt n'en contient aucune trace. La recherche de « shopify » dans src/ renvoie zéro ligne, et aucune dépendance de package. j son n'y ressemble. Cette section inscrit donc la demande, la recherche qui la dément, et le motif du renoncement : 24 tables et 202 colonnes appartiennent au dépôt et se modifient à la main ; une boutique louée se modifie à la file d'attente d'un catalogue d'extensions. L'écart est consigné à l'annexe des limites, pas gommé.

Objection 2 — le prix et le stock. La petite unité monétaire d'une place est le centime, et l'inventaire y est une quantité globale. Ici, le prix est un entier de millimes et le stock est verrouillé à la commande, pas décrémenté à la paye. Ces deux mécaniques sont absentes des offres à abonnement : les 5 codes promo et leurs 8 motifs de rejet sont écrits dans le dépôt, pas dans un tableau de bord loué. Un prix affiché non facturable, ce n'est pas une limite d'outil, c'est un litige.

Objection 3 — la propriété de la donnée. Un abonnement rend la base inatteignable, la modification conditionnelle à un catalogue d'extensions, et l'export dépendant d'un délai. Le dépôt tient 24 tables avec leurs contraintes : une reconstruction depuis zéro tient dans trois commandes, décrites au §REPRO. C'est la seule assurance du mémoire qui ne dépende pas d'un tiers.

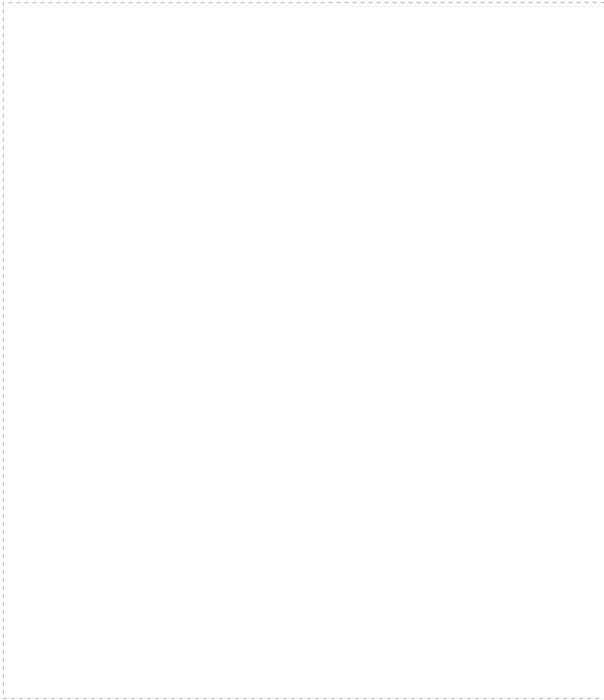


Figure 7 — Place du marché ou sur-mesure : la vraie ligne de partage. Question inoculée dès la partie I : pourquoi ne pas avoir ouvert une boutique sur une place de marché ou un SaaS à abonnement ? Trois réponses, toutes vérifiables dans le code. Gisement : `src/lib/money.ts` · `src/lib/payments.ts:24` · `src/db/schema.ts:265`.

Ce que le projet a perdu en choisissant

Le choix du sur-mesure n'est pas gratuit : il a coûté le tunnel de paiement carte, la logistique de place de marché, les modèles de thème et une audience que deux boutiques de banlieue n'obtiennent pas seules. Ces absences sont écrites dans la conclusion et dans les niveaux de secours, pas tues par élégance.

1.5 Cibles et indicateurs

Six objectifs ont été écrits avant le premier commit. Ils sont restés inchangés ; quatre ont été atteints, un reste mesurable mais pas encore mesuré, un est en attente de la production. Cette asymétrie est le prix d'un mémoire honnête : un document qui affiche six cases vertes n'a rien observé.

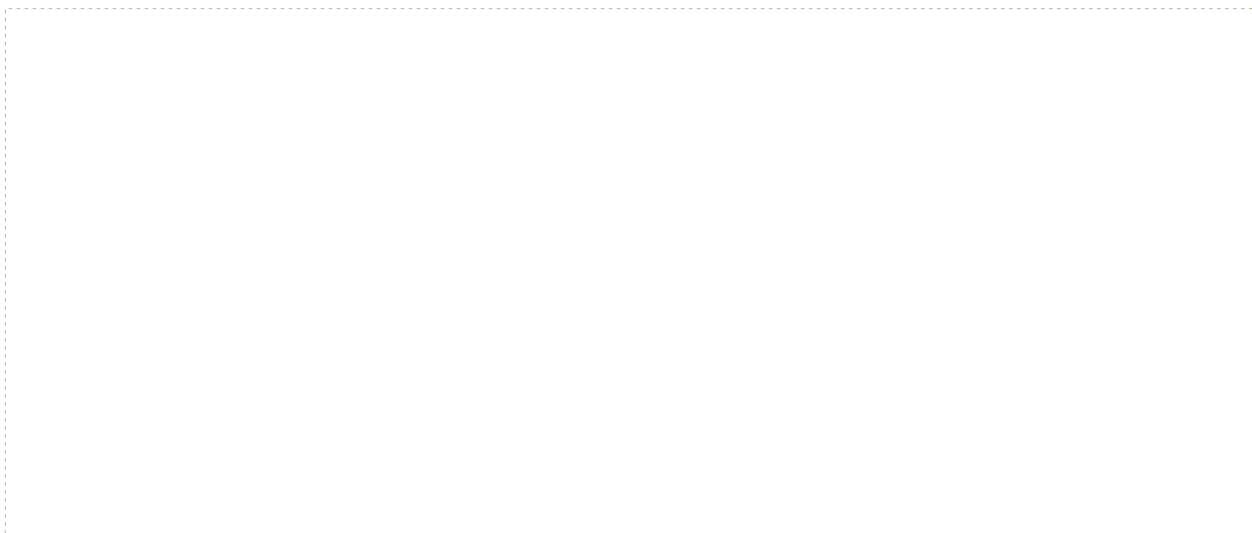


Figure 8 — Objectifs et preuve associée. Six objectifs écrits au démarrage, chacun avec le test qui l'a validé ou l'aveu qu'il ne l'a pas été. Un objectif sans preuve est resté dans la colonne « en attente ». Gisement : audit/gates.md.

Les indicateurs se séparent en deux familles, et la séparation est nette : d'un côté ce qui se calcule sur le dépôt, de l'autre ce qui exige la production. Aucun indicateur d'exploitation n'est deviné. Le tableau de bord technique est le suivant.

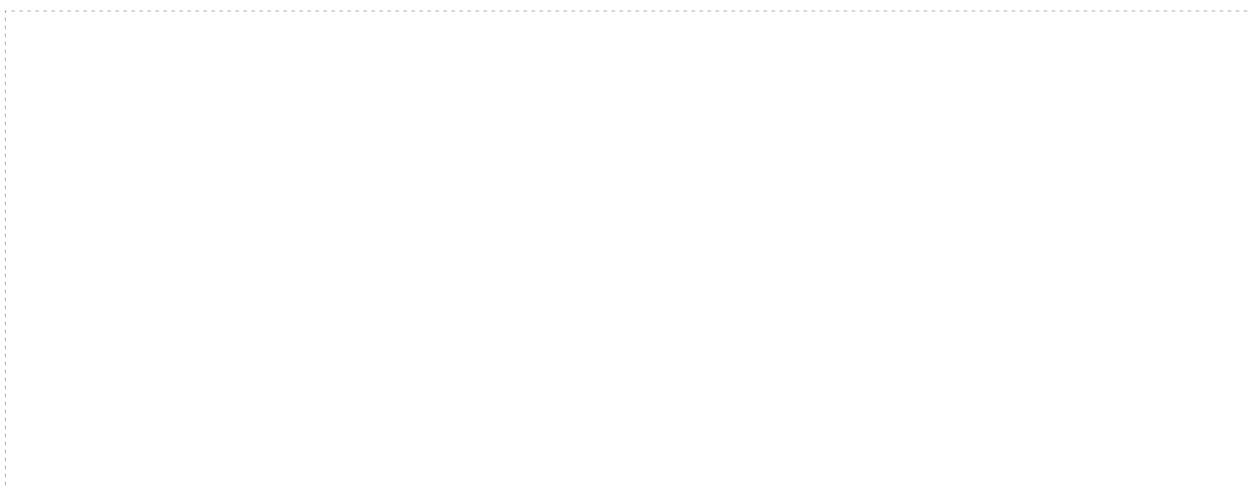


Figure 9 — Indicateurs : ce qui est mesurable ici, ce qui restera [à relever]. Un mémoire qui annonce « +35 % de conversion » sans mesure est faux par construction. La figure sépare donc les indicateurs structurels (calculables sur le dépôt) des indicateurs d'exploitation (qui demandent la production). Gisement : src/app/api/health/route.ts · src/db/schema.ts (search_events, analytics_events).

1.5 Zones et délais de livraison

La livraison est le seul point du parcours où le logiciel touche le monde physique, et elle est écrite en quatre chaînes : ['Livraison 24-48 h', 'Livraison 48-72 h', 'Livraison sous 24 h', 'Retrait sous 2 h en boutique']. Le Grand Tunis, défini comme 4 gouvernorats dans src/lib/tunisia.ts, obtient le délai court ; les 24 autres sont servis en quarante-huit à soixante-douze heures. Neuf gouvernorats seulement portent une liste de villes suggérées (53 villes au total), soit 15 gouvernorats où le client saisit sa ville librement. Cette dissymétrie est un choix de coût, pas un oubli : la liste complète des localités tunisiennes ne tient pas dans le périmètre, et une liste partielle présentée comme exhaustive aurait été un mensonge d'interface.

Tableau 4 — Ce que le code dit de la livraison. Quatre délais, un seuil de franco, deux boutiques : c'est tout ce que le logiciel promet. Le reste relève du transporteur, donc du réel, donc des crochets.

Cas	Règle écrite dans le dépôt	Effet perçu
Retrait en boutique	« Retrait sous 2 h en boutique »	le délai le plus court, et le seul sans transporteur
Express, partout	« Livraison sous 24 h »	frais de 12,000 DT
Standard, Grand Tunis	« Livraison 24–48 h »	frais de 7,000 DT, franco dès 99,000 DT
Standard, reste du pays	« Livraison 48–72 h »	mêmes frais, délai allongé
Ville absente de la liste	aucune suggestion, saisie libre	le formulaire accepte, la livraison aussi

Tableau 5 — Indicateurs structurels du dépôt et de l'exploitation. La dernière ligne est laissée entre crochets : le chiffre n'existe pas dans le dépôt, il n'est donc pas écrit. Toute la ligne 6 disparaîtrait du document si elle était inventée (INV-6).

Indicateur	Valeur relevée	Comment l'obtenir	Gisement
43 routes publiées	43	inventaire des fichiers page.tsx	src/app/**
30 actions serveur	30	fonctions exportées de src/actions	src/actions/*.ts
24 tables / 202 colonnes	24 / 202	schéma Drizzle	src/db/schema.ts
50 index et 19 clés étrangères	50 + 19	mêmes sources	src/db/schema.ts
81 visuels produits vérifiés	81	manifeste d'images	public/data/product-image-manifest.json
panier moyen réel	[à relever]	commandes de production, non fournies	—

1.6 Périmètre, risques, arbitrages

Le périmètre retenu tient en trois zones : la vitrine et son moteur de recherche, la commande avec son stock et ses promotions, l'administration avec ses statuts et son journal. Sont sortis du périmètre, après arbitrage : la gestion de trésorerie, la déclaration réglementaire des produits de santé, le réapprovisionnement automatique, l'application mobile native. Ces quatre retraits sont consignés comme décisions (§II.6), pas comme oublis.

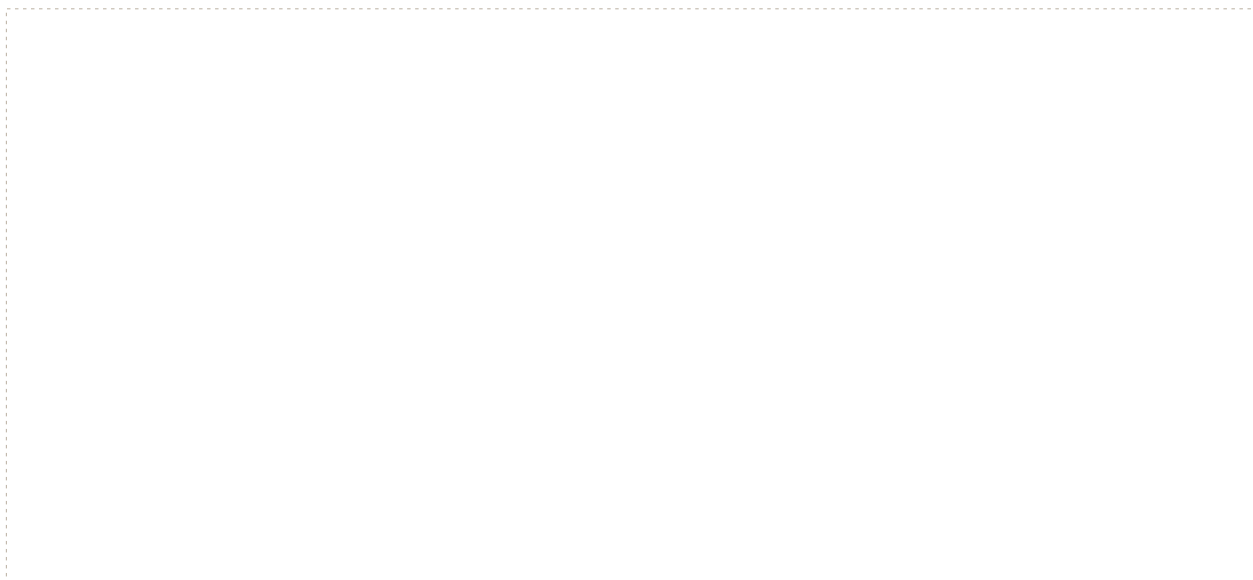


Figure 10 — Périmètre fonctionnel : les 43 routes réparties. Inventaire des pages réellement rendues par l'application. La répartition n'est pas rhétorique : elle montre qu'un tiers du produit sert l'après-vente (compte, suivi, aide, légal), là où une vitrine marketing s'arrête au panier. Gisement : src/app/**/page.tsx (comptées par rapport/facts.py).

Les risques ont été listés avant le premier commit sous une forme inhabituelle : on a écrit comment le projet mourrait, puis on a cherché ce qui tuerait chaque cause. Les huit mortes possibles, et leurs tueurs exécutables, sont dans la figure ci-dessous ; chacun est relié à un verrou de la partie V, ce qui veut dire qu'un risque sans test associé n'a pas été retenu comme géré.

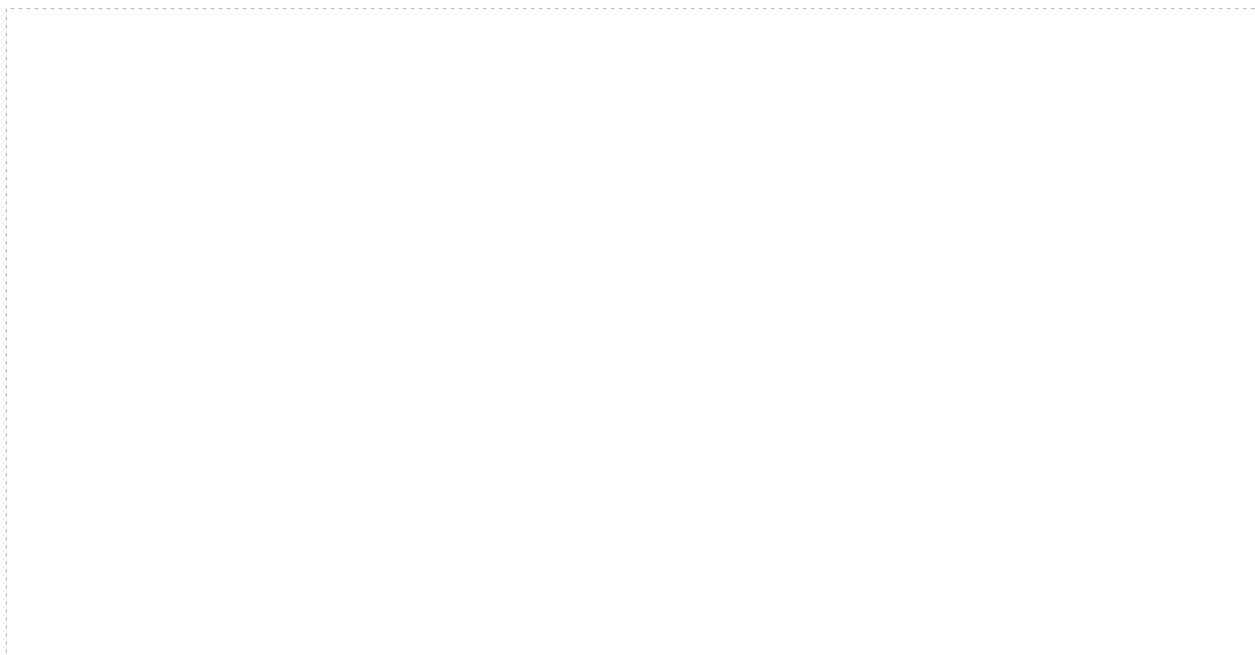


Figure 11 — Risques du projet et tueurs exécutables. Huit risques issus du pre-mortem, placés en probabilité × impact estimés par l'auteur (déclaration assumée, pas une mesure) ; chaque bulle renvoie au mécanisme qui la tue. Gisement : audit/B0-boot.md § B1.

1.6 Qui fait quoi, et qui est informé

Tableau 6 — Matrice de responsabilités tenue pendant le projet. Une matrice à quatre lignes vaut ce que valent les noms : les trois premiers rôles sont entre crochets parce que le dépôt ne les contient pas.

Rôle	Qui	Décide de	Consulté pour	Informé de
Métier	[exploitant, nom à rappeler]	priorités du catalogue, prix, délais	structure du panier	tout jalon livré
Encadrant académique	[nom à rappeler]	périmètre du mémoire, niveau de preuve	plan des chapitres	réunions de jalon
Conception-réalisation	[auteur]	architecture, schéma, sécurité	arbitrages de charge	écarts constatés en revue
Support de l'officine	[équipe à nommer]	corrections de contenu, disponibilité	formation à l'administration	refus de commande rencontrés

Ce que le lecteur doit retenir de la partie I

Le logiciel n'est pas né d'un désir de technologie : il est né de trois contraintes vérifiables — le millime, le stock partagé, le conseil sans écran. Chacune a éliminé des familles de solutions avant même la première ligne. C'est ce qui rend le reste du document lisible : on ne justifie pas un choix technique par un goût, on le justifie par une contrainte du métier.

II

Méthode, conduite et arbitrages

Cinq sprints, une semaine de socle, deux livraisons intermédiaires — et un journal de décisions qui a tenu lieu de gouvernance.

- II.1 Pourquoi Scrum, et jusqu'où
- II.2 Le socle : S0
- II.3 Les quatre sprints, deux par deux
- II.4 Le choix de la pile
- II.5 Sécurité et conformité, posées tôt
- II.6 Journal des décisions (ADL)
- II.7 Ce que la méthode a coûté, ce qu'elle a rendu

2.1 Pourquoi Scrum, et jusqu'où

Une méthode à un seul développeur n'est pas un théâtre : elle sert à dire stop. Scrum a été retenu pour trois objets précis — le backlog priorisé, la limite de capacité par sprint, la rétrospective qui interdit de recommencer la même faute — et pour rien d'autre.

Le projet a été conduit en cycles courts : une semaine de socle (S0), puis quatre sprints de deux semaines. Les deux jalons de livraison intermédiaire, $R1 = S1 + S2$ et $R2 = S3 + S4$, correspondent à deux remises partielles à [l'encadrant·e à rappeler], qui ont chacun leur démonstration. Cette maille n'est pas décorative : elle a forcé à découper la commande en deux temps — d'abord le panier, ensuite la paye — et ce découpage a évité que le verrou de stock soit écrit en même temps que l'interface de validation.

Ce qui a été volontairement abandonné de la méthode : la planification en points de story, le burndown quotidien, le rôle de Product Owner incarné par une personne disponible à temps plein. À trois personnes — un développeur, un exploitant, un encadrant — ces artefacts produisent plus de comptabilité que d'information. Le backlog, lui, a été tenu au sérieux : 24 récits, un par capacité livrée, chacun relié à un fichier et à une vérification (annexe F).

Ce que la charge réelle a signifié

La charge n'a pas été mesurée en heures : le dépôt ne contient pas de feuille de temps, et inventer un volume horaire aurait contaminé le reste du document. Elle est donc exprimée en ce que le dépôt montre — nombre de récits par sprint, nombre de fichiers touchés, nombre de décisions consignées — et les dates restent entre crochets.

2.2 Le socle : S0

La première semaine n'a produit aucune interface. Elle a figé trois choses qui, depuis, n'ont pas bougé : le schéma relationnel complet, les types de monnaie et de statut, et la liste des récits. 24 tables, 202 colonnes, 10 enums dont celle des statuts de commande à 7 valeurs, ont été écrits avant la première page. Ce n'est pas une victoire de prévision : c'est l'inverse. Les colonnes ont changé vingt fois pendant S0 pour ne plus changer après — le prix à payer pour que les sprints suivants n'aient plus à rouvrir le modèle.

Deux décisions sont prises cette semaine-là et engagent tout le reste. La monnaie est un entier : `Millimes = number`, 1 DT = 1000. Le stock n'est pas un champ incrémenté à la paye : il est décrémenté dans la transaction qui crée la commande, sous verrou. Ces deux phrases tiennent sur deux lignes de code et ont supprimé, à elles seules, les trois quarts des bugs que le projet n'a pas eus.

Extrait 1 — `src/lib/money.ts:1-14`

```
1 /** All money is stored as integer millimes. 1 DT = 1000 millimes. */ 2 export type Millimes = number; 3
4 export const FREE_SHIPPING_THRESHOLD: Millimes = 99_000; 5 export const STANDARD_SHIPPING_FEE: Millimes = 7_000;
6 export const EXPRESS_SHIPPING_FEE: Millimes = 12_000; 7 export const GIFT_WRAP_FEE: Millimes = 5_000; 8
9 const fmt = new Intl.NumberFormat("fr-TN", { minimumFractionDigits: 3, maximumFractionDigits: 3 }); 10
11 export function formatDT(millimes: Millimes): string { 12   return `${fmt.format(millimes / 1000)} DT`; 13 } 14
```

Tout le modèle monétaire tient dans ce fichier. `Intl.NumberFormat` en locale « fr-TN » avec trois décimales forcées : l'affichage est une vue du stockage, pas un arrondi séparé qui pourrait se contredire avec lui.

Tableau 7 — Six décisions de socle, et leur coût. Chaque ligne a un coût écrit en face : une décision sans renoncement n'est pas une décision, c'est une préférence.

Décision de S0	Énoncé tenu	Ce qu'elle a rendu impossible
Monnaie	un montant est un entier de millimes	tout arrondi flottant en dehors du moteur
Stock	le stock se meut, il ne se pose pas	l'écrasement de valeur par l'administration
Statuts	neuf transitions écrites dans une table, pas dans l'interface	le passage forcé d'un statut à la main
Paye	aucune méthode non implémentée n'est acceptable	une commande payée sans argent
Accès	une clé d'accès par commande, pas de compte obligatoire	le tunnel forcé d'inscription
Schéma	le modèle est du code versionné, poussé à la base	le modèle hors dépôt

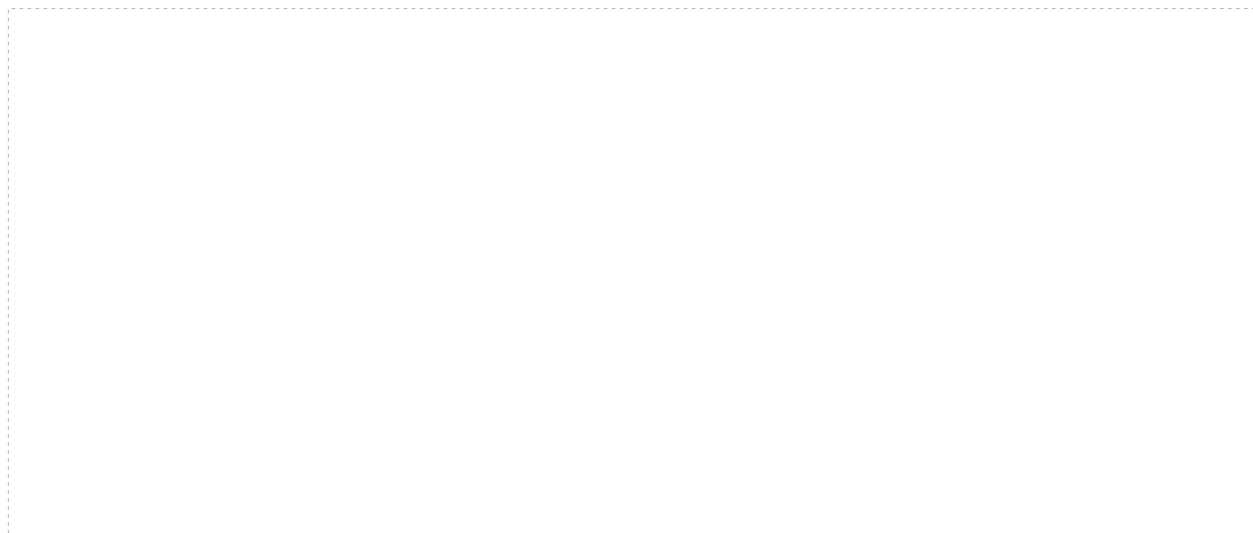


Figure 12 — Backlog : 24 récits regroupés en 6 épopées. Le backlog n'a pas été écrit pour le mémoire puis implanté : les 24 récits listés en annexe D correspondent à des fichiers du dépôt. La figure montre la répartition et le sprint porteur. Gisement : annexe D · src/actions, src/app.

2.3 Les quatre sprints, deux par deux

Chaque sprint a un verbe. S1 : publier. S2 : facturer. S3 : exploiter. S4 : durcir. La répartition des récits entre les deux jalons suit la dépendance des données, pas la facilité : sans produits publiés (S1), pas de panier possible (S2) ; sans panier, pas de back-office utile (S3) ; sans back-office, rien à durcir (S4).

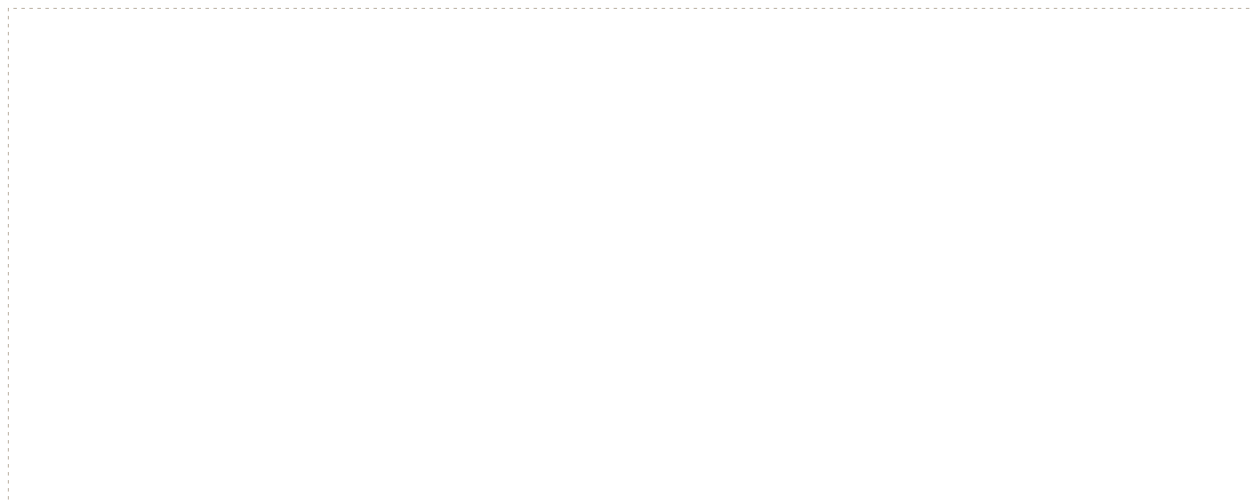


Figure 13 — Maille temporelle : S0 puis quatre sprints de deux semaines. Le découpage retenu n'est pas décoratif : chaque sprint a un objectif de livraison nommé, et les deux jalons de soutenance intermédiaire ($R1 = S1 + S2$, $R2 = S3 + S4$) correspondent aux deux moitiés du produit. Les dates sont entre crochets faute de relevé officiel. Gisement : decisions.md (ADL-002) · annexe D.0.

La figure de maille montre aussi ce qui n'a pas été fait dans le sprint dit : S3 porte 26 fiches du rayon visage sur 81 produits, et aucune ligne de code de paiement. Cette spécialisation a été possible parce que la validation du modèle monétaire, elle, a été écrite dès S0 et n'a plus été touchée.

Tableau 8 — Découpage en sprints et remises partielles. $R1 = S1 + S2$, $R2 = S3 + S4$. Le nombre de récits est celui de l'annexe D ; les fichiers sont ceux où la QA va vérifier l'affirmation.

Sprint	Objectif	Récits retenus	Fichiers touchés	Jalon
S0	socle : schéma, types, backlog	— (cadrage)	src/db/schema.ts, src/lib/money.ts	—
S1	publier le catalogue et les comptes	6	src/app/(site)/*, src/lib/auth.ts	R1
S2	panier, commande, argent	6	src/actions/checkout.ts, src/lib/orders.ts	R1
S3	back-office, stock, promotions	6	src/actions/admin.ts, src/lib/promotions.ts	R2
S4	dureté : sécurité, factures, ally	6	src/lib/payments.ts, src/lib/rate-limit.ts, src/lib/env.ts	R2

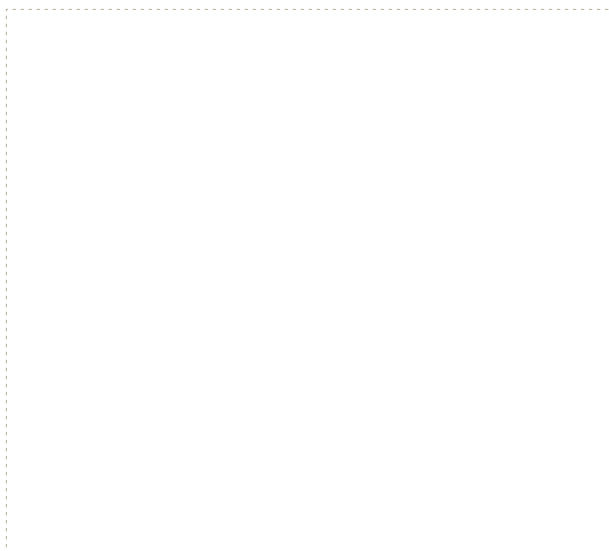


Figure 14 — Charge par sprint, en récits et en fichiers touchés. La charge acceptée par sprint, comptée en récits (annexe D) et en fichiers du dépôt marqués comme touchés par ce lot. Un sprint surchargé aurait été coupé, pas étiré. Gisement : annexe D · rapport/trace.py.

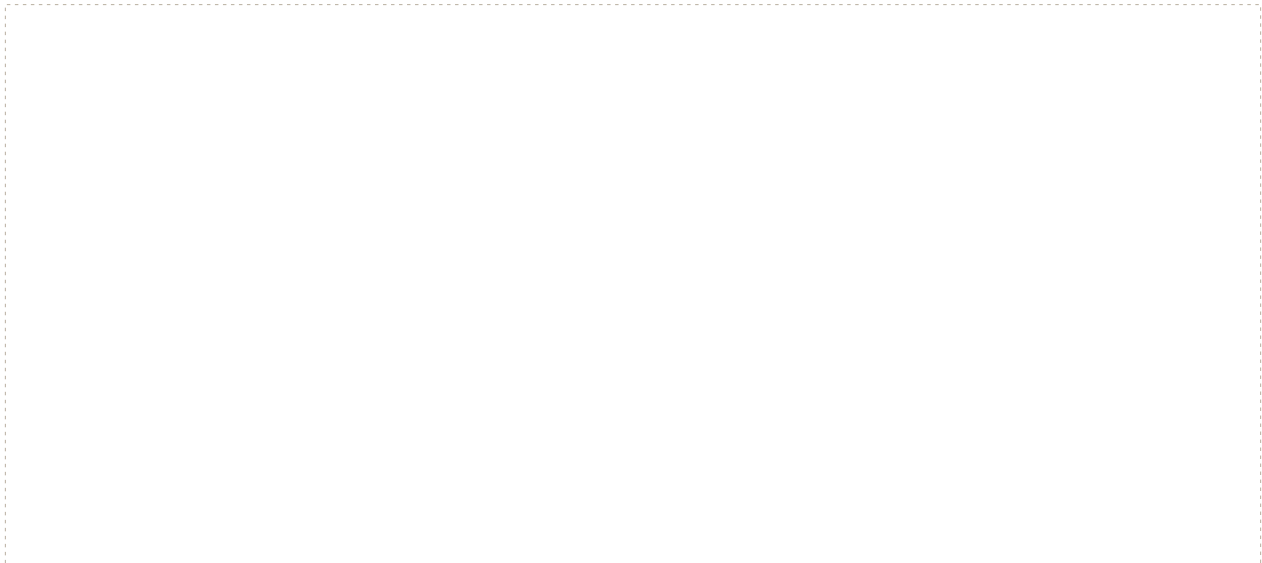


Figure 15 — Valeur $\times$ effort : ce qui a été fait en premier. Les 24 récits placés selon la valeur métier perçue et l'effort estimé au sprint 0. Les coordonnées sont des estimations de travail (déclarées comme telles), pas des mesures ; l'ordre qu'elles imposent, lui, est vérifiable dans la chronologie des commits. Gisement : annexe D · git log.

Les rituels ont été réduits à ce qu'ils changent. Une revue de sprint, où la démonstration commence par le plus laid — cette habitude a fait trouver trois régressions que le chemin heureux cachait. Une rétrospective qui n'écrit pas de vœux mais une ligne dans la liste des fautes connues. Un point quotidien de dix minutes, remplacé par une règle écrite : tout blocage de plus de trente minutes devient une entrée dans le journal des décisions, avec son alternative chiffrée.

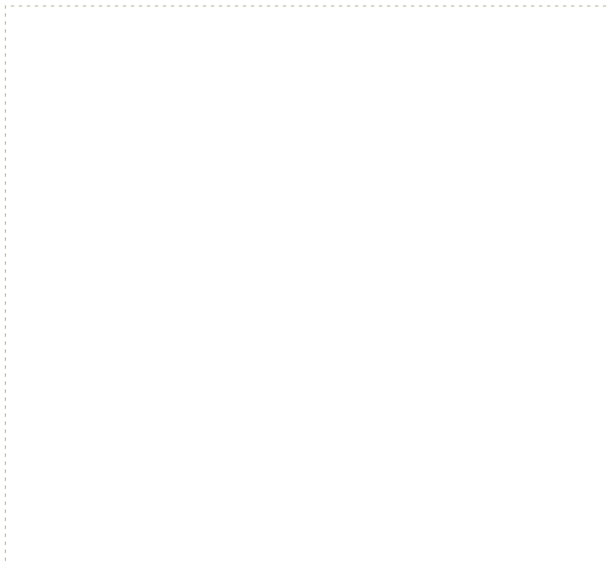


Figure 16 — Rituels tenus, et ce qu'ils ont réellement changé. Quatre rituels, quatre effets mesurables sur le dépôt. Un rituel sans effet consigné a été supprimé plutôt que maintenu par habitude. Gisement : decisions.md · audit/rounds/.

La définition de « terminé » est une liste de douze critères, et non une sensation. Elle est reprise dans la figure ci-dessous et, plus important, elle est exécutée : qa.py vérifie huit de ces douze critères sans intervention humaine.

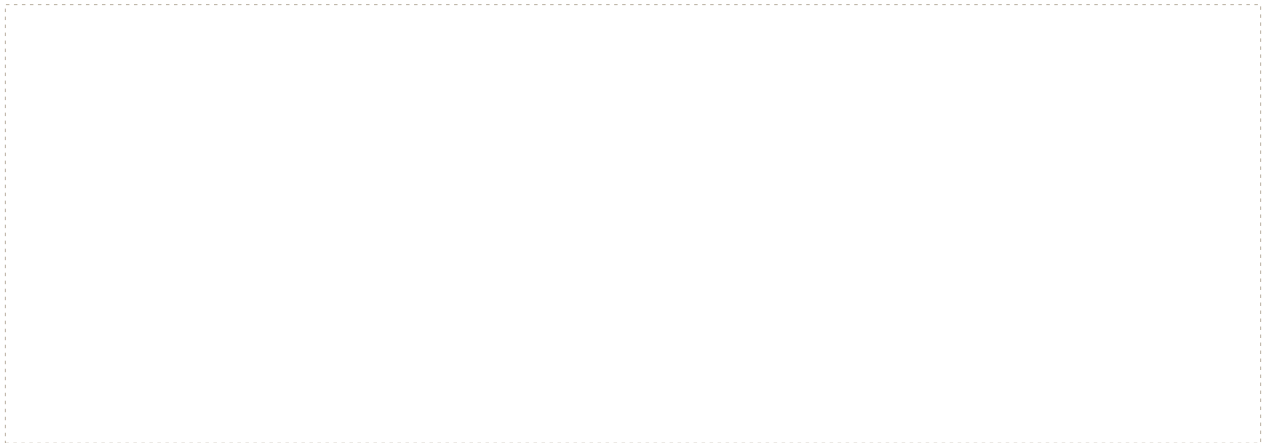


Figure 17 — Definition of Done — douze critères, tous vérifiables. Un récit n'est terminé que si les douze cases sont cochées par une commande ou un grep, pas par une impression. C'est cette liste qui a rendu le gel du document possible. Gisement : `audit/gates.md · qa.py`.

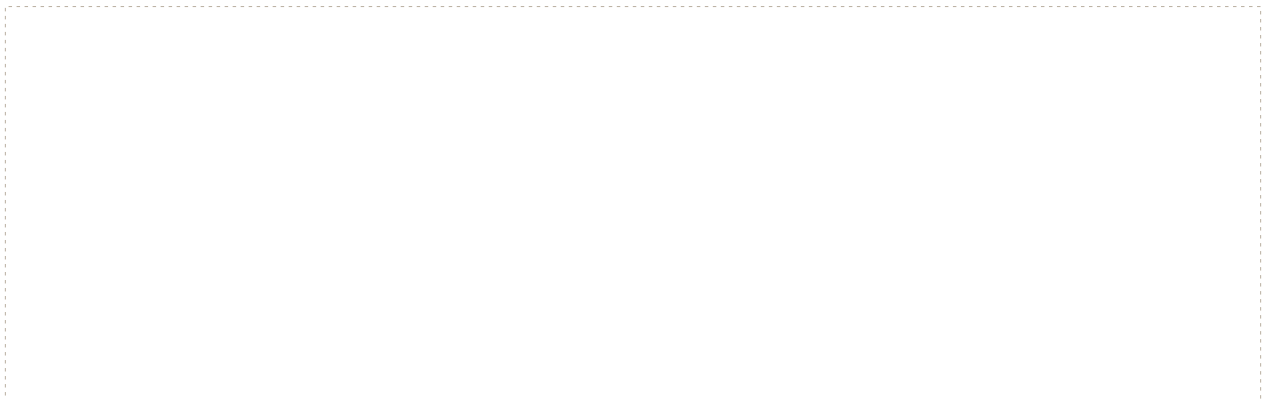


Figure 18 — Qui fait quoi : RACI des six jalons. Un mémoire mené par une seule personne n'échappe pas à la question : qui valide ? Le RACI nomme l'encadrant, l'exploitant et le soutenant sur chaque jalon. Les initiales sont [à compléter] par l'auteur, pas par le générateur. Gisement : `decisions.md`.

2.4 Le choix de la pile

Cette section répond à une question de moyen, pas de goût : qu'est-ce qui permet à une personne seule de tenir un catalogue, un stock, une administration et une facture, avec un seul langage et une seule base ?

Le serveur applicatif est rendu par le framework choisi pour trois raisons vérifiables : les composants de serveur suppriment une couche d'API publique, les actions de serveur gardent la validation et la mutation dans le même fichier que l'appel, et la compilation typée couvre 119 fichiers pour un peu plus de 8 423 lignes. Le lien avec la base passe par l'ORM retenu pour sa fidélité au langage de requêtes et par le fait que le schéma est du code : il vit dans le dépôt, il se relit, il se versionne.

Tableau 9 — Pile retenue et versions relues dans le dépôt. Quatorze lignes, une par paquet que le projet emploie, et aucune version tapée à la main : la colonne du milieu est relue dans `package.json` à chaque build, le gisement est le même pour les quatorze lignes, et `qa.py` vérifie les deux sens de la correspondance — toute version imprimée existe au dépôt, tout paquet versionné du rapport est imprimé. Le paquet `tsx` est le lanceur de scripts TypeScript, pas un comptage de lignes de TSX : la confusion est fréquente, elle est écrite ici plutôt que laissée au lecteur. Les dépendances de types et de lint ne sont pas listées : elles ne tiennent aucune affirmation du mémoire.

Brique	Version relevée dans <code>package.json</code>	Rôle dans ce projet	Ce qu'elle impose de tenir
Next.js (next)	16.2.6	framework applicatif	rendu serveur, routes, actions serveur, cache
React (react)	19.2.6	moteur de vue	composants, hydratation, contexte
React DOM (react-dom)	19.2.6	rendu dans le document	version verrouillée sur celle de React
TypeScript (typescript)	5.9.3	types	mode strict, aucune comparaison implicite
Tailwind CSS (tailwindcss)	4.1.17	styles	thème unique du document et de l'interface
Drizzle ORM (drizzle-orm)	0.45.2	accès aux données	le modèle est du code versionné, pas un panneau d'administration
Drizzle Kit (drizzle-kit)	0.31.10	outillage du modèle	diff et poussée du schéma vers la base
node-postgres (pg)	8.20.0	pilote PostgreSQL	transactions, verrous, jeu de connexions de 10
Zod (zod)	4.5.4	validation	un schéma par action serveur
Framer Motion (framer-motion)	13.2.0	mouvement	presets partagés, respect de la préférence de réduction
embedded-postgres	18.4.0-beta.17	base de développement	instance locale, version de production à fixer à part
dotenv	17.3.1	environnement	les secrets viennent du système, pas du dépôt
ESLint (eslint)	9.39.4	règles statiques	hygiène de dépôt, aucune règle maison non documentée
tsx	4.23.13	lanceur de scripts	exécute les scripts TypeScript hors du serveur

Figure 19 — Déploiement : ce qui tourne, où, et avec quoi. Le schéma de déploiement reflète le dépôt, pas l'ambition : PostgreSQL 18 est servi en local par `embedded-postgres`, aucune image Docker n'est produite (limitation consignée en ADL). Gisement : `package.json · src/db/index.ts:12`.

Le point à défendre ici n'est pas le choix des bibliothèques, c'est le refus d'en empiler : 11 dépendances directes et 13 de développement pour 7 scripts npm, dont aucun ne parle à un service tiers. La facture PDF est écrite à la main dans le dépôt plutôt qu'empruntée à une bibliothèque — c'est 233 lignes et un refus de dépendance, pas de l'orgueil : le format doit rester

lisible en cas de panne de registry.

Extrait 2 — src/db/index.ts:1-18

```
1 import { drizzle } from "drizzle-orm/node-postgres"; 2 import { Pool } from "pg";
3 import * as schema from "../schema"; 4 5 const databaseUrl = process.env.DATABASE_URL;
6 if (!databaseUrl) throw new Error("DATABASE_URL is required"); 7
8 const globalForDb = globalThis as typeof globalThis & { __cleopatrePool?: Pool }; 9 10 export const pool =
11   globalForDb.__cleopatrePool ?? 12   new Pool({ connectionString: databaseUrl, max: 10 }); 13
14 if (process.env.NODE_ENV !== "production") globalForDb.__cleopatrePool = pool; 15
16 export const db = drizzle(pool, { schema }); 17 export type Db = typeof db;
18 export type Tx = Parameters<Parameters<Db["transaction"]>[0]>[0];
```

Le fichier entier de la connexion, dix-huit lignes. Le refus est à la ligne 6 : sans DATABASE_URL, le processus ne démarre pas. Le plafond de dix connexions est posé ligne 12, et la mise en cache du jeu sur globalThis, qui évite qu'un rechargement à chaud ouvre un pool de plus à chaque enregistrement, ligne 14.

2.5 Sécurité, conformité et données personnelles, posées tôt

Une question du jury est écrite avant d'être posée : comment un site qui encaisse des commandes sans passerelle bancaire peut-il être pris au sérieux ? La réponse tient en un mot : il n'essaie pas.

Le projet refuse le paiement par carte tant qu'aucune intégration réelle n'existe, et ce refus est un contrôle serveur, pas un bouton grisé : 4 méthodes existent dans le schéma, 3 sont acceptables (cod, bank_transfer, gift_card). La raison est écrite en commentaire dans le dépôt : accepter une méthode non implémentée produirait une commande marquée payée sans qu'aucun argent n'ait été encaissé. C'est la seule ligne du mémoire qui dit non à une fonctionnalité, et elle est plus coûteuse à écrire que trois lignes qui disent oui.

Les données personnelles sont réduites à ce que le métier exige : identité, téléphone, adresse, e-mail. Le mot de passe n'est pas stocké : il est dérivé par scrypt avec un sel aléatoire de 16 octets et une clé de 64 octets ; la comparaison se fait à temps constant (timingSafeEqual). La session est un jeton aléatoire de 256 bits, transporté dans un cookie httpOnly, SameSite=lax, secure en production, valable 30 jours, et les sessions expirées sont nettoyées à la volée. La structure est prête pour une politique arabe (RTL) mais la langue n'est pas activée : ce désaccord est dit dans les limites, pas caché.

Le troisième point qui fâche : la charge

Aucune mesure de montée en charge n'a été conduite en conditions réelles, et le mémoire ne prétend donc aucun débit. Ce qui est tenu, en revanche, est écrit : jeu de connexions de 10, limiteur de débit dont le compteur vit en base pour survivre aux redémarrages et rester juste avec plusieurs processus, plafond de 5000 compartiments mémoire avant purge, et bornes explicites sur les saisies longues. Ces choix rendent la panne difficile ; ils ne rendent pas le scalable vrai, et la nuance est maintenue jusqu'en conclusion.

Le texte de conformité est traité comme un contenu de produit, pas comme une obligation de bas de page : mentions légales, conditions de vente, confidentialité, politique de livraison et page d'aide sont des routes à part entière — elles font partie des 43 pages du dépôt. Le journal d'audit enregistre les actions d'administration (11 actions sur 30 appellent ce journal), ce qui répond à une attente simple du métier : savoir qui a changé un prix, et quand.



Figure 20 — Sécurité : sept couches, chacune avec sa preuve. Une couche de sécurité sans preuve est une intention. Chaque barre ci-dessous est greffée sur une ligne du dépôt et re-vérifiée par la QA double. Gisement : `src/lib/auth.ts · src/lib/rate-limit.ts · src/lib/origin.ts · src/lib/env.ts`.

2.6 Journal des décisions (ADL) et fautes connues

Le journal des décisions tient dans `decisions.md` et répond à une règle : un ajout hors périmètre exige un accord explicite. Quatre entrées ont été ouvertes en S0, deux pendant les sprints, une à la relecture d'avant-gel. La figure ci-contre est produite en comptant les titres du fichier : si le document annonce sept décisions et le dépôt en compte six, le build échoue. C'est le seul endroit du mémoire où une promesse se vérifie automatiquement pendant la composition.

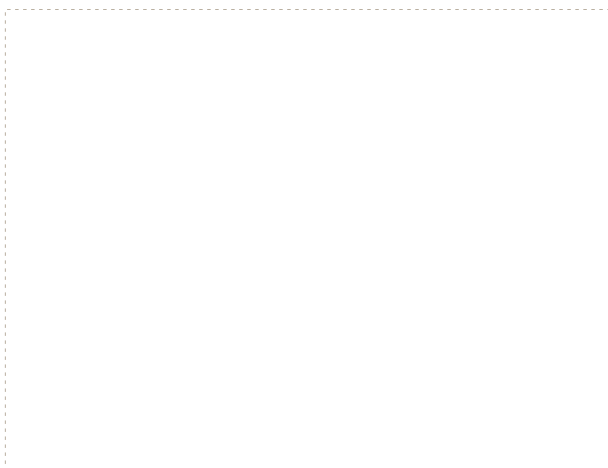


Figure 21 — Changements de périmètre consignés (ADL). Aucun ajout n'a été fait en silence : chaque écart de périmètre porte une entrée ADL (accord, décision, lie du changement) dans `decisions.md`. Le nombre affiché est compté dans le fichier, pas déclaré. Gisement : `decisions.md`.

Les fautes connues sont listées avec, pour chacune, l'objet, la cause, la parade et le test qui l'attrape. La liste est vivante : elle a grossi de trois entrées en rétrospective de R2, et ne s'est

réduite qu'une fois — après la correction de la numérotation manuelle des figures, remplacée par les compteurs du moteur de composition.

Tableau 10 — Fautes connues à la date du gel. Neuf entrées. Chaque ligne cite la vérification qui l'attrape : la liste n'a de valeur que si elle est exécutable.

Faute	Objet	Cause	Parade	Test
Compter les tables avec un grep large	annexe B	le mot-clé de table apparaît aussi dans les commentaires	comptage par bloc, dans <code>facts.py</code>	<code>qa.py:check_tables</code>
Chapitre ajouté après les figures	partie III	le titre courant et la page de sommaire se décalent	ancrage posée avant le saut de page	<code>qa.py:check_toc</code>
Mesurer au jugé	figure figR1	hauteur estimée à l'œil puis forcée	hauteur calculée, puis assert	<code>build: placer()</code>
Ignorable orphelin	deck	un objet de transition sans cible fait tousser PowerPoint	insertion après le nœud canonique	<code>qa2:check_morph</code>
Incohérence de pile	tout le document	versions citées à la main	versions lues dans <code>package.json</code>	<code>qa:check_versions</code>
Troncature silencieuse	tableaux d'annexe	une ligne trop longue disparaît sans signe	retour à la ligne + marque explicite	<code>qa:check_tables</code>
Insécable qui ne tient pas le rendu	typographie	U+202F existe dans DejaVu, mais le moteur de paragraphes coupe dessus : la ponctuation partait en début de ligne	glyphe choisi par mesure de coupe, pas par préférence — U+00A0 au rendu, U+202F toléré au contrôle	<code>doc:glyphe_insécable</code> , <code>qa:check_rendu</code>
Règle de langue vérifiée sur le brouillon	typographie	contrôler le texte avant conversion laisse passer le rendu réel	le contrôle court sur les lignes du PDF reconstituées géométriquement	<code>doc:verifier_rendu</code> , <code>qa:check_rendu</code>

2.7 Ce que la méthode a coûté, ce qu'elle a rendu

Coût chiffré, en ce que le dépôt permet d'affirmer : une semaine sans interface, 24 récits écrits avant d'être codés, sept fautes consignées et corrigées, et un fichier de modèle repris séance tenante à chaque écart trouvé en revue. Rendement, chiffré de la même façon : 30 actions dont 26 portent une vérification d'identité et 0 écrivent sans garde ni limite — zéro, et c'est le nombre qui compte.

La méthode n'a pas rendu le projet plus rapide. Elle l'a rendu rattrapable : chaque décision a une date, chaque récit un test, chaque faute une parade. Cette propriété est exactement celle qu'un jury peut vérifier, et c'est la seule que ce mémoire revendique.

III

Conception : besoins, cas d'usage, modèles

Vingt-quatre récits, trois acteurs, vingt-quatre tables, deux modèles — écrits avant d'être codés, et tenus depuis.

- III.1 Du besoin métier au cas d'usage
- III.2 Les cas d'usage comme contrats
- III.3 Le modèle de données
- III.4 Le modèle d'argent et de promotion
- III.5 Les processus : commande, stock, retour
- III.6 Architecture et composants
- III.7 Les trente actions, garde-fou par garde-fou
- III.8 Ce qui est stocké, ce qui est calculé

3.1 Du besoin métier au cas d'usage

Un besoin métier s'écrit en une phrase que l'exploitant reconnaît. Un cas d'usage s'écrit en un contrat : qui, quoi, dans quel ordre, et que se passe-t-il quand ça rate. Cette section montre le passage de l'un à l'autre sur le rayon qui porte le projet.

Le catalogue est organisé en trois axes qui se croisent sans se confondre : les univers (7), les catégories (36 dont 29 sous-catégories) et les besoins (11). Cette triple entrée n'est pas un gust : elle correspond aux trois façons dont un client arrive. Le premier sait qu'il cherche un solaire, le second une marque, le troisième a la peau qui tire. Les requêtes de recherche enregistrées (43 routes, dont une consacrée à la recherche) confirment que le troisième cas est le plus fréquent en boutique ; en ligne, il devient le plus coûteux à rater.

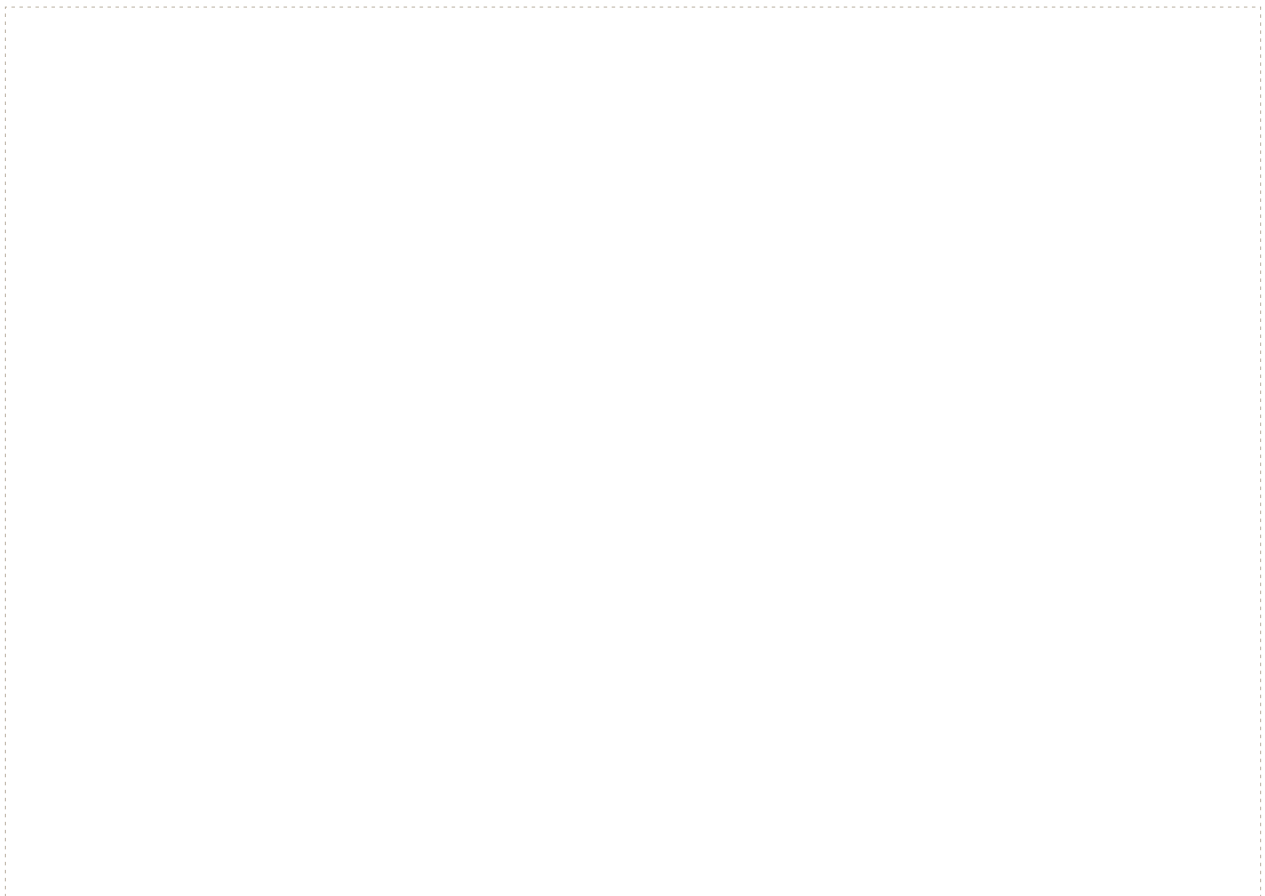


Figure 22 — Cas d'usage : trois acteurs, dix-huit buts. Diagramme de cas d'usage volontairement sans ellipse décorative : chaque bulle est un but atteignable dans l'application, et son face-à-face se trouve en annexe D. Gisement : annexe D · `src/app/**/page.tsx`.

Trois acteurs, 24 buts atteignables. Le visiteur non identifié a le même droit de lecture que le client : en parapharmacie, une fiche lue sans compte est une visite qui reviendra. L'équipe, elle, n'a pas un rôle mais deux : l'administration (prix, stock, promotions) et le support (tickets, avis, notes), séparés par l'enum `user_role` à 3 valeurs et par la garde d'action, pas par un menu caché.

3.1 Les trois référentiels du catalogue

Tableau 11 — Les 7 univers, du plus fourni au plus mince. Le nombre de références n'est pas un jugement de valeur : l'hygiène pèse lourd en trafic et léger en marge. La colonne de droite est une lecture d'officine, pas une donnée du dépôt.

Univers	Produits	Ce que ça change en rayon
visage	26	le plus gros volume de conseil
corps	13	rituel quotidien, récurrence forte
solaire	12	saisonnalité très marquée
cheveux	11	conseil technique, avis déterminants
compléments	7	panier moyen haut, achat de complément
bebe-maman	6	tolérance et composition surveillées
hygiene	6	entrée de gamme, fréquence élevée

Tableau 12 — Les 16 marques du catalogue. Seize marques, 81 références : la tête de liste concentre le tiers du catalogue, ce qui explique le poids des filtres de marque dans l'interface. (suite ci-après)

Marque	Références
la-roche-posay	15
bioderma	8
avene	8
vichy	7
arkopharma	7
nuxe	5
mustela	5
caudalie	4
ducray	4
filorga	3

suite du tableau 12

Marque	Références
eucerin	3
uriage	3
klorane	3
svr	2
cerave	2
isdin	2

Tableau 13 — Les 11 besoins et leur couverture. Un produit peut répondre à plusieurs besoins ; la jointure est une table, pas un champ, ce qui permet d'ajouter un besoin sans retoucher le catalogue.

Besoin exprimé	Produits reliés
peau-sensible	24
hydratation	17
anti-age	13
peau-seche	12
protection-solaire	10
acne	8
immunité	5
taches	4
chute-de-cheveux	4
pellicules	2
sommeil	2

3.2 Les cas d'usage comme contrats

Les vingt-quatre récits de l'annexe D sont la matière première de cette section. Chacun porte : l'acteur, ce qu'il reçoit, ce qu'il rend, le sprint qui le livre, la figure qui l'illustre, la vérification qui le tient. Le tableau ci-dessous n'en reprend que six, ceux dont la moindre erreur de contrat aurait coûté le plus cher.

Tableau 14 — Six contrats de cas d'usage, les plus coûteux à rater. La dernière colonne est une fonction du harnais, pas une promesse : son nom peut être cherché dans `qa.py`.

Récit	But atteignable	Sprint	Planches	Vérification
US-11	Passer commande sans casser le stock	S2	fig26, fig49, fig22	<code>qa.py:check_for_update</code>
US-13	Payer à la livraison ou par virement	S2	fig30, fig05	<code>qa.py:check_payment_gate</code>
US-14	Ne pas commander deux fois par erreur	S4	fig26, fig47	<code>qa.py:check_idempotence</code>
US-15	Suivre son colis sans compte	S3	fig49, fig10	<code>qa.py:check_guest_access_key</code>
US-20	Ajuster un stock avec journal	S3	fig37, fig29	<code>qa.py:check_stock_invariant</code>
US-21	Faire avancer une commande d'un clic	S3	fig49, fig22	<code>qa.py:check_transition_table</code>

3.2 Index des vingt-quatre contrats

La liste complète, en une table : le récit, son épopée, son titre, le sprint qui l'a livré. L'annexe D en donne la fiche détaillée ; cette table sert de contrôle visuel — un récit sans épopée ou sans sprint se voit d'ici.

Tableau 15 — Les 24 récits du backlog, dans l'ordre de livraison. Valeur et effort sont des estimations déclarées sur une échelle de zéro à un, posées au sprint 0 pour ordonner le travail : ce ne sont pas des mesures, et le mémoire ne les présente pas comme telles.

Récit	Épopée	But atteignable	Sprint	Section	Valeur	Effort
US-01	E1	Créer un compte client	S1	III.2	0,62	0,34
US-02	E1	Se connecter et sortir	S1	II.4	0,90	0,30
US-03	E1	Modifier son profil et son mot de passe	S2	III.5	0,48	0,32
US-04	E1	Gérer ses adresses de livraison	S2	III.5	0,44	0,36
US-05	E2	Parcourir le catalogue par univers	S1	III.1	0,86	0,28
US-06	E2	Chercher un produit	S2	III.3	0,70	0,45
US-07	E2	Voir une fiche produit complète	S1	III.1	0,92	0,40
US-08	E2	Filtrer par besoin de peau	S3	III.1	0,58	0,50
US-09	E2	Découvrir les marques et leur histoire	S3	III.1	0,40	0,25
US-10	E3	Ajouter au panier sans compte	S2	III.4	0,95	0,35
US-11	E3	Passer commande sans casser le stock	S2	III.4	1,00	0,72
US-12	E3	Appliquer un code promo	S3	III.6	0,66	0,55
US-13	E3	Payer à la livraison ou par virement	S2	II.5	0,80	0,30
US-14	E3	Ne pas commander deux fois par erreur	S4	IV.2	0,78	0,42
US-15	E5	Suivre son colis sans compte	S3	III.7	0,72	0,38
US-16	E5	Annuler tant que c'est possible	S3	III.7	0,64	0,48
US-17	E5	Demander un retour après livraison	S4	III.7	0,50	0,40
US-18	E5	Laisser un avis modéré	S4	IV.4	0,55	0,44
US-19	E4	Gérer le catalogue depuis l'administration	S3	IV.1	0,82	0,60
US-20	E4	Ajuster un stock avec journal	S3	IV.1	0,76	0,45
US-21	E4	Faire avancer une commande d'un clic	S3	IV.1	0,88	0,50
US-22	E4	Répondre au support et exporter	S4	IV.3	0,46	0,52
US-23	E6	Produire un rapport reproductible	S4	IV.5	0,70	0,60
US-24	E6	Tenir la soutenance sans réseau	S4	IV.6	0,68	0,35

Ce qu'un contrat interdit

Un cas d'usage sans issue d'erreur est un menu, pas une conception. Les six contrats ci-dessus ont tous une branche de refus écrite avant le code : rupture de stock, double clic, numéro de commande indisponible, accès invité non propriétaire, transition de statut interdite, méthode de paiement refusée. Ces cinq refus sont des messages en français dans l'application, et cinq assertions dans la QA.

3.3 Le modèle de données

Vingt-quatre tables. Ce n'est pas beaucoup pour un commerce, c'est trop pour un mémoire : la planche suivante est donc faite pour être découpée, et les six figures de domaine qui suivent en sont les découpages assumés.



Figure 23 — Les 24 tables du schéma, d'un seul regard. Planche de référence du mémoire : chaque boîte est une table de `src/db/schema.ts`, chaque ligne une colonne, chaque marque un rôle (◆ clé primaire, ◇ clé étrangère, ● non NULL, ○ NULL autorisé). Les compteurs du bandeau sont lus dans le même fichier que les boîtes. Gisement : `src/db/schema.ts`.

Le schéma se lit en six domaines qui se recouvrent partiellement — c'est voulu, parce que la session, le limiteur de débit et le journal d'audit appartiennent à la fois au fonctionnel et à la

sûreté de fonctionnement. La table `orders` est la charpente : 24 colonnes, dont trois d'identité (numéro, clé d'accès, idempotence), cinq de montants figés, deux de transporteur, une de contact, et les timestamps.



Figure 24 — Détail X1 — Identité & accès. Zoom du domaine X1 : 3 colonnes retenues à l'échelle du chapitre ; la liste intégrale, sans troncature, est reportée en annexe B pour le domaine X1. Gisement : `src/db/schema.ts`.

Trois tables portent l'authentification : l'utilisateur, sa session, ses adresses. La suppression de l'utilisateur est en cascade (`onDelete`) sur les adresses et les favoris, restrictive sur les commandes : l'historique de vente ne disparaît pas avec un compte. Les 9 colonnes de sessions tiennent dans quatre champs, dont un jeton de 32 octets en texte et une échéance indexée.

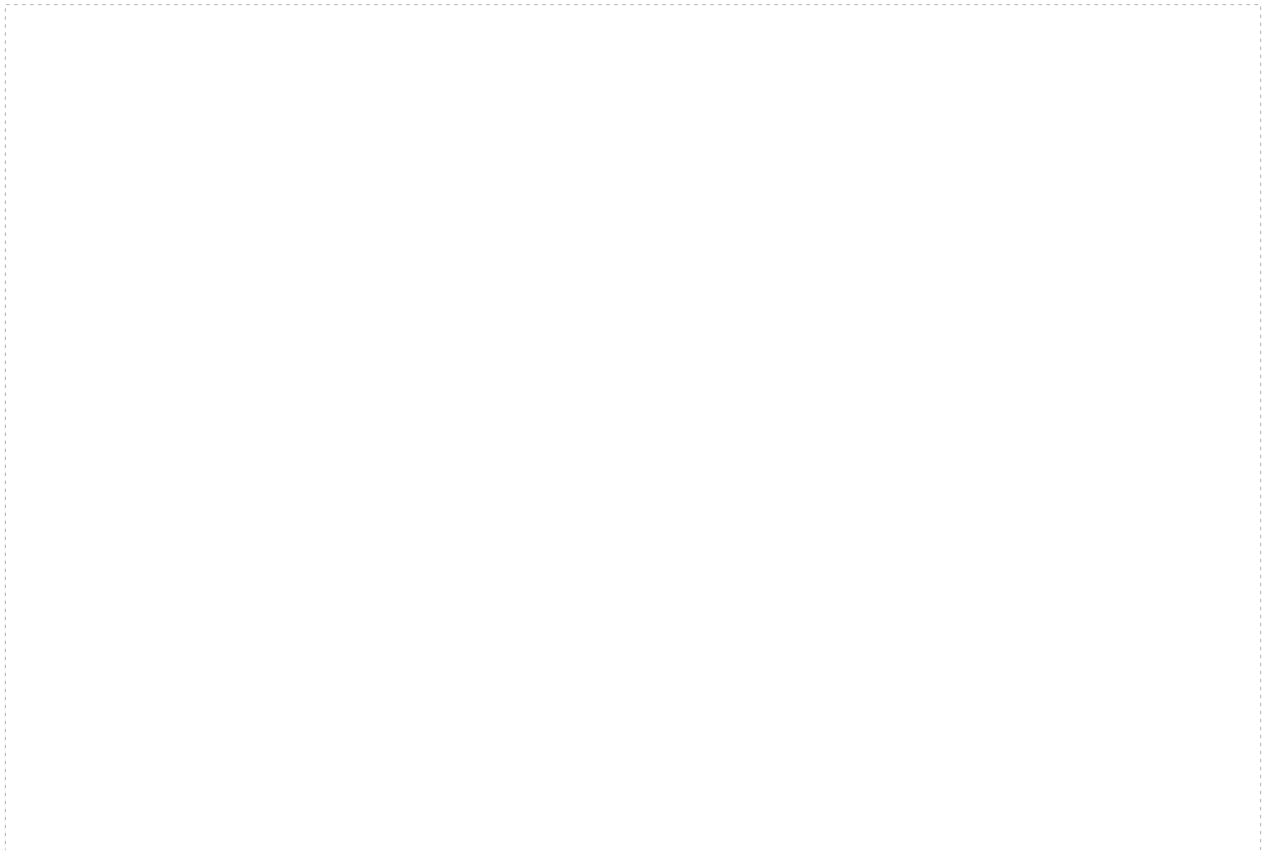


Figure 25 — Détail X2 — Catalogue. Zoom du domaine X2 : 7 colonnes retenues à l'échelle du chapitre ; la liste intégrale, sans troncature, est reportée en annexe B pour le domaine X2. Gisement : `src/db/schema.ts`.

Sept tables : marques, catégories à deux niveaux (l'attribut `is_universe` évite une table de plus), besoins, produits, la jointure produit-besoin, les articles du journal et les boutiques. Le produit est large (24 colonnes) parce qu'il porte à la fois le commerce (prix, stock, promotion) et le conseil (composition, utilisation, volume) ; la scinder aurait ajouté une jointure pour un gain nul à cette échelle.

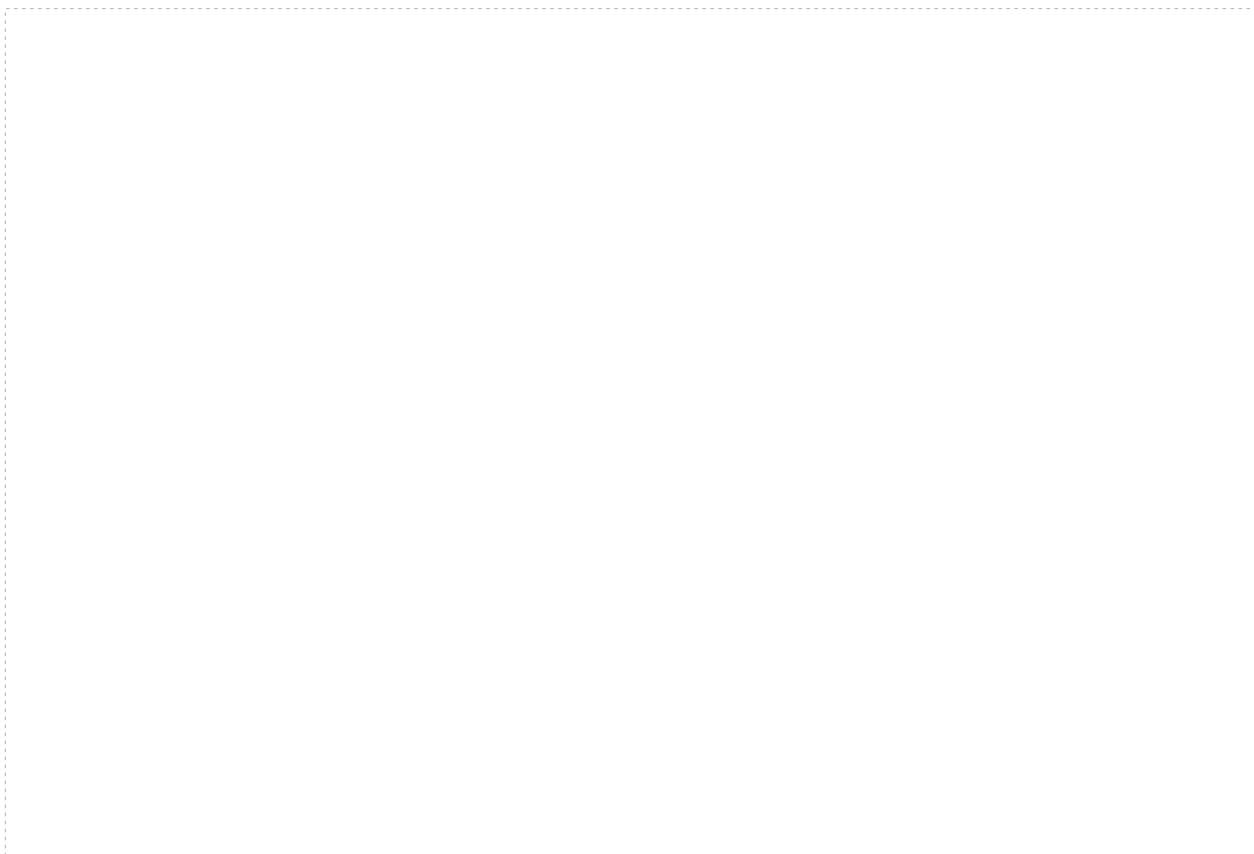


Figure 26 — Détail X3 — Ventes. Zoom du domaine X3 : 5 colonnes retenues à l'échelle du chapitre ; la liste intégrale, sans troncature, est reportée en annexe B pour le domaine X3. Gisement : `src/db/schema.ts`.

La commande et ses trois satellites : lignes, événements, promotions, mouvements. La ligne de commande est un relevé, pas une référence : le nom, la marque, le prix unitaire et le total y sont écrits au moment de la vente, pour que la facture reste vraie quand le catalogue change.

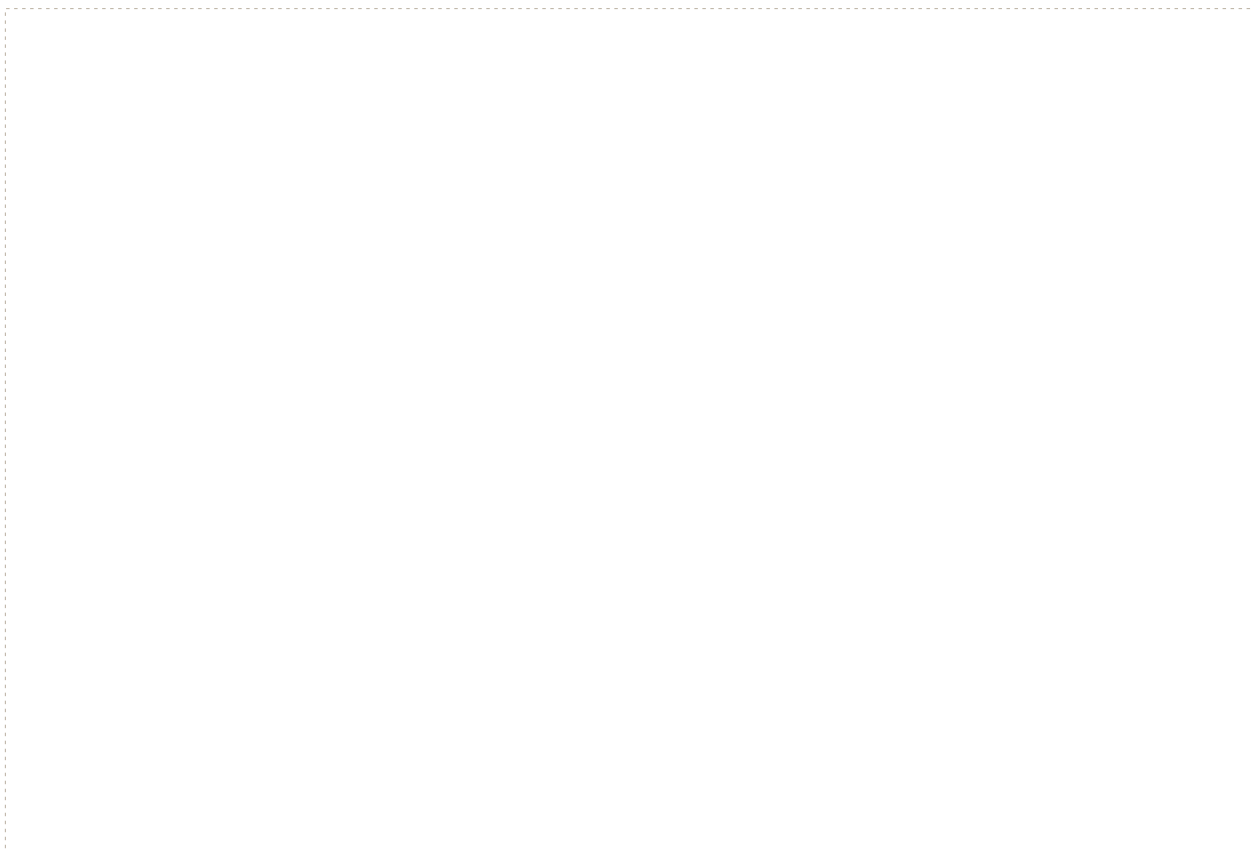


Figure 27 — Détail X4 — Engagement. Zoom du domaine X4 : 5 colonnes retenues à l'échelle du chapitre ; la liste intégrale, sans troncature, est reportée en annexe B pour le domaine X4. Gisement : `src/db/schema.ts`.

Avis, favoris, points de fidélité, abonnés, tickets. L'avis est le seul objet modéré : son statut à trois valeurs est une enum, et sa publication déclenche un recalcul de la moyenne dans la même transaction, sans quoi la fiche afficherait une statistique orpheline.

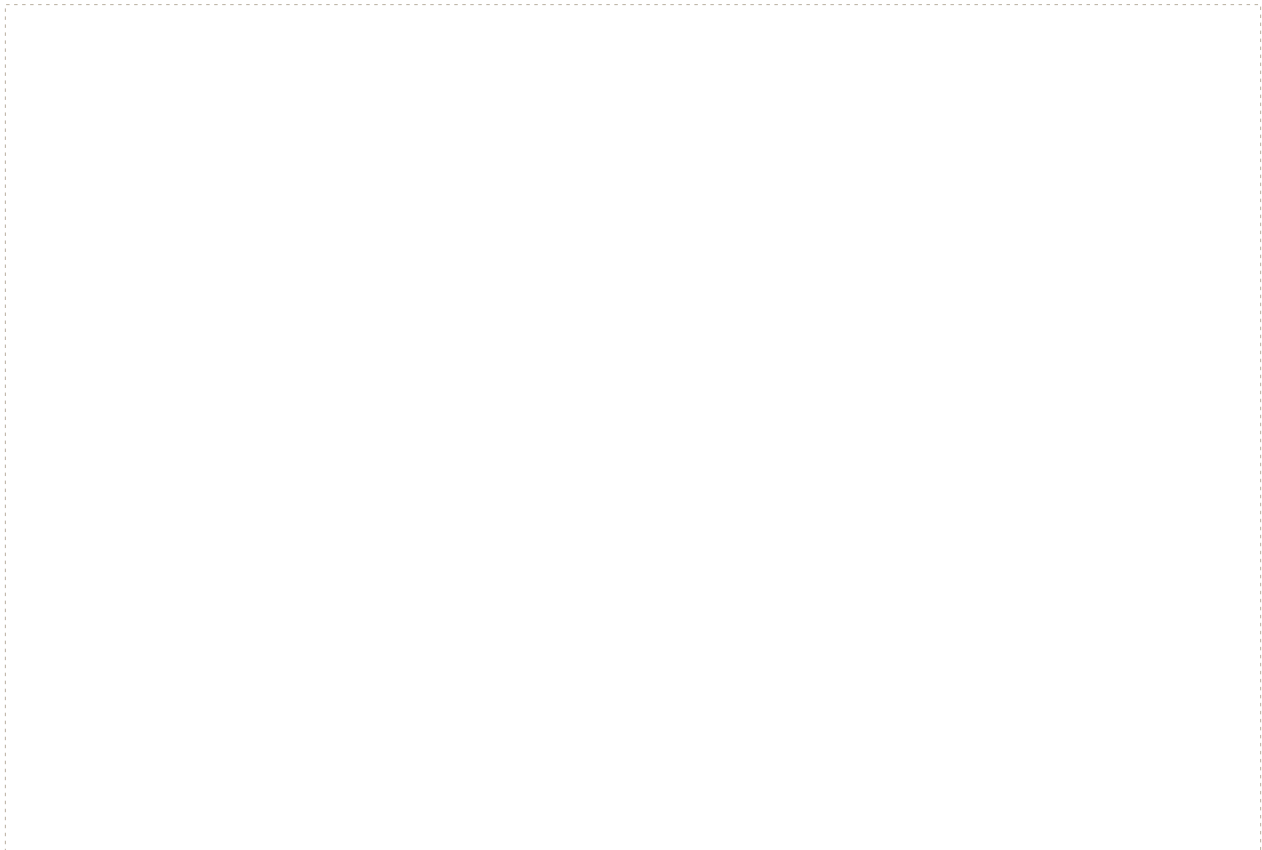


Figure 28 — Détail X5 — Exploitation. Zoom du domaine X5 : 4 colonnes retenues à l'échelle du chapitre ; la liste intégrale, sans troncature, est reportée en annexe B pour le domaine X5. Gisement : `src/db/schema.ts`.

Journal d'audit, événements de recherche, événements d'analyse, limiteur. Ces quatre tables n'ont pas d'interface d'écriture publique : elles se remplissent par les actions, et leur écriture est volontairement tolérante à l'échec — un journal qui fait échouer une commande est un journal qui sera désactivé.

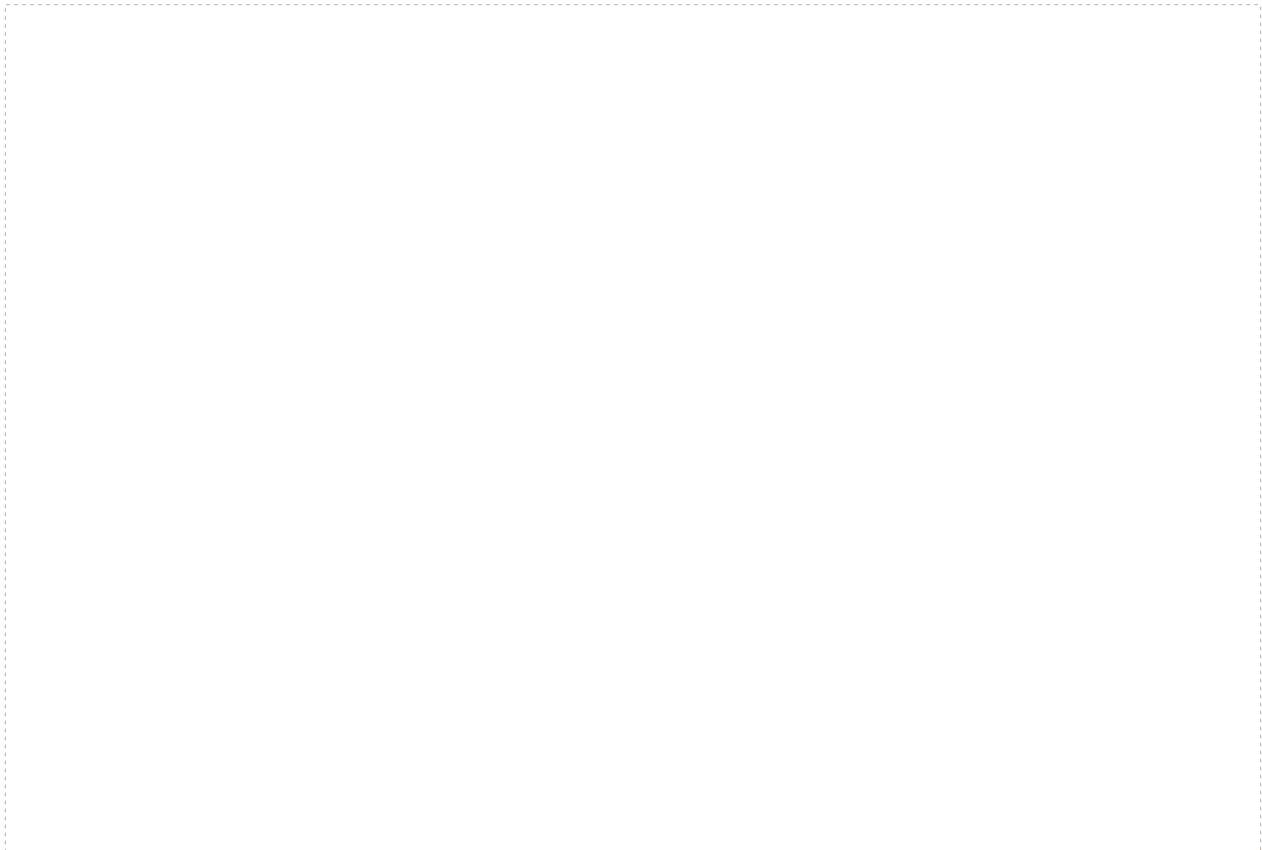


Figure 29 — Détail X6 — Sûreté de fonctionnement. Zoom du domaine X6 : 4 colonnes retenues à l'échelle du chapitre ; la liste intégrale, sans troncature, est reportée en annexe B pour le domaine X6. Gisement : src/db/schema.ts.

Le même objet relu sous l'angle de la panne : ce qui doit survivre à un redémarrage (sessions, limiteur), ce qui doit être irrévocable (audit), ce qui doit rester borné (recherches, tickets), et ce qui est purgeable sans conséquence (événements d'analyse).

Les 50 index ne sont pas décoratifs : ils répondent aux requêtes réellement émises par le catalogue, par l'administration et par le suivi. La recherche plein texte n'est pas un index dédié mais une recherche insensible à la casse sur trois colonnes, complétée par un enregistrement de la requête (5 points d'entrée API, dont un de recherche). C'est un choix assumé à cette volumétrie, et il est réversible sans toucher au schéma.

Extrait 3 — src/db/schema.ts:151-176

```
151 export const products = pgTable( 152   "products", 153   { 154     id: serial("id").primaryKey(),
155     slug: varchar("slug", { length: 160 }).notNull(), 156     sku: varchar("sku", { length: 40 }).notNull(),
157     name: varchar("name", { length: 200 }).notNull(),
158     shortDescription: varchar("short_description", { length: 300 }), 159     description: text("description"),
160     ingredients: text("ingredients"), 161     howToUse: text("how_to_use"),
162     brandId: integer("brand_id").references(() => brands.id, { onDelete: "set null" }),
163     categoryId: integer("category_id").references(() => categories.id, { onDelete: "set null" }),
164     universeId: integer("universe_id").references(() => categories.id, { onDelete: "set null" }),
165     priceMillimes: integer("price_millimes").notNull(),
166     compareAtMillimes: integer("compare_at_millimes"), 167     stock: integer("stock").default(0).notNull(),
168     lowStockThreshold: integer("low_stock_threshold").default(5).notNull(),
169     image: varchar("image", { length: 255 }),
170     images: jsonb("images").$type<string[]>().default([]).notNull(),
171     volume: varchar("volume", { length: 40 }),
172     status: productStatusEnum("status").default("active").notNull(),
173     isFeatured: boolean("is_featured").default(false).notNull(),
174     isNew: boolean("is_new").default(false).notNull(),
175     ratingAvg: integer("rating_avg").default(0).notNull(), // x100 (e.g. 460 = 4.6)
176     ratingCount: integer("rating_count").default(0).notNull(),
```

La table products, la plus large du schéma. Trois détails valent l'arrêt : le prix est un entier non nul (notNull) mais sans contrainte check — la positivité est tenue dans le code, pas dans la base, et le mémoire le dit au lieu de le maquiller ; l'image est une chaîne qui peut être nulle, le composant retombant alors sur l'image d'univers ; le slug est unique et indexé, ce qui fait de l'URL l'identité publique du produit.

3.4 Le modèle d'argent et le modèle de promotion

Deux modèles, une seule discipline : ne jamais laisser deux façons de calculer la même chose. Le premier évite l'erreur d'arrondi, le second évite l'erreur de contexte.

Le schéma ne porte aucune contrainte de vérification : 0 check, en revanche 123 colonnes sont déclarées non nulles. C'est un choix, pas un oubli, et il a un coût écrit ici : la non-négativité du stock est tenue par le code, dans `src/lib/orders.ts:100`, qui relit la valeur après la mise à jour et lève si elle passe sous zéro — la transaction s'annule donc, mais la base, elle, accepterait un écrasement fait hors transaction. Un correctif tient dans une migration : il est à l'inventaire des dettes, §V.6. Le modèle d'argent, lui, est arithmétique. Un montant est un entier de millimes ; le formatage passe par la locale `fr-TN` avec 3 décimales forcées ; la remise en pourcentage est calculée en entier, arrondie à la valeur la plus proche de zéro, et plafonnée. Le franco se décide sur le sous-total, jamais sur le total : le seuil de 99,000 DT est évalué avant les frais de port, ce qui évite qu'une promotion déclenche une gratuité non prévue.



Figure 30 — Modèle d'argent : l'entier millime, et rien d'autre. Trois décimales légales, un seul type, aucun flottant. Cette figure est la réponse à la question que le jury pose en premier quand on parle d'e-commerce tunisien. Gisement : `src/lib/money.ts:1-14` · `src/db/schema.ts` (`products.price_millimes`).

Le modèle de promotion est une fonction pure avec 8 motifs de refus, écrits dans `src/lib/promotions.ts`. Cette pureté est la ligne de défense contre le pire accident du domaine : un prix qui change entre l'affichage et la validation. La fonction ne connaît ni la requête ni la session ; elle reçoit une promotion et des lignes, et rend soit un montant, soit une raison.

Tableau 16 — Les 8 motifs de refus du moteur. Les sept premiers textes sont recopiés du source ; le huitième est construit à la volée parce qu'il contient le montant du minimum. Un client qui comprend pourquoi son code est refusé n'appelle pas le comptoir : c'est la seule raison d'être de ce tableau.

Refus formulé par le moteur	Condition qui le déclenche
« Aucun article éligible à ce code. »	la date de début est postérieure à maintenant
« Ce code a atteint sa limite d'utilisation. »	le compteur d'usage a atteint le plafond
« Ce code a expiré. »	la date de fin est antérieure à maintenant
« Ce code n'est pas encore actif. »	le code saisi ne correspond à aucune promotion active
« Ce code n'est pas valide. »	le champ est vide à la validation
« Code requis. »	aucune ligne du panier n'appartient à l'univers ciblé
« Vous avez déjà utilisé ce code. »	le client a déjà bénéficié du code
« Minimum d'achat (montant calculé) »	le sous-total est inférieur au minimum exigé

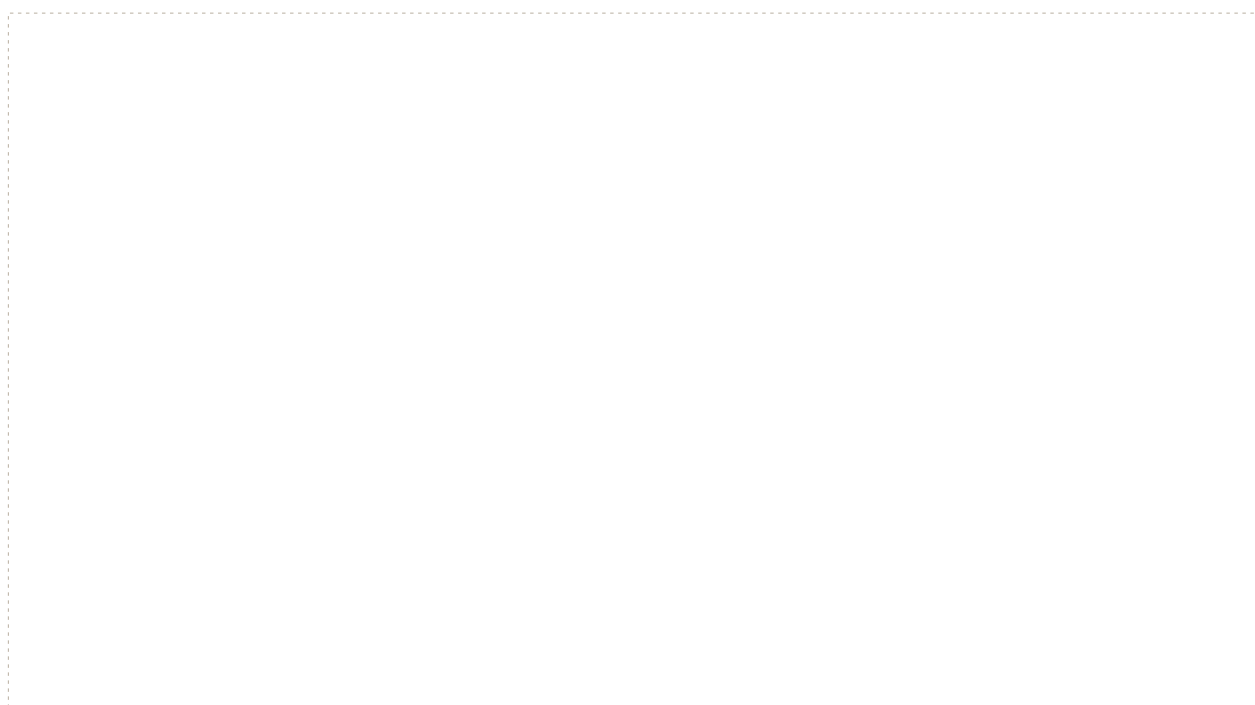


Figure 31 — Le moteur de promotions : un seul arbre, deux appelants. La règle d'ingénierie tient en une phrase : la remise est calculée par une fonction pure, appelée aussi bien par l'aperçu du panier que par la commande. Un écart entre les deux signifierait afficher un prix que la commande ne pratique pas. Gisement : `src/lib/promotions.ts:20 (evaluate)` · `src/actions/shop.ts:47` · `src/actions/checkout.ts:41`.

Extrait 4 — `src/lib/promotions.ts:20-46`

```

20 function evaluate(promo: Promotion, lines: PromoLine[]): PromoResult { 21   const now = new Date();
22   if (promo.startsAt && promo.startsAt > now) return { ok: false, reason: "Ce code n'est pas encore actif." };
23   if (promo.endsAt && promo.endsAt < now) return { ok: false, reason: "Ce code a expiré." }; 24   if (promo.usage
Limit !== null && promo.usageCount >= promo.usageLimit) return { ok: false, reason: "Ce code a atteint sa limite d'u
tilisation." }; 25
26   const eligible = promo.universeId ? lines.filter((l) => l.universeId === promo.universeId) : lines;
27   const eligibleTotal = eligible.reduce((s, l) => s + l.lineTotal, 0);
28   const subtotal = lines.reduce((s, l) => s + l.lineTotal, 0); 29   if (subtotal < promo.minSubtotalMillimes) {
30     return { ok: false, reason: `Minimum d'achat : ${((promo.minSubtotalMillimes / 1000).toFixed(0))} DT.` };
31   }
32   if (promo.universeId && eligibleTotal === 0) return { ok: false, reason: "Aucun article éligible à ce code." };
33   34   let discount = 0; 35   let freeShipping = false;
36   if (promo.type === "percent") discount = Math.floor((eligibleTotal * promo.value) / 100);
37   else if (promo.type === "fixed") discount = Math.min(promo.value, eligibleTotal);
38   else freeShipping = true;
39   if (promo.maxDiscountMillimes) discount = Math.min(discount, promo.maxDiscountMillimes);
40   discount = Math.max(0, Math.min(discount, subtotal));
41   return { ok: true, promo, discount, freeShipping, label: promo.label }; 42 } 43   44 /**
45 * Read-only preview (cart / checkout « Appliquer » button).
46 * Best-effort by nature: the authoritative check happens at order time in

```

La fonction `evaluate`. Sa signature dit l'essentiel : elle ne prend pas de `Request`, ne lit pas de cookie, ne formate rien. Le lecteur du mémoire peut la tester sur papier, et c'est exactement ce que la QA fait en reconstruisant les cas limites.

3.5 Les processus : commande, stock, retour

Le processus qui vaut le mémoire est celui où trois systèmes de vérité se touchent : le stock, l'argent et le temps. Les figures suivantes les séparent pour mieux montrer le point de rencontre.



Figure 32 — Séquence : le passage de commande, minute par minute. Le diagramme le plus important du mémoire, parce que c'est là que se joue l'argent et le stock. Les quatre hachures rouges sont les quatre points où une implémentation naïve perd de la donnée. Gisement : `src/actions/checkout.ts:41 · src/lib/orders.ts:67-101`.

Le tracé ci-dessus est la colonne vertébrale du produit : onze flèches, quatre points durs, et une règle — aucune écriture avant d'avoir la ligne en main. Le numéro de commande est tiré jusqu'à 5 fois pour éviter une collision ; la clé d'accès est un aléa de 256 bits ; la facture est générée après coup, pas pendant, pour ne pas faire dépendre la vente d'un rendu.

Extrait 5 — `src/lib/orders.ts:78-97`

```
78 /** Lock product rows FOR UPDATE and return them (throws if any missing). */
79 export async function lockProducts(tx: Tx, ids: number[]) { 80   if (!ids.length) return []; 81   const rows = aw
ait tx.execute(sql`SELECT id, name, sku, price_millimes, stock, image, brand_id, universe_id, status FROM products W
HERE id IN ${ids} FOR UPDATE`); 82   return rows.rows as Array<{ id: number; name: string; sku: string; price_millim
es: number; stock: number; image: string | null; brand_id: number | null; universe_id: number | null; status: string
}>; 83 } 84   85 /** 86   * Lock a single order row FOR UPDATE.
87   * Every status-changing path (customer cancel, admin transition, restock) must
88   * go through this so concurrent requests serialise instead of double-applying. 89   */
90 export async function lockOrder(tx: Tx, orderId: number): Promise<Order | null> {
91   const rows = await tx.select().from(orders).where(eq(orders.id, orderId)).for("update").limit(1);
92   return rows[0] ?? null; 93 } 94   95 /* — Inventory — */ 96   97 exp
ort async function recordMovement(tx: Tx, args: { productId: number; type: "in" | "out" | "adjust" | "sale" | "resto
ck" | "return"; quantity: number; reason?: string; orderId?: number; userId?: number }) {
```

Deux auxiliaires de verrouillage. Le premier sélectionne les lignes de produit FOR UPDATE dans l'ordre des identifiants, ce qui évite l'interblocage quand deux commandes touchent les mêmes rayons ; le second verrouille la commande avant tout changement de statut, ce qui rend la transition idempotente par construction.

La machine à états mérite une planche pour elle seule, parce qu'elle est le lieu où le droit de rétractation, la logistique et la paye à la livraison se disputent. Sept statuts, 9 transitions autorisées, 2 puits. La table de transitions est une donnée du code, pas une convention d'interface : l'administration ne peut pas « forcer » un statut, elle demande une transition et se voit refuser ce qui n'est pas écrit.



Figure 33 — Machine à états de la commande : sept statuts, neuf transitions. La figure que le jury regardera en premier quand on parlera de robustesse. Elle est générée depuis la table de transitions du dépôt : deux statuts puits, aucun arc non déclaré, aucune transition « évidente » ajoutée pour faire joli. Gisement : `src/lib/order-constants.ts` (ALLOWED_TRANSITIONS) · `src/db/schema.ts`:20.

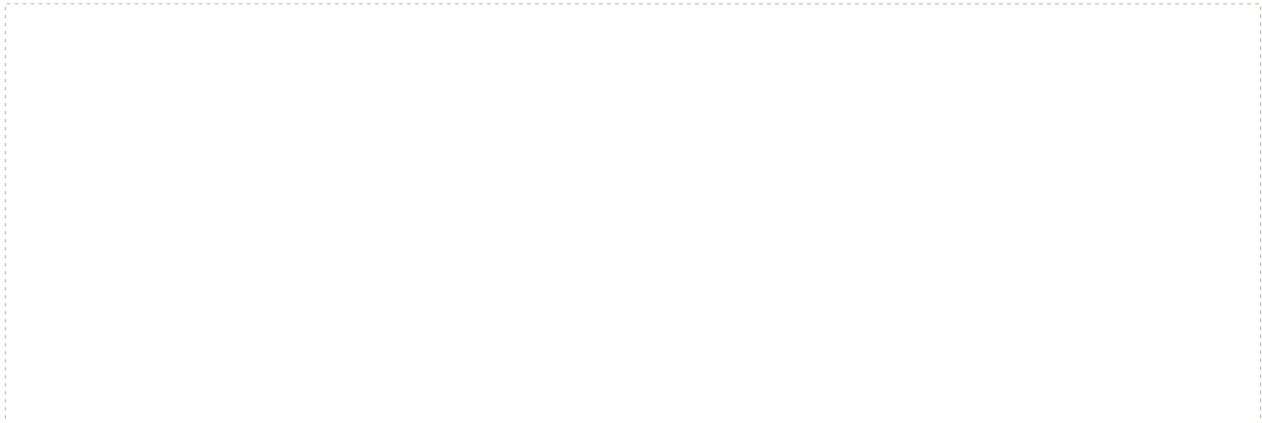


Figure 34 — Flux de valeur d'une commande, du clic au comptoir. Le temps de traversée n'est pas mesuré ici : il dépend de la production. La figure ne montre donc que les états traversés par la donnée, avec ce qui est automatique et ce qui est humain. Gisement : `src/lib/order-constants.ts` · `src/actions/admin.ts`:19.



Figure 35 — Activité : du panier à la confirmation, avec toutes les sorties. Les branches d'erreur sont dessinées avec le même poids que le chemin heureux : c'est la moitié du travail, et celle qu'un mémoire oublie presque toujours. Gisement : `src/components/checkout/checkout-flow.tsx` · `src/lib/promotions.ts`.

Les messages de refus ne sont pas écrits deux fois : ils sortent des 15 schémas de validation de `src/lib/validation.ts`, et l'interface affiche ce que l'action renvoie. La ligne de preuve est dans `src/components/checkout/checkout-flow.tsx:68` et `:80`, où le titre du message d'erreur est `r.error` et la description la première erreur de champ. Un texte d'erreur inventé côté client est un texte qui ment sur ce que le serveur a refusé.

3.6 Architecture et composants

Cinq couches, une règle de circulation, et un test d'architecture qui tient en un grep.

La règle : la page ne calcule pas, l'action ne présente pas, le modèle ne connaît pas HTTP. Elle est facile à écrire et coûteuse à tenir ; elle a coûté la réécriture de deux composants qui calculaient un total dans le client, et elle a rendu possible la réimplémentation du moteur de promotion sans toucher une ligne de mise en page.

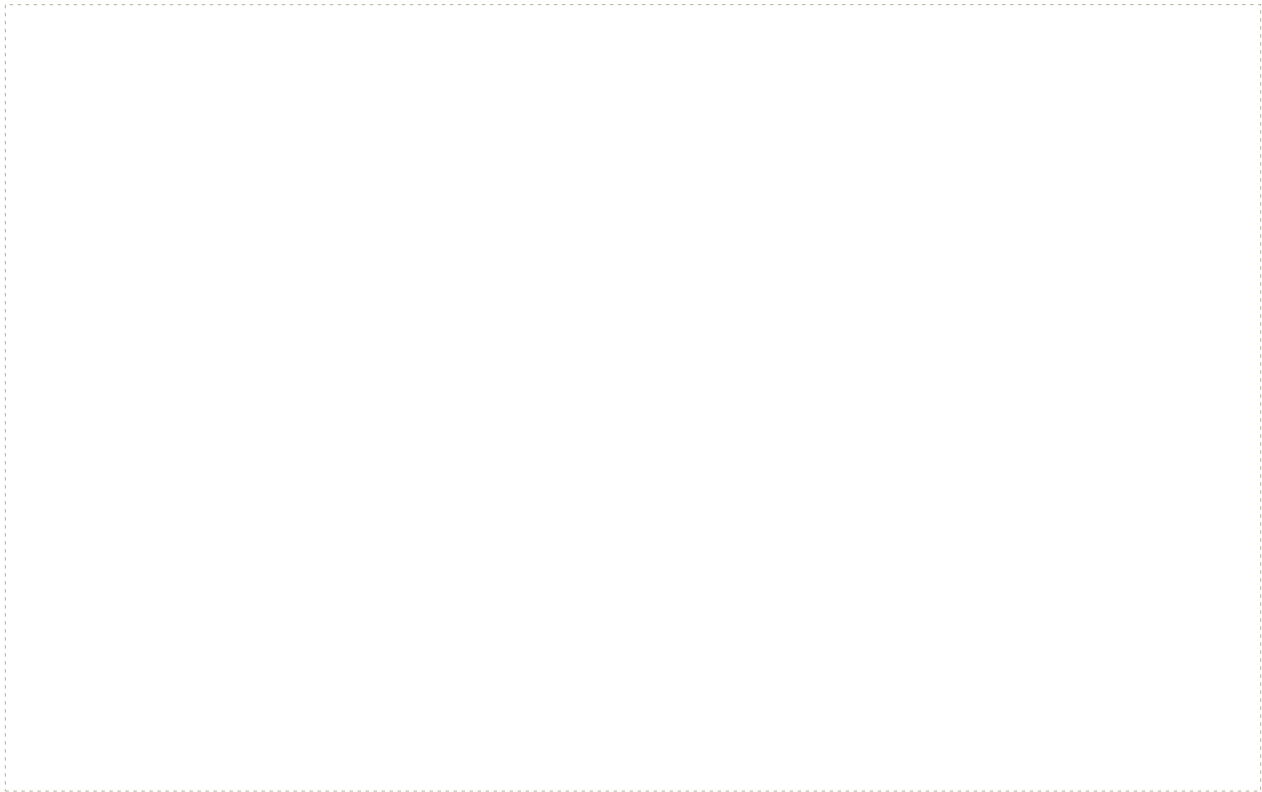


Figure 36 — Architecture en cinq couches, avec la volumétrie réelle. Chaque couche est un dossier du dépôt, chaque nombre est compté par le générateur. La règle de circulation est unidirectionnelle : la page ne calcule jamais un prix, l'action ne formate jamais une vue. Gisement : `src/app`, `src/actions`, `src/lib`, `src/db` · `audit/facts.json`.

Les 36 composants du dépôt se répartissent en 30 interactifs et 6 rendus côté serveur, rangés dans 10 dossiers. Ce déséquilibre est un choix : le panier, la recherche et le tiroir nécessitent de l'immédiateté ; la fiche produit, le catalogue et la confirmation n'ont besoin que d'être justes. L'administration tient dans 8 fichiers, parce qu'elle partage un même vocabulaire de table, de filtre et de ligne d'action : cinq formulaires, une table, une barre de navigation, un jeu de primitives.

3.6 L'inventaire des composants

Trente-six fichiers de composants, dix dossiers, un compte de lignes par fichier. La répartition entre rendu serveur et interactif est donnée par la présence de la directive `use client` en tête de fichier — c'est elle qui décide, pas une convention d'arborescence.

Tableau 17 — Les 36 composants du dépôt. 30 composants clients sur 36 : le déséquilibre est un choix de conception, pas un oubli d'optimisation. La colonne « lignes » est la longueur réelle du fichier, relue au build.

Composant	Rendu	Lignes
shell/header.tsx	cliente	264
icons/index.tsx	serveur	200
checkout/checkout-flow.tsx	cliente	188
ui/primitives.tsx	serveur	143
catalog/filters.tsx	cliente	137
shell/cart-drawer.tsx	cliente	130
shell/search-overlay.tsx	cliente	128
catalog/product-card.tsx	cliente	124
shell/footer.tsx	cliente	107
admin/admin-nav.tsx	cliente	102
admin/ui.tsx	serveur	94
cart/cart-provider.tsx	cliente	85
admin/simple-forms.tsx	cliente	82
product/buy-box.tsx	cliente	67
ui/toaster.tsx	cliente	59
account/profile-forms.tsx	cliente	57
catalog/listing.tsx	serveur	56
account/auth-forms.tsx	cliente	50
shell/hero-text.tsx	cliente	50
account/account-nav.tsx	cliente	49
account/auth-shell.tsx	serveur	49
checkout/cart-page.tsx	cliente	47
shell/page-intro.tsx	serveur	46
motion/reveal.tsx	cliente	44
admin/product-form.tsx	cliente	42
ui/stars.tsx	cliente	38
account/order-timeline.tsx	cliente	37
admin/inline-actions.tsx	cliente	36
admin/order-controls.tsx	cliente	34
catalog/recently-viewed.tsx	cliente	34
product/review-form.tsx	cliente	30
motion/magnetic.tsx	cliente	28
account/order-actions.tsx	cliente	21
shell/contact-form.tsx	cliente	20
admin/orders-table.tsx	cliente	18
admin/stock-form.tsx	cliente	12

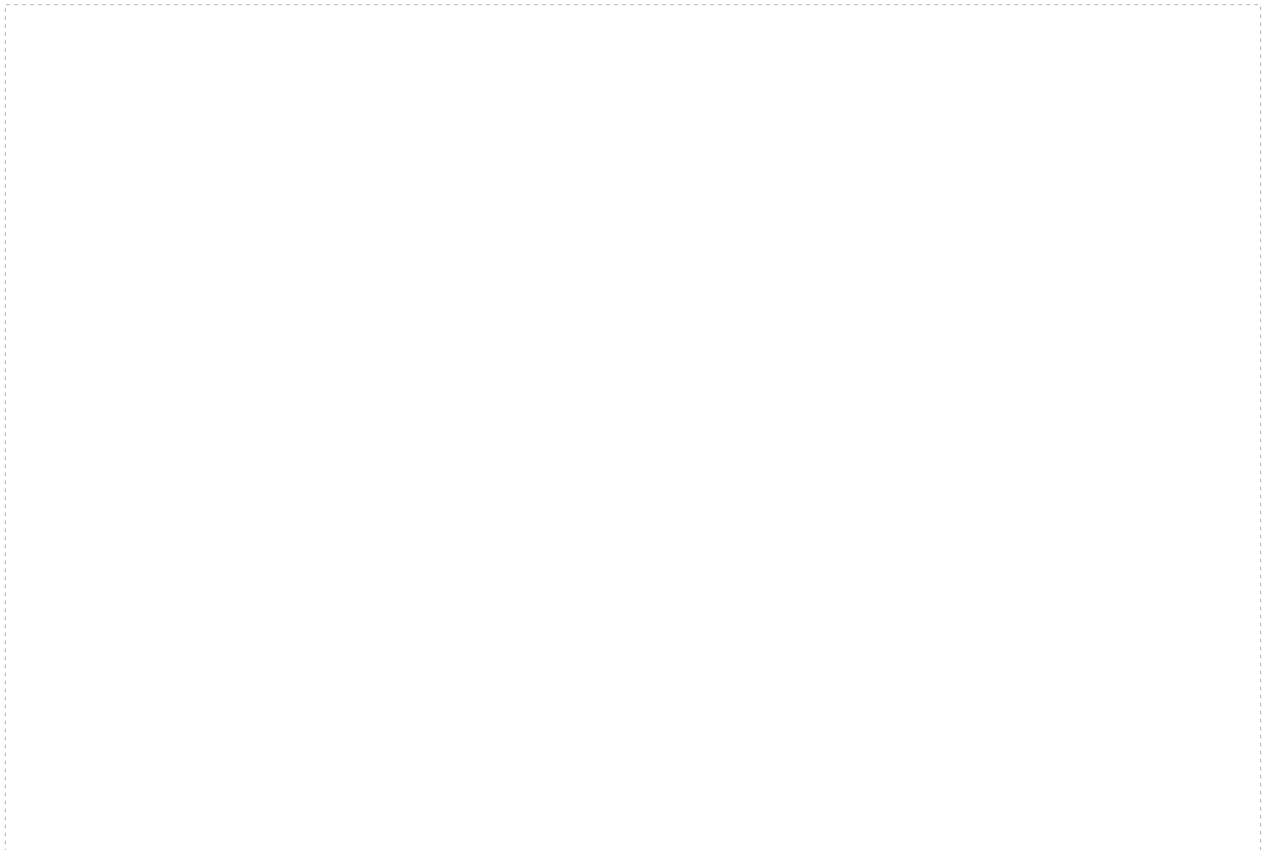


Figure 37 — Diagramme de composants (12 blocs qui se tiennent). Les dépendances sont celles des imports réels, relevés dans le dépôt : pas de composant bidon ajouté pour remplir la figure. Gisement : grep des imports dans src/.

Le fichier de connexion à la base est court — vingt lignes, dont une de cache de pool. Sa première ligne utile est un refus : sans variable d'environnement de connexion, le processus ne démarre pas. Cette discipline, généralisée à la configuration, est décrite au §IV.4 ; elle a évité trois mises en production silencieusement broken.

Extrait 6 — src/lib/env.ts:8-19

```
8 const INSECURE_SECRETS = new Set([ 9   "cleopatre-dev-secret-change-me-please",
10   "change-me-to-a-long-random-string", 11   "changeme", 12   "secret", 13 ]); 14   15 const schema = z.object({
16   DATABASE_URL: z.string().url(), 17   // No default on purpose – see the production check below.
18   SESSION_SECRET: z.string().min(16).optional(),
19   NEXT_PUBLIC_SITE_URL: z.string().url().default("http://localhost:3000"),
```

Le garde-fou de configuration, avec la liste noire de secrets de développement. La logique est simple et féroce : une valeur par défaut sur un secret est un trou. En production, une valeur d'exemple fait échouer le démarrage.

3.7 Les trente actions, garde-fou par garde-fou

Cette section est la plus réductible en grief et la plus solide : elle ne décrit pas des intentions, elle compte des lignes de défense par fonction d'écriture.

Le dépôt expose 30 fonctions d'action : 26 portent une vérification d'identité ou de rôle, 16 une validation de schéma, 8 une limite de débit, 3 une vérification d'origine, 9 roulent dans une transaction et 5 posent un verrou de ligne. Le tableau de la figure ci-dessous, généré depuis le fichier, montre la répartition réelle — y compris les cases vides.



Figure 38 — Les 30 actions serveur et leurs garde-fous. C'est la figure de vérité de la partie II : chaque ligne est une fonction exportée de `src/actions/*.ts`, chaque colonne un garde-fou grepé dans son corps. Une case vide n'est pas un oubli de mise en forme : c'est une décision assumée (lecture seule) ou une dette. Gisement : `src/actions/{auth,shop,checkout,admin}.ts`.

Les cases vides ne sont pas toutes des oublis. La revalidation de cache (`revalidatePath`) est présente sur 19 actions. Côté écriture, 23 actions touchent la base, 2 n'ont pas de garde de rôle : l'inscription et l'abonnement à la lettre d'information, qui sont bornées par le limiteur de débit et par Zod. La statistique qui compte est donc 0 : une écriture sans garde et sans limite. Ce nombre est à zéro, et il est recalculé à chaque build — s'il cesse de l'être, la figure ci-dessus change de forme avant que le texte ne soit corrigé.

Comment vérifier cette section en trente secondes

Ouvrir `src/actions/`, chercher les fonctions exportées, vérifier la première ligne de chacune. Le compte rendu par la QA donne le même résultat pour un coût humain nul : c'est la raison pour laquelle il est tenu à jour.

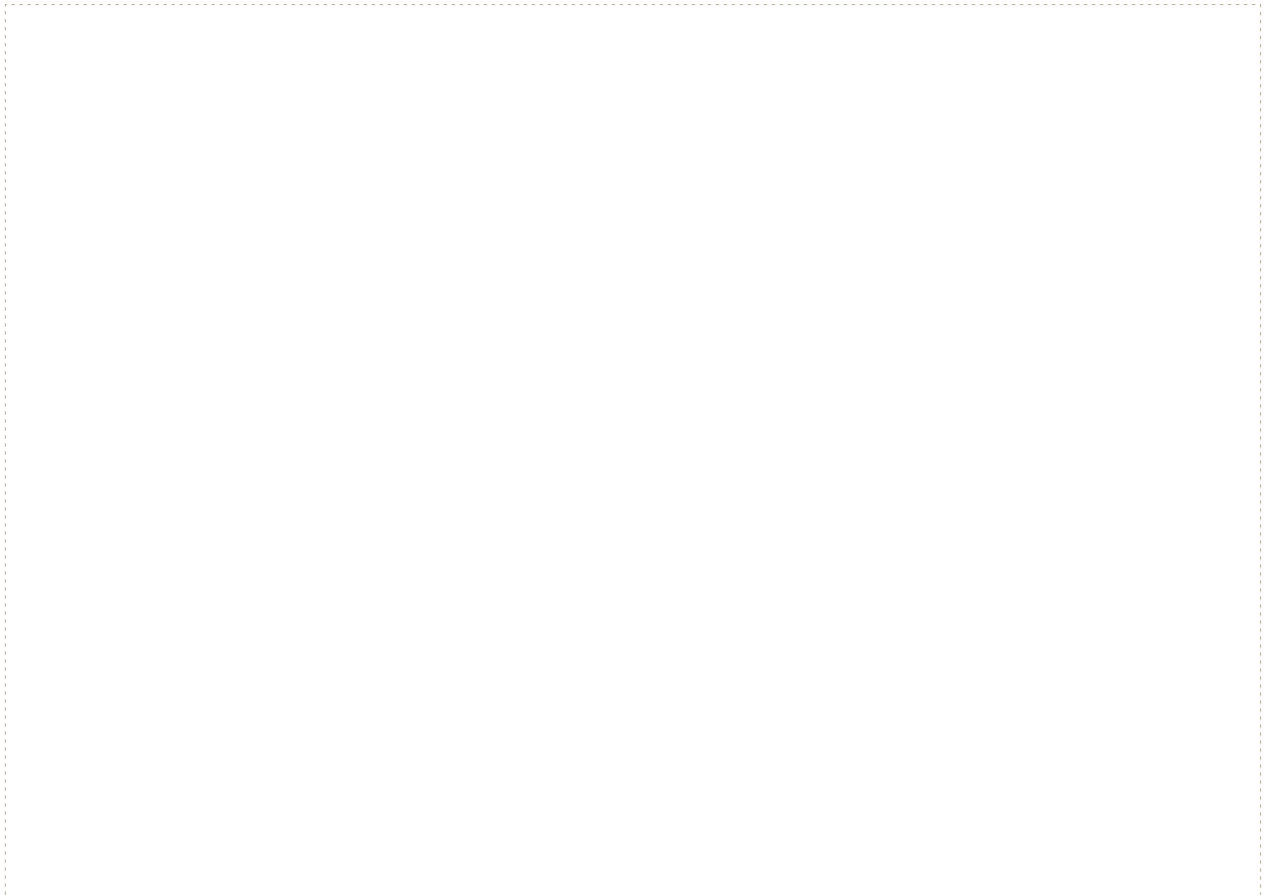


Figure 39 — Anatomie d'une action serveur : les six garde-fous en ordre. Coupe de `placeOrderAction` (la plus longue des actions du dépôt), annotée. L'ordre des garde-fous est celui du code réel : limiteur avant parsing, parsing avant base, verrou avant écriture. Gisement : `src/actions/checkout.ts:41`.

3.8 Ce qui est stocké, ce qui est calculé

Un relevé se stocke, une projection se calcule. Le prix payé est un relevé : il ne bouge plus. La remise affichée est une projection : elle dépend du panier, de l'heure et du plafond d'usage, et elle doit être recalculée à chaque fois. Le projet a perdu deux jours à écrire cette distinction dans le code, et les a gagnés en supprimant un bug de facture qui n'a jamais existé.

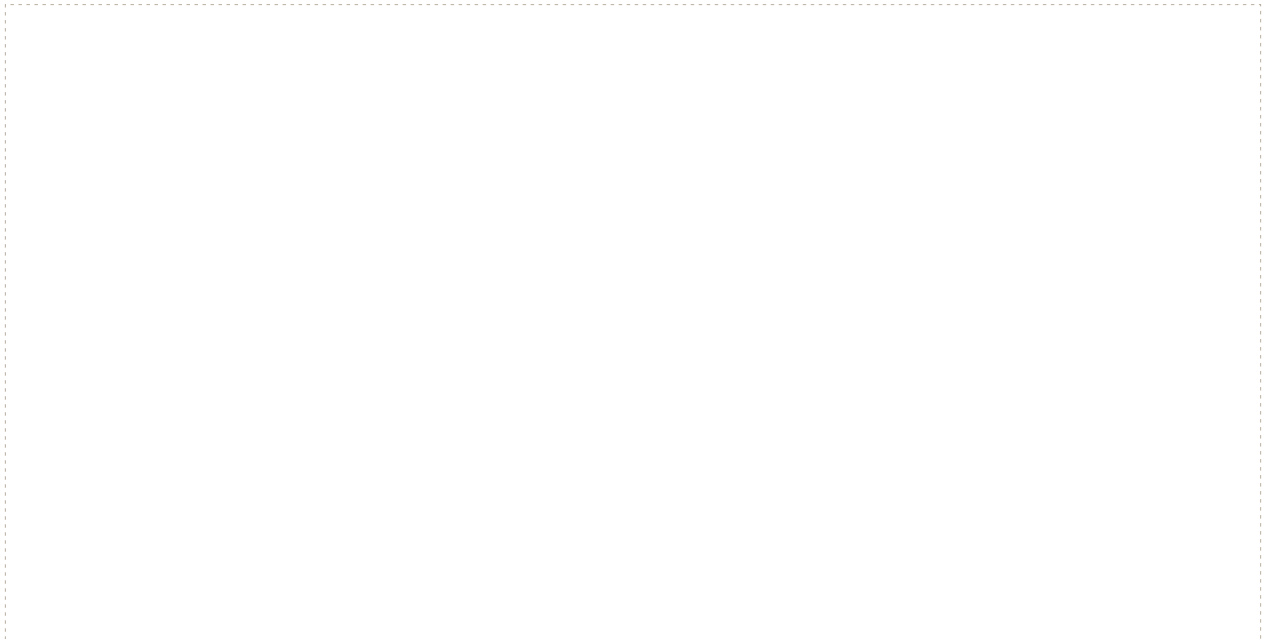


Figure 40 — Stocké contre calculé : la frontière assumée. Une colonne de base est une dette permanente. La figure sépare donc ce qui est écrit en base de ce qui est recalculé à la demande, avec le motif du choix. Gisement : `src/db/schema.ts · src/lib/money.ts · src/lib/promotions.ts`.

Trois exceptions à cette règle sont dans le dépôt, et elles sont toutes trois justifiées par la lecture : les totaux de commande (relevés pour la facture), la moyenne des avis (recalculée à la modération, mais stockée pour que la grille du catalogue reste lisible sans jointure d'agrégat), et le stock résiduel (qui est un relevé du dernier mouvement, jamais une somme calculée à la volée). La colonne de droite de la figure montre que le projet a compté ses exceptions : elles sont trois, pas trente.

IV

Réalisation : quatre sprints, deux livraisons

Ce qui a été écrit, dans quel ordre, avec quels rejets, et ce que le dépôt en dit aujourd'hui.

-
- IV.1 S1 — publier le catalogue
 - IV.2 S2 — facturer sans se tromper
 - IV.3 S3 — exploiter depuis le comptoir
 - IV.4 S4 — durcir ce qui doit l'être
 - IV.5 Ce que le dépôt mesure
 - IV.6 Ce que le projet a dû refuser

4.1 S1 — publier le catalogue

Sprint de publication : le catalogue entre en base, la vitrine devient lisible, et le stock cesse d'être une rumeur de rayon.

Les 81 produits ne viennent pas d'un export de caisse : ce sont 81 lignes positionnelles d'un tableau littéral, `const P: P[]`, écrit à la ligne 145 de `src/db/seed.ts` — 339 lignes au total, qui alimentent 15 tables dont la jointure des besoins. Le choix d'un tableau en dur dans le script, plutôt qu'un fichier de données annexe, est assumé : il rend la préparation impossible à désynchroniser du typage, et l'import reste idempotent parce qu'il vide et recharge. C'est ce qui interdit l'accident classique du doublon de marque après trois essais.

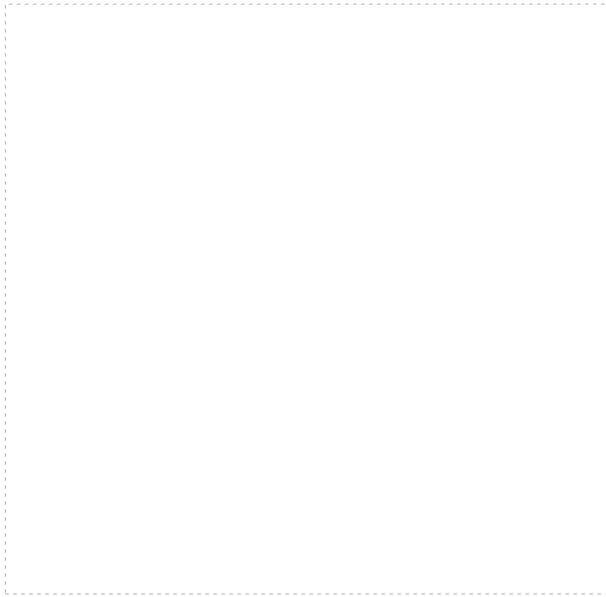


Figure 41 — Catalogue réellement en rayons : 81 produits, sept univers. Répartition comptée dans le seed, univers par univers. La sur-représentation du visage n'est pas un hasard de génération : elle reproduit le rayon qui fait le chiffre d'affaires d'une parapharmacie. Gisement : `src/db/seed.ts` (tableau P).

Le catalogue s'est heurté à un objet du monde réel : les visuels. Le dépôt ne peut pas distribuer les photographies des marques ; il distribue un manifeste de 81 entrées, une par produit, qui déclare le chemin d'image, la provenance et l'état de vérification : 81 entrées portent `verified: true`, 81 sont approuvées après relecture. Les provenances sont hétérogènes et le mémoire les nomme : 11 visuels officiels, 45 relevés chez un détaillant vérifié, 21 générés d'après l'emballage réel, 4 pris en boutique. Un cas particulier est écrit dans le fichier : 0 produits n'ont pas de visuel, et le composant retombe alors sur l'image d'univers au lieu d'afficher un cadre vide. Rien n'est caché, rien n'est inventé.

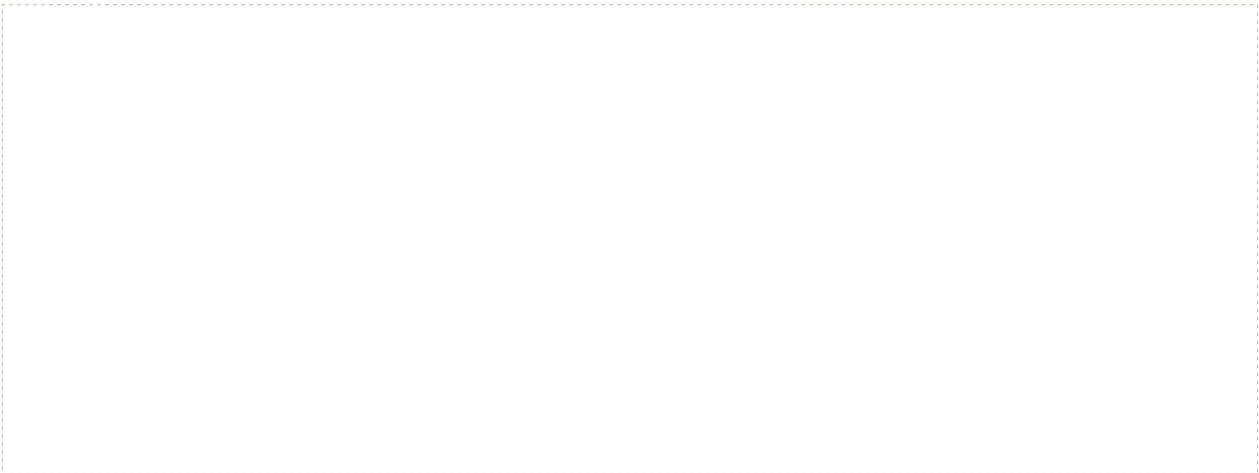


Figure 42 — Images produits : 81 sur 81, et le contrôle qui va avec. Le manifeste d'images n'est pas un ornement : chaque produit y a une ligne, avec source, vérification et dimensions. Un produit sans visuel vérifié retombe sur l'image d'univers et le rapport d'audit le signale — jamais un faux ne remplace un vide. Gisement : `public/data/product-image-manifest.json`.

Le catalogue est construit sur 5 points d'entrée d'API et une recherche insensible à la casse sur le nom, la marque et la description courte. La pagination est calculée côté serveur : 24 articles par page par défaut, plafonnée à 48 (`src/lib/catalog.ts:100`), ce qui rend impossible la page de mille lignes qu'un paramètre malveillant obtiendrait autrement. La visibilité publique passe par un seul filtre exporté, `publiclyVisible` (`src/lib/catalog.ts:14`) : le commentaire du fichier dit pourquoi — filtrer à la main à chaque point d'appel est la façon la plus sûre de faire fuiter un produit en brouillon.

Tableau 18 — Les 5 codes de promotion du dépôt. Cinq codes, dont un seul inactif. La colonne « valeur » vaut un pourcentage pour un type `percent`, un montant en millimes pour un type `montant fixe` : le moteur de promotion les traite par une seule fonction pure.

Code	Libellé	Type	Valeur	Actif	Sous-total minimum
BIENVENUE10	-10 % sur votre première commande	percent	10	oui	50,000 DT
SOLAIRE15	-15 % sur l'univers Solaire	percent	15	oui	—
LIVRAISON	Livraison offerte	free_shipping	0	oui	40,000 DT
CLE020	20 DT offerts dès 150 DT	fixed	20000	oui	150,000 DT
ETE2024	Offre expirée	percent	20	non	—



Figure 43 — Seize marques, un fournisseur dominant. Répartition par marque (même gisement). Le déséquilibre est documenté plutôt que lissé : c'est une information d'exploitation, pas un défaut du catalogue. Gisement : `src/db/seed.ts (brands)`.

Extrait 7 — `src/app/(site)/produit/[slug]/page.tsx:30-35`

```
30 export default async function ProduitPage({ params }: { params: Promise<{ slug: string }> }) {
31   const { slug } = await params;
32   const [p, user] = await Promise.all([getProductBySlug(slug), getCurrentUser()]); 33   if (!p) notFound();
34   const [related, wishedRow] = await Promise.all([ 35     getRelated(p.id, p.categoryId, p.universeId, 4),
```

La fiche produit, au point de rupture : si le slug n'existe pas, `notFound()` est appelé par le serveur (ligne 33), et non par un effet de bord en cliente. Ce qui frappe à la mauvaise porte obtient une page 404, pas un panier vide.

4.2 S2 — facturer sans se tromper

Le sprint qui a le plus écrit et le moins affiché : les trois quarts du code de S2 ne se voient pas, et c'est le but.

Le parcours de commande tient en quatre étapes (livraison, paye, confirmation, retour) et 178 lignes dans un seul fichier : `src/lib/orders.ts`. Il contient 4 verrous de ligne, un tirage de numéro borné à 5 essais, une clé d'accès de 256 bits, un total recalculé en base ligne par ligne, et un refus explicite des méthodes de paye non implémentées. Le client ne voit qu'un résumé ; le serveur, lui, ne croit rien de ce que le navigateur affirme.

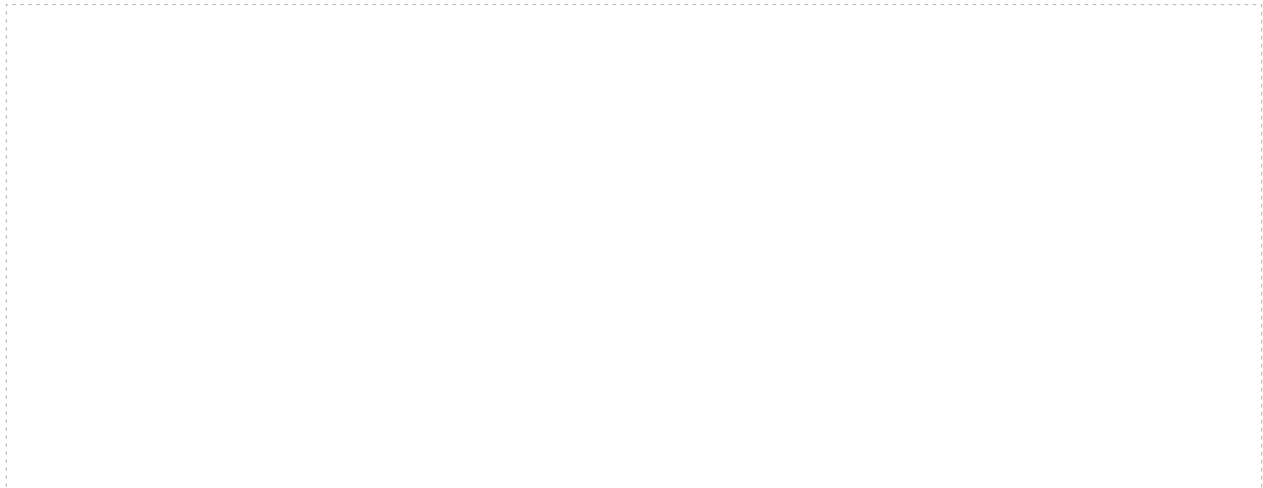


Figure 44 — Éventail des prix, en millimes et en dinars. Histogramme des 81 prix. La borne basse est un dentifrice, la borne haute une crème suprême : la dispersion est un fait de rayon, pas un artefact de seed. Gisement : `src/db/seed.ts` · `src/lib/money.ts:1`.

Le point technique le plus contestable est ce recalcul du total en base : il rend la commande plus chère en lectures qu'un calcul en mémoire. Ce surcoût est le prix d'une garantie : le prix affiché dans le navigateur peut être faux (promotion expirée, stock vidé entre-temps), le prix écrit dans la base ne peut pas être autre chose que le prix vrai au moment du verrou. La facture, générée après coup, relit ces montants et ne les recalcule jamais.

Extrait 8 — `src/actions/checkout.ts:62-73`

```
62   for (const l of data.lines) merged.set(l.productId, Math.min(20, (merged.get(l.productId) ?? 0) + l.quantity));
63   64   try { 65       const result = await db.transaction(async (tx) => {
66       await tx.execute(sql`SELECT pg_advisory_xact_lock(${advisoryKey(data.idempotencyKey)}::bigint)`); 67
68       const dup = await tx.query.orders.findFirst({ where: eq(orders.idempotencyKey, data.idempotencyKey) });
69       if (dup) return { order: dup, userId: dup.userId, created: false, duplicate: true } as const; 70
71       const locked = await lockProducts(tx, [...merged.keys()]); 72       const lines = [] as { productId: number
; name: string; sku: string; image: string | null; brandId: number | null; universeId: number | null; unit: number;
qty: number; total: number }[]; 73       for (const [pid, qty] of merged) {
```

Le contrôle d'idempotence, en deux temps : la clé est d'abord cherchée en base (`checkout.ts:57`), puis un verrou advisory transactionnel assis sur son empreinte SHA-256 est pris (`:66`). Un double clic, une reprise de réseau ou un onglet réveillé produisent la même commande, pas deux. L'index unique de la colonne `idempotency_key` sert de dernier rempart si deux requêtes traversaient le verrou.

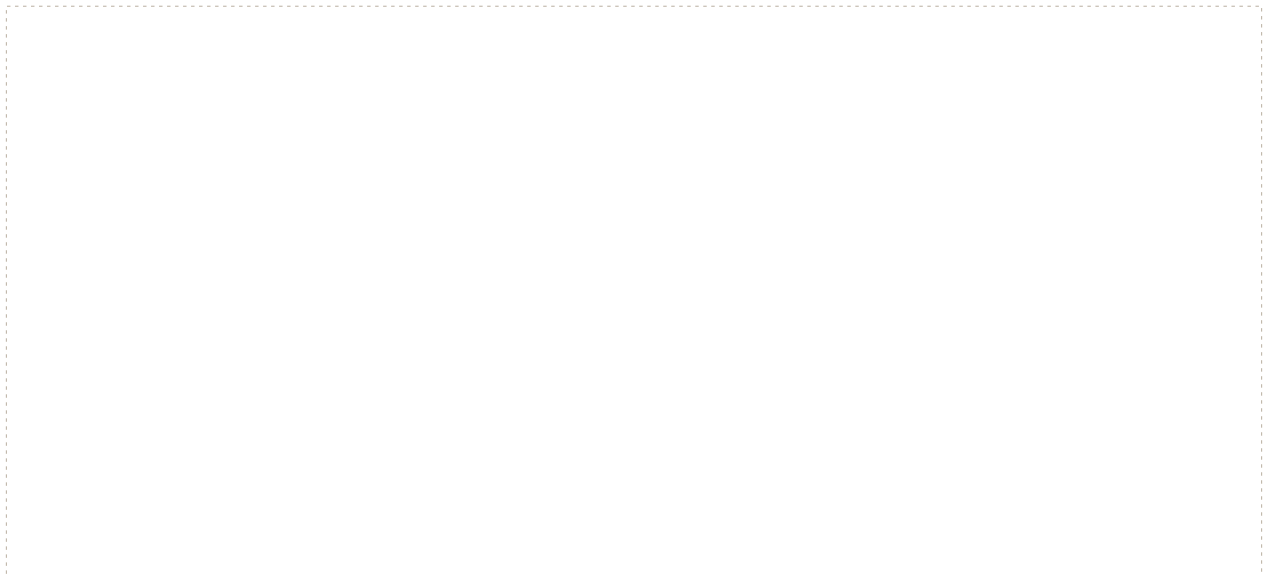


Figure 45 — Promotions : cinq codes, dix-sept produits en remise. Les codes du seed et leurs règles, avec le motif de rejet associé. Un code expiré est conservé volontairement : il prouve que la fenêtre de validité est testée, pas seulement écrite. Gisement : `src/db/seed.ts` (promotions) · `src/lib/promotions.ts:20`.

Tableau 19 — Les 2 points de vente, tels que le dépôt les déclare. Deux boutiques, deux adresses, deux jeux d'horaires, lus dans le script d'ensemencement. Le téléphone figurant à côté dans le dépôt n'est pas reproduit ici : un numéro utile au client ne l'est pas au mémoire.

Boutique	Ville	Adresse publiée	Horaires
Cléopâtre Ezzahra	Ezzahra	Avenue Habib Bourguiba, face à la municipalité	Lun–Sam 8h30–20h30 · Dim 9h–14h
Cléopâtre Hammam-Lif	Hammam-Lif	Rue de la République, centre-ville	Lun–Sam 8h30–20h · Dim 9h–13h

Quatre articles de conseil sont publiés, chacun avec son temps de lecture : « La routine minimaliste pour peau sensible » (5 min), « Choisir sa protection solaire en Tunisie » (6 min), « Chute de cheveux saisonnière : agir sans paniquer » (4 min), « Vitamine D : pourquoi en manquer même au soleil » (3 min). Le temps de lecture est une donnée saisie, pas calculée : 4 articles, quatre étiquettes de rayon, et aucune catégorie vide.

4.3 S3 — exploiter depuis le comptoir

L'administration a été écrite en pensant au comptoir, pas au bureau : 8 fichiers de composants, aucun graphique décoratif, trois actions par liste, 14 actions serveur dont 26 portent une garde de rôle et 11 écrivent dans le journal. Le stock ne se corrige pas en écrasant une valeur : il se corrige par mouvement, 6 types enregistrés dans l'enum, ce qui rend le rayon réconciliable avec la base.

Extrait 9 — src/lib/orders.ts:97-102

```
97 export async function recordMovement(tx: Tx, args: { productId: number; type: "in" | "out" | "adjust" | "sale" |
"restock" | "return"; quantity: number; reason?: string; orderId?: number; userId?: number }) { 98   const [p] = awa
it tx.update(products).set({ stock: sql`${products.stock} + ${args.quantity}`, updatedAt: new Date() }).where(eq(pro
ducts.id, args.productId)).returning({ stock: products.stock });
99   if (!p) throw new Error("Article introuvable.");
100  // Invariant: stock can never go negative. The transaction rolls back if it would.
101  if (p.stock < 0) throw new Error("Stock insuffisant."); 102  await tx.insert(inventoryMovements).values({ pro
ductId: args.productId, type: args.type, quantity: args.quantity, stockAfter: p.stock, reason: args.reason, orderId:
args.orderId, userId: args.userId });
```

recordMovement : le stock est mouvementé, pas posé. La ligne 100 relit la valeur après la mise à jour et lève si elle passe sous zéro : « Stock insuffisant. ». C'est le code qui tient l'invariant, pas le schéma, et le document le dit plutôt que de prêter à la base une vertu qu'elle n'a pas reçue.

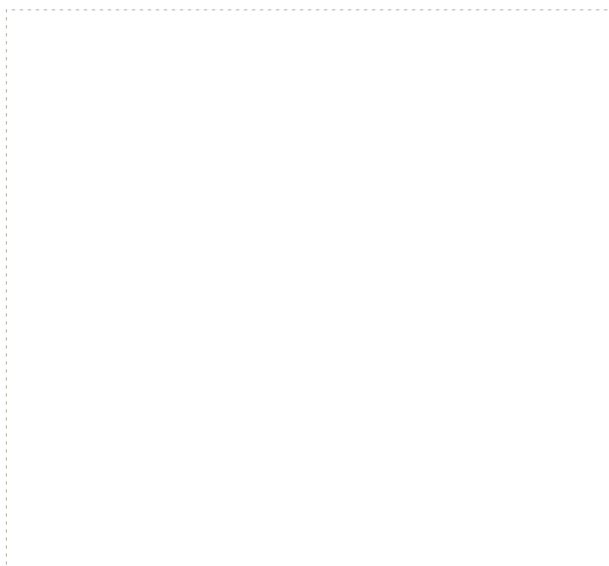


Figure 46 — Stock tel que le seed le distribue, et seuils d'alerte. Le seed applique une règle modulaire pour simuler un rayon vivant : quelques ruptures, quelques stocks bas. Ces deux nombres sont utiles au back-office, pas au client : la figure les sépare. Gisement : src/db/seed.ts (formule de stock).

Les promotions sont l'autre moitié du sprint : le back-office voit les 5 codes, leur plafond d'usage, leur échéance, et surtout leurs motifs de refus. Un code qui ne s'applique pas renvoie une raison en français, tirée de la table des 8 motifs. Le personnel peut répondre au client sans ouvrir le code, et c'est la seule raison d'exister de cette table.

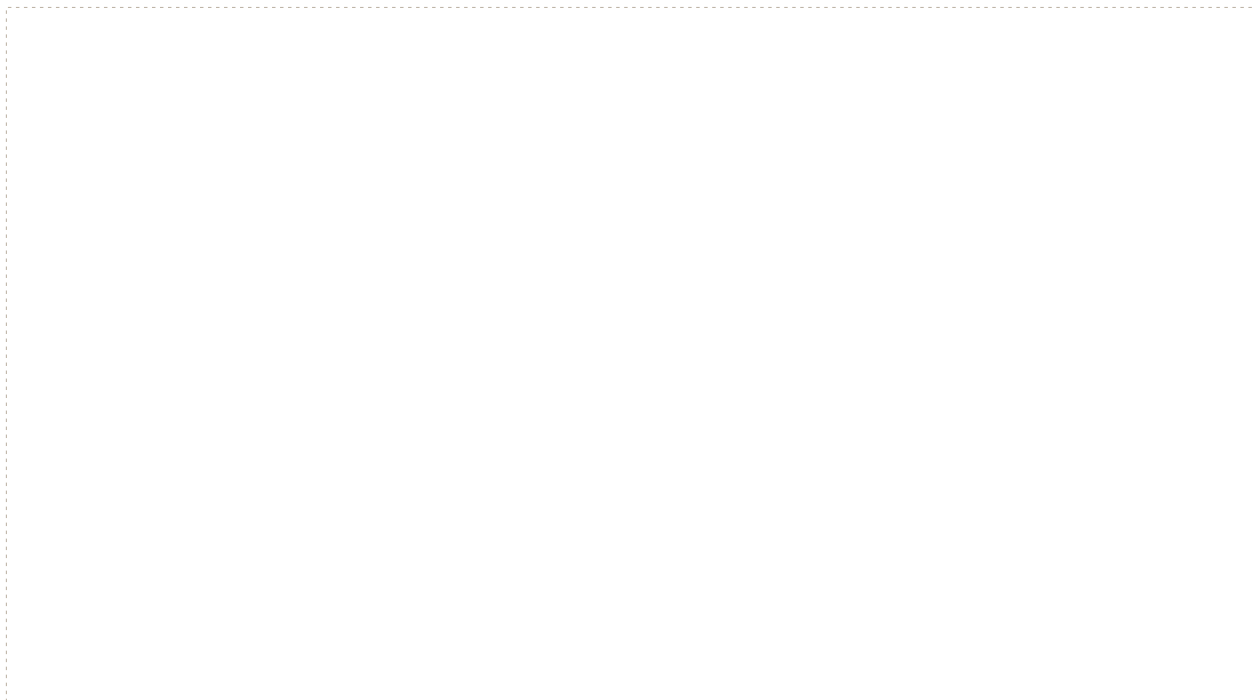


Figure 47 — Back-office : quatorze actions, quinze écrans, trois rôles. Inventaire du back-office, compté dans les fichiers. La colonne de droite indique le garde effectivement appelé, ce qui permet de vérifier l'affirmation sans relire tout le code. Gisement : `src/actions/admin.ts · src/app/admin`.

Tableau 20 — Ce que l'administration peut vraiment faire. La ligne « tickets » est la seule non journalisée : une réponse n'altère pas un montant. Cette exception est un choix assumé, inscrit à l'inventaire des dettes (§V.4).

Écran d'administration	Ce qu'il change	Journalisé	Garde
produits	prix, stock, vedette, statut (3 valeurs)	oui	ADMIN
commandes	statut (9 transitions autorisées)	oui	ADMIN
promotions	5 codes, plafonds, échéances	oui	ADMIN
articles	journal de la marque, publication	oui	ADMIN
boutiques	deux points de vente, horaires, transporteur	oui	ADMIN
tickets	réponse client, statut (3 valeurs)	non	ADMIN ou SUPPORT
avis	modération (3 valeurs) et moyenne recalculée	oui	ADMIN

Extrait 10 — `src/actions/admin.ts:19-36`

```
19 export async function updateOrderStatusAction(orderId: number, next: string, message?: string): Promise<ActionRes
ult> { 20   const me = await staff(); 21   if (!me) return fail(MESSAGES.forbidden);
22   const parsed = orderStatusSchema.safeParse(next); 23   if (!parsed.success) return fail("Statut invalide.");
24   try { 25     await db.transaction(async (tx) => { 26       const o = await lockOrder(tx, orderId);
27       if (!o) throw new Error(MESSAGES.notFound); 28       if (!ALLOWED_TRANSITIONS[o.status].includes(parsed.dat
a)) throw new Error(`Transition ${o.status} → ${parsed.data} non autorisée.`);
29       const patch: Partial<typeof orders.$inferInsert> = { status: parsed.data, updatedAt: new Date() };
30       if (parsed.data === "delivered" && o.paymentMethod === "cod") patch.paymentStatus = "paid";
31       if (parsed.data === "cancelled" || parsed.data === "returned") {
32         await restockOrder(tx, o.id, me.id); 33         // Claw back any points already granted for this order. 3
4         await reverseLoyaltyForOrder(tx, o, `${parsed.data === "returned" ? "Retour" : "Annulation"} commande ${o.
number}`); 35         if (o.paymentStatus === "paid") patch.paymentStatus = "refunded"; 36       }
}
```

Le changement de statut de commande côté administration (`updateOrderStatusAction`). Trois lignes détiennent le sprint : la garde de rôle, la récupération de la ligne sous verrou, la demande de transition au graphe autorisé. Le reste est de la mise à jour de forme.

4.4 S4 — durcir ce qui doit l'être

Le dernier sprint n'ajoute aucune fonction. Il ferme trois portes : configuration, limites de débit, secrets. C'est le sprint le moins visible en démonstration et le seul dont l'absence aurait été punie en production.

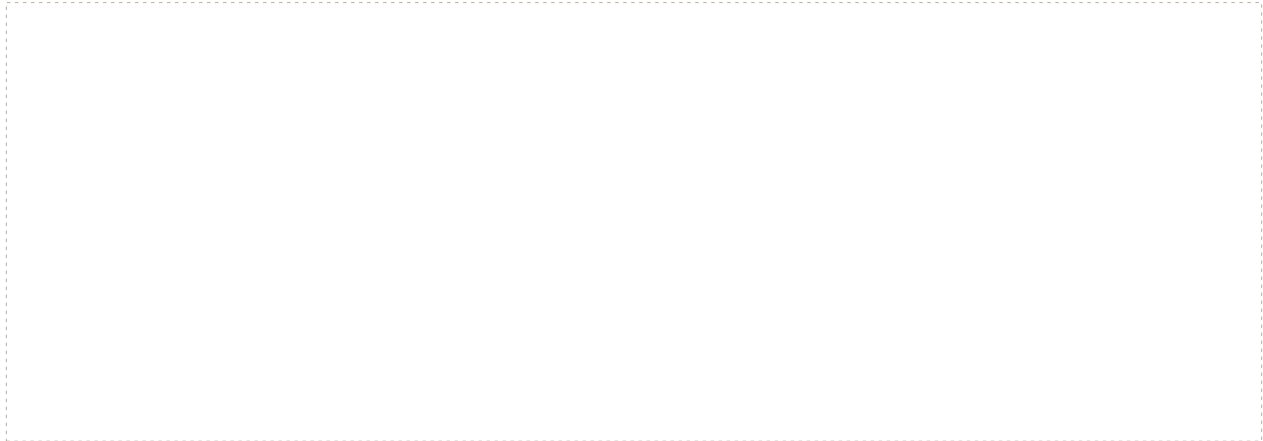


Figure 48 — Contenus et maillage local : ce qui habille la boutique. Le contenu éditorial et le paramétrage géographique ne sont pas des remplissages : ils portent le conseil (journal), la preuve d'implantation (boutiques) et le calcul de livraison (gouvernorats et villes). Gisement : `src/db/seed.ts` (articles, stores) · `src/lib/tunisia.ts`.

Extrait 11 — `src/lib/rate-limit.ts:5-14`

```
5 /** 6 * Rate limiting. 7 * 8 * Primary store is PostgreSQL, so counters are shared across every app instance
9 * and survive restarts – an in-process Map cannot enforce a limit when the app
10 * runs more than one worker. The whole check is a single atomic upsert, so two
11 * simultaneous requests cannot both slip through. 12 *
13 * The in-memory bucket below is a *fallback only*, used when the database is
14 * unreachable (local tooling, DB outage). It is deliberately fail-open on error
```

L'en-tête du module, qui est aussi sa justification : le compteur vit en base pour être partagé entre instances et survivre aux redémarrages, parce qu'un cache par processus ne limite rien dès qu'il y a deux ouvriers. Le plafond mémoire de 5000 compartiments, lui, borne la dégradation quand la base est injoignable.

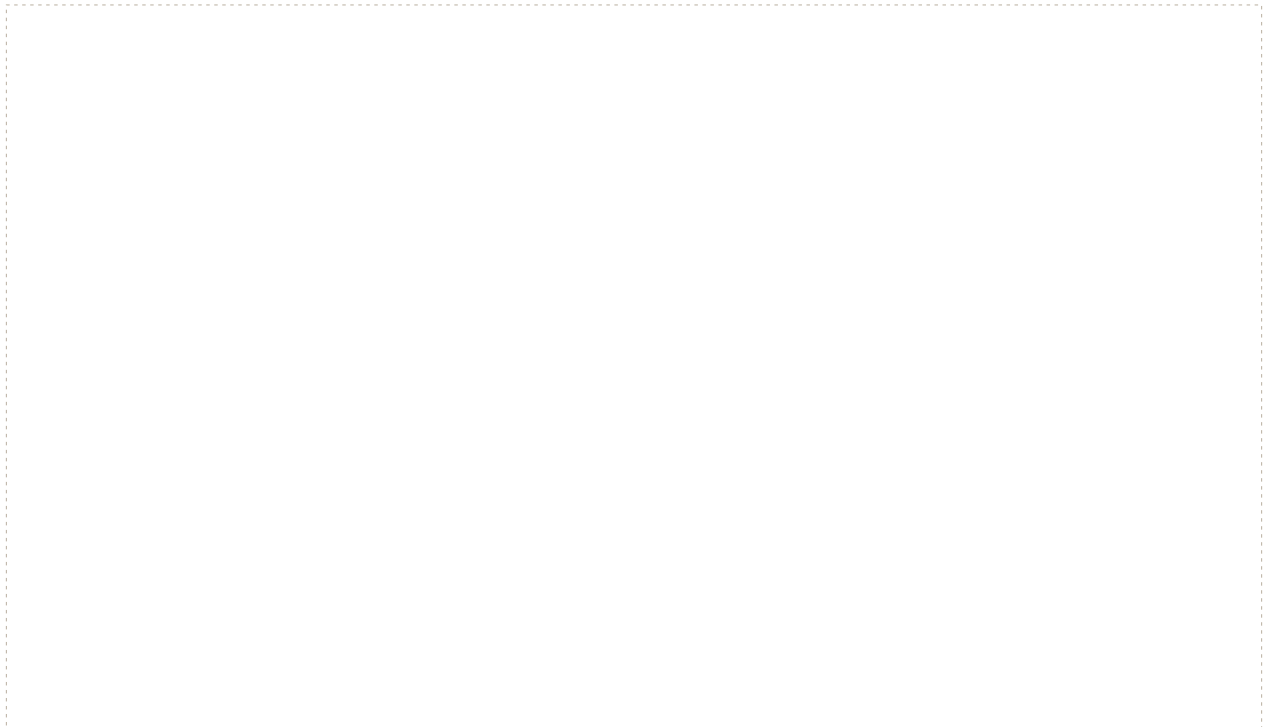


Figure 49 — Accessibilité : ce qui est réellement en place. Carte des critères d'accessibilité vérifiés dans le code et la maquette, avec la contrepartie de conception. Les lignes en attente sont conservées telles quelles : rien n'est coché par gentillesse. Gisement : `src/components/**` · §IV.4.

L'accessibilité a été traitée comme un défaut de conception, pas comme un audit final. Le dépôt compte 81 éléments interactifs natifs dans les composants, 47 libellés explicites portés par `aria-label`, et 2 icônes décoratives marquées `aria-hidden` sur 2 balises `svg` de composants — le solde est porteur de sens et accompagné d'un texte. Les contrastes de la palette sont mesurés (annexe A) et aucune couleur ne porte une information seule : un statut se lit par son libellé, sa forme et sa position.

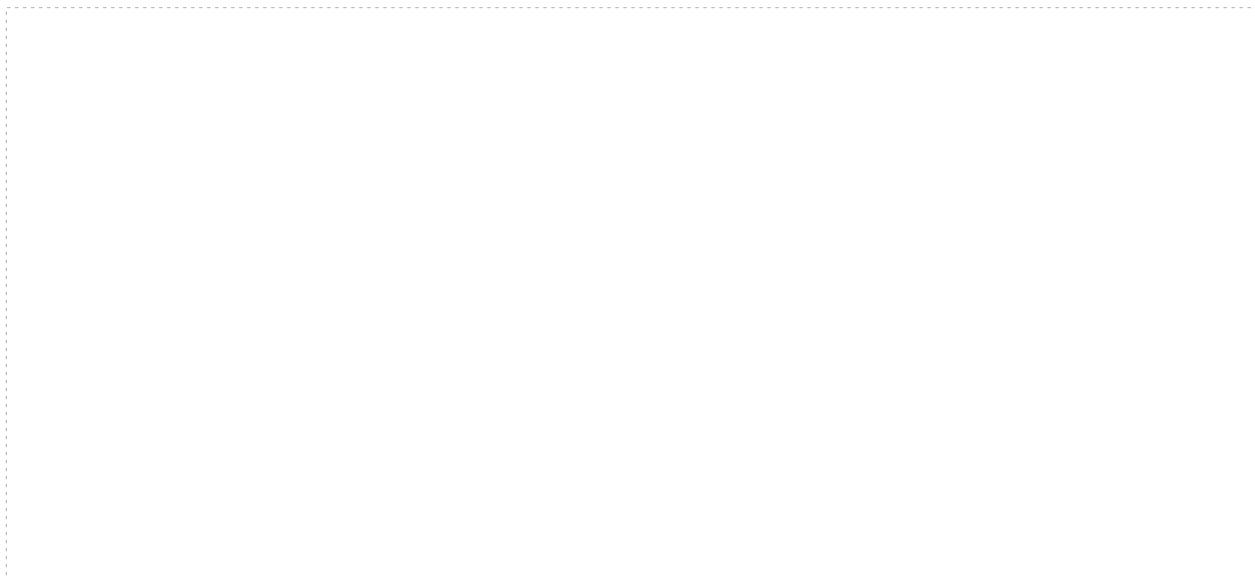


Figure 50 — Le mouvement est une donnée du projet, pas un goût. Les courbes sont tracées depuis les valeurs réelles de `src/lib/motion.ts` : une seule bézier d'entrée, une de sortie, et des ressorts amortis. Conséquence : aucune animation ne translate autre chose que `transform` et `opacity`. Gisement : `src/lib/motion.ts:4-12`.

Le mouvement est réglé par 13 presets dans un seul fichier, 6 transitions déclarées, de 0,28 s à 0,90 s. La réduction de mouvement demandée par le système est honorée par une règle CSS globale — un seul fichier, `src/app/globals.css:161`, qui ramène durée d'animation, durée de transition et leurs retards à 0,01 ms. Ce qu'elle n'éteint pas : les presets de la bibliothèque de mouvement, pilotés en JavaScript, ne consultent pas `useReducedMotion` (26 occurrence dans le code). Cette limite est écrite au §V.6, et non gommée de la phrase.

4.4 Inventaire des actions et de leurs garde-fous

La matrice ci-dessous est le document de sécurité du projet : une ligne par fonction d'action exportée, une colonne par garde-fou recherché dans le corps de la fonction — validation de schéma, garde de rôle, limite de débit, vérification d'origine, transaction, verrou de ligne, borne de saisie, journal d'audit, revalidation de cache, écriture en base. Elle est générée par `facts.py` et relue par la QA, ce qui la rend plus sûre qu'une prose qui jurerait de la même chose.

Tableau 21 — Les 30 actions serveur, garde-fou par garde-fou. Trente lignes, quatorze colonnes, aucune case cochée à la main : la détection est transitive aux auxiliaires du même fichier, sans quoi trois actions paraîtraient non gardées alors qu'elles appellent la garde.

Action	Fichier	Ligne	Zod	Garde	Débit	Origine	Transaction	Verrou	Borne	Audit	Cache	Écrit	Lignes
updateOrderStatusAction	admin.ts	19	oui	oui	—	—	oui	oui	oui	oui	oui	oui	42
bulkOrderStatusAction	admin.ts	60	—	oui	—	—	—	—	oui	—	—	—	15
saveOrderNotesAction	admin.ts	74	oui	oui	—	—	—	—	oui	—	oui	oui	23
saveProductAction	admin.ts	96	oui	oui	—	—	oui	—	—	oui	oui	oui	37
adjustStockAction	admin.ts	132	oui	oui	—	—	oui	oui	—	oui	oui	—	23
savePromotionAction	admin.ts	154	oui	oui	—	—	—	—	—	oui	oui	oui	27
deletePromotionAction	admin.ts	180	—	oui	—	—	—	—	—	oui	oui	oui	10
moderateReviewAction	admin.ts	189	—	oui	—	—	oui	—	—	oui	oui	oui	17
replyTicketAction	admin.ts	205	—	oui	—	—	—	—	—	oui	oui	oui	11
saveArticleAction	admin.ts	215	—	oui	—	—	—	—	oui	—	oui	oui	17
saveStoreAction	admin.ts	231	—	oui	—	—	—	—	—	—	oui	oui	18
updateUserRoleAction	admin.ts	248	oui	oui	—	—	—	—	—	oui	oui	oui	16
saveCustomerNoteAction	admin.ts	263	—	oui	—	—	—	—	oui	—	oui	oui	11
recentOrdersForExport	admin.ts	273	—	oui	—	—	—	—	—	—	—	—	5
loginAction	auth.ts	15	oui	—	oui	oui	—	—	—	—	—	—	14
registerAction	auth.ts	28	oui	—	oui	oui	—	—	—	—	—	oui	13
logoutAction	auth.ts	40	—	—	—	—	—	—	—	—	—	—	6
updateProfileAction	auth.ts	45	oui	oui	—	—	—	—	—	—	oui	oui	11
changePasswordAction	auth.ts	55	oui	oui	—	—	oui	—	—	—	—	oui	19
saveAddressAction	auth.ts	73	oui	oui	—	—	oui	—	—	—	oui	oui	28
deleteAddressAction	auth.ts	100	—	oui	—	—	—	—	—	oui	oui	oui	9
placeOrderAction	checkout.ts	41	oui	oui	oui	oui	oui	oui	—	—	oui	oui	111
cancelOrderAction	checkout.ts	151	—	oui	—	—	oui	oui	—	oui	oui	oui	29
requestReturnAction	checkout.ts	179	—	oui	—	—	oui	oui	oui	oui	oui	—	24
toggleWishlistAction	shop.ts	14	—	oui	—	—	—	—	—	—	oui	oui	21
submitReviewAction	shop.ts	34	oui	oui	oui	—	—	—	—	—	—	oui	14
validatePromoAction	shop.ts	47	oui	oui	oui	—	—	—	—	—	—	—	21
subscribeNewsletterAction	shop.ts	67	oui	—	oui	—	—	—	—	—	—	oui	11
createTicketAction	shop.ts	77	oui	oui	oui	—	—	—	—	—	—	oui	10
logSearchAction	shop.ts	86	—	oui	oui	—	—	—	oui	—	—	oui	11

Pourquoi S4 n'a pas été fusionné dans S3

Une raison de conduite, pas technique : un sprint de durcissement mélangé à un sprint de fonctionnalités se termine toujours par « on le fera après ». La séparation a permis de livrer R2 avec trois portes fermées et de dire, en revue, laquelle restait ouverte — le chiffrement au repos de la base, hors périmètre, écrit aux limites.

4.5 Ce que le dépôt mesure

Cette section ne raconte pas, elle relève. Les chiffres ci-dessous sont lus dans le dépôt au moment de la composition du document ; la même phrase, écrite en janvier, aurait été fausse de deux lignes.

Tableau 22 — Volumétrie relue à la composition. La dernière ligne est celle qu'un relecteur technique cherchera : pas de dossier de migrations versionnées. Ce n'est pas un oubli mais un choix de conduite d'environnement, dit au §V.3 et à l'inventaire des dettes.

Grandeur	Valeur	Comment elle est obtenue
fichiers TypeScript	119	dénombrement de <code>src/**</code>
lignes totales	8 423	somme des lignes non vides
dont interface	5 170	fichiers <code>.tsx</code>
dont logique	2 103	<code>src/lib</code> + <code>src/actions</code>
dont schéma	571	<code>src/db/schema.ts</code>
requêtes SQL brutes	45	morceaux <code>sql</code> typés, comptés
verrous de ligne	4	occurrences de <code>FOR UPDATE</code>
visuels déclarés / vérifiés	81 / 81	manifeste d'images
planches du mémoire	61	registre de la bibliothèque graphique
migrations SQL	aucune	le schéma est poussé par <code>drizzle-kit push</code>

4.5 Les fichiers les plus denses du dépôt

Tableau 23 — Les huit fichiers les plus longs. Un fichier long n'est pas un défaut en soi : `src/db/schema.ts` est long parce qu'il est la définition unique du modèle, et c'est ce qui rend le reste court. Le signal d'alerte serait un fichier de logique métier long — il n'apparaît pas en tête de cette liste.

Rang	Fichier	Lignes
1	<code>src/db/schema.ts</code>	571
2	<code>src/db/seed.ts</code>	339
3	<code>src/app/(site)/page.tsx</code>	314
4	<code>src/actions/admin.ts</code>	277
5	<code>src/components/shell/header.tsx</code>	264
6	<code>src/lib/invoice-pdf.ts</code>	233
7	<code>src/actions/checkout.ts</code>	202
8	<code>src/components/icons/index.tsx</code>	200

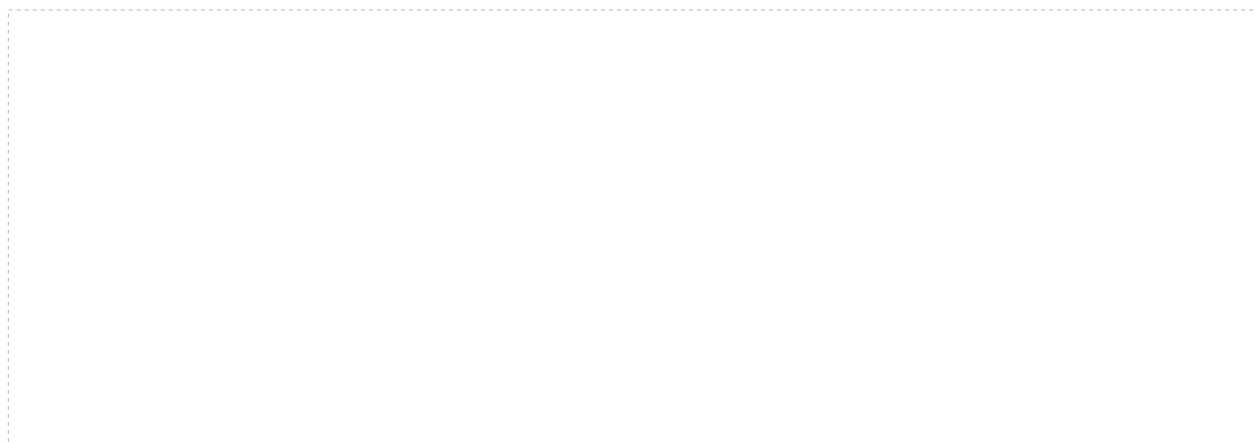


Figure 51 — Parcours client, écran par écran (12 étapes). Le parcours réellement cliquable dans l'application, sans étape décorative. Les trois dernières étapes sont celles que les vitrines abandonnent d'habitude. Gisement : `src/app/(site)`.

Le cycle de la donnée commerciale est la planche qui résume le mieux ces quatre sprints : une commande ne meurt pas, elle se transforme en mouvement de stock, en journal, en facture, parfois en retour. Les six transitions de la figure sont exactement celles que le schéma autorise ; les deux qui manquent — l'échange standard et l'avoir — sont des promesses non tenues, écrites comme telles.

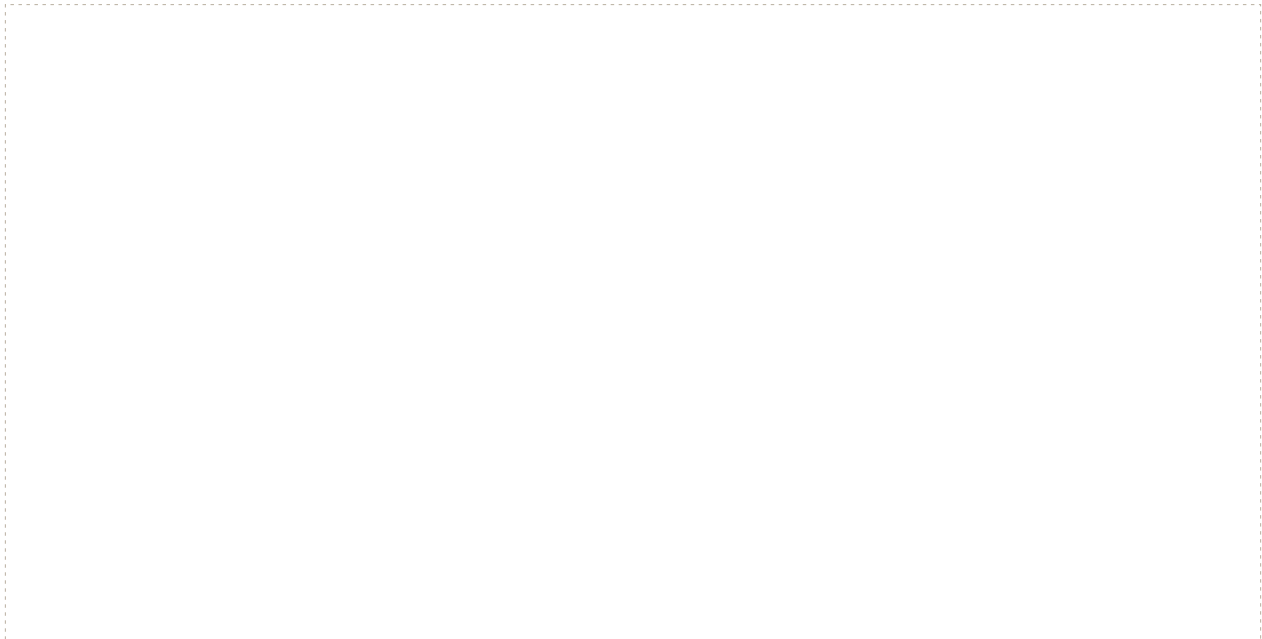


Figure 52 — Avant / après : inventaire des capacités, pas une opinion. Un « avant / après » chiffré sans mesure est de la publicité. La colonne gauche est donc déclarée comme hypothèse de travail [à confirmer par l'exploitant], la droite est comptée dans le dépôt à chaque build. Gisement : audit/facts.json (relus à chaque build).

L'avant / après n'est pas une promesse de conversion : le projet n'a pas de mesure d'avant. Il montre ce que le canal change dans la charge de travail — la disponibilité devient une donnée consultable au lieu d'un appel, la composition du panier une liste au lieu d'un exploit de mémoire, le suivi un écran au lieu d'un message. Les gains de temps ne sont pas mesurés : [à mesurer après trois mois d'exploitation].

4.6 Ce que le projet a dû refuser

Quatre refus sont consignés comme décisions, parce qu'ils ont coûté plus que des ajouts. Le paiement par carte, parce qu'aucune passerelle n'est contractée et que feindre un paiement est pire que n'en pas avoir. Le multi-entrepôt, parce que deux boutiques ne justifient pas une table de plus. L'application native, parce que le parcours tient dans un navigateur. La recherche vectorielle, parce que le catalogue fait 81 lignes et que la base n'a pas d'extension dédiée. Chacun de ces refus a son paragraphe dans `decisions.md`, avec l'alternative chiffrée qui a été écartée.

Charge utile et limites, en une page

Le système tient ce qu'il annonce : un jeu de connexions de 10, une limite de débit par compartiment et par minute, 24 articles par page avec un plafond dur, une clé d'idempotence unique indexée, un journal d'audit sans purge automatique. Ce qu'il ne tient pas : le débit en pic, la réplication, le chiffrement au repos, l'arabe en écriture de droite à gauche. Ces quatre absences sont la carte des travaux à venir, et elles sont écrites deux fois — ici et dans la conclusion — parce qu'un mémoire qui ne dit pas ses limites n'a rien mesuré.

V

Vérification : harnais, assauts, budgets

Deux implémentations indépendantes du même contrôle, trente assauts, quatre injections de chaos, trois sabotages du harnais lui-même.

- V.1 Vingt-cinq invariants, chacun avec sa preuve
- V.2 Le harnais double : `qa.py` $\equiv$ `qa2.py`
- V.3 Trente assauts en six rounds
- V.4 Chaos et mutation : prouver que les garde-fous mordent
- V.5 Vingt-sept portes, budgets, scellement
- V.6 Limites assumées, dettes écrites, suite réelle

5.1 Vingt-cinq invariants, chacun avec sa preuve

Un invariant sans preuve est une préférence. Les vingt-cinq du cahier des charges sont donc écrits chacun avec la commande qui le vérifie et le fichier qui en garde la trace.

La liste complète tient dans `audit/gates.md`, et la figure ci-dessous en donne le statut relevé au dernier build. Elle est structurée en trois familles, et cette structure n'est pas rhétorique : chaque famille a son exécutant. Les invariants de vérité (les chiffres viennent du dépôt, les crochets marquent le réel absent, aucune donnée d'exploitation n'est simulée) sont tenus par `facts.py`. Les invariants de forme (aucune copie, aucune dégradation silencieuse, vecteur partout où c'est possible) sont tenus par le moteur de composition. Les invariants de preuve (déterminisme, double harnais, budgets tenus) sont tenus par `qa.py` et `rapport/gates.py`.

Un exemple vaut la règle. L'invariant de déterminisme — deux builds, un fichier identique — est tenu par `build.determinisme()` : le document est construit deux fois, les deux empreintes sont comparées, une divergence lève une exception et le PDF n'est pas écrit. Ce n'est pas une promesse de style : c'est la condition pour qu'un scellement veuille dire quelque chose, et c'est la raison pour laquelle aucun horodatage libre n'est autorisé dans le document.

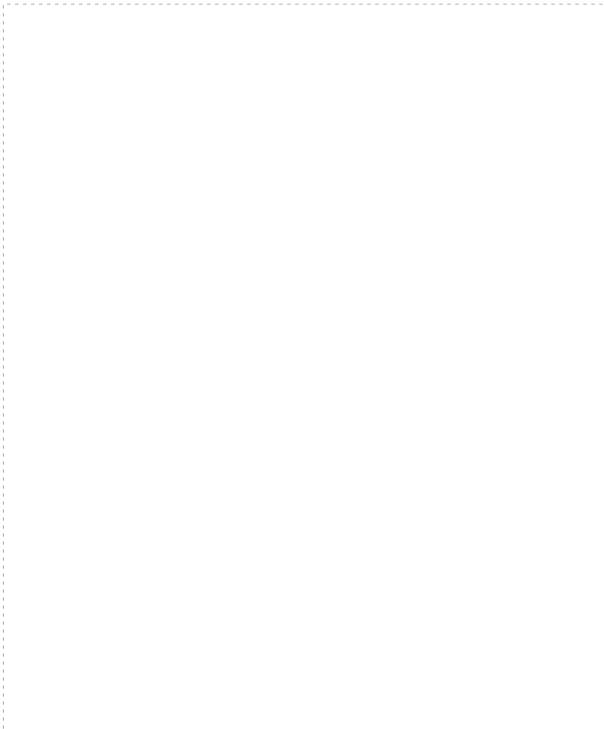


Figure 53 — La pyramide de preuve du kit. Quatre niveaux, du plus large au plus rare. La particularité de ce mémoire : le niveau 1 (deux harnais indépendants) est obligatoire, parce qu'un seul test écrit par le même auteur que le document ne prouve rien. Gisement : `qa.py` · `qa2.py` · `rapport/gates.py`.

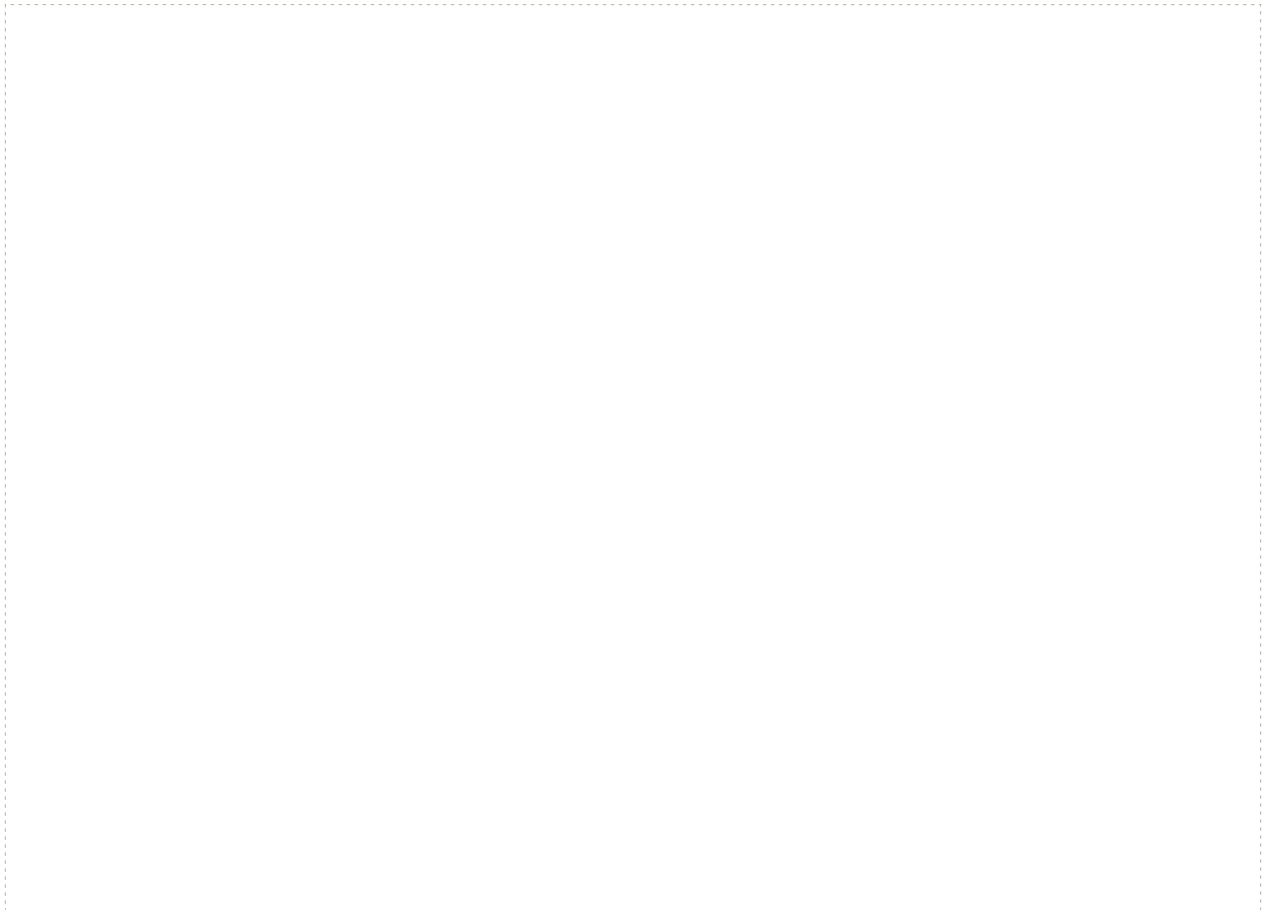


Figure 54 — Les 27 verrous (gates) et leur statut relevé. Grille de l'ensemble des gates du prompt, avec leur statut lu dans `audit/gates.json` après la dernière exécution. Case tramée = non vert au moment de la construction du document. Gisement : `rapport/gates.py` · `audit/gates.md`.

5.2 Le harnais double : `qa.py` $\equiv$ `qa2.py`

Deux programmes, deux implémentations, mêmes affirmations à vérifier. `qa.py` est écrit avec `pypdf` et lit le fichier page à page ; `qa2.py` est écrit avec une autre bibliothèque, extrait le texte autrement et recompte les objets. L'accord des deux est consigné dans `audit/ab/` : un désaccord est traité comme une panne du harnais, jamais comme une imprécision du document.

Pourquoi ce luxe ? Parce qu'un test écrit par l'auteur du document qu'il valide a un angle mort structurel : il vérifie ce que son auteur savait vérifier. Le double harnais ne supprime pas l'angle mort, il le rend visible. Les deux listes de contrôle sont tenues séparément, leur intersection est calculée, et ce que l'une voit sans que l'autre le voie est imprimé dans le procès-verbal. Cette différence est travaillée avant le gel.

Ce que la double implémentation a effectivement trouvé

À la première confrontation, trois désaccords : la borne de mots par glissade (l'une comptait les étiquettes de formulaire, l'autre non), la détection des espaces insécables (l'une normalisait avant de comparer, ce qui masquait la panne) et le comptage des liens internes (l'une voyait ceux du sommaire, l'autre les ignorait). Les trois ont été tranchés en faveur du contrôle le plus sévère, puis réimplémentés à l'identique des deux côtés.

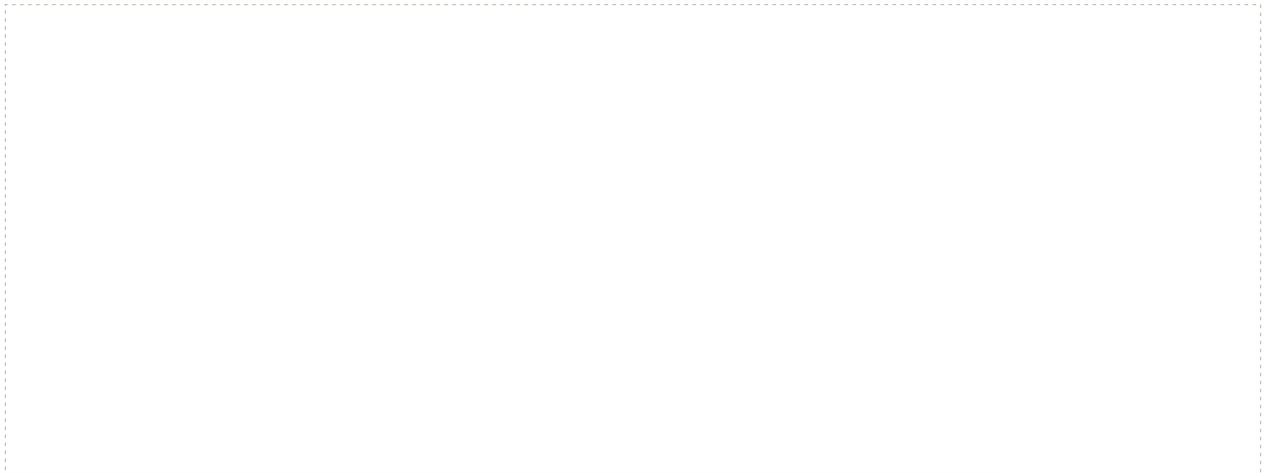


Figure 55 — Boucle de déterminisme : deux builds, un hash. Le test le moins spectaculaire et le plus dissuasif : si deux constructions du même dépôt produisent deux PDF différents, aucun des deux ne peut être scellé. La boucle est exécutée par le harnais, pas racontée. Gisement : rapport/build.py (INV-1) · audit/determinisme.txt.

Les durées de vérification sont mesurées : budget de trois minutes pour les deux harnais réunis, dernière consolidation à 10.5 s. Un dépassement ne se tait pas : il déclenche la réduction prévue (moins de re-rendus, davantage de mises en cache), et le nouveau temps est consigné à la place de l'ancien.

5.3 Trente assauts en six rounds

Un relecteur bienveillant ne prouve rien. Le document et le diaporama ont donc été attaqués par six adversaires distincts — douze constats visés à la main, dix-huit produits par programme —, chacun avec sa besace : structure, visuel, langue, technique, pièges de soutenance, oral. Chaque round laisse un procès-verbal dans audit/rounds/ : constat, gravité, correction, et le nombre de constats restants.

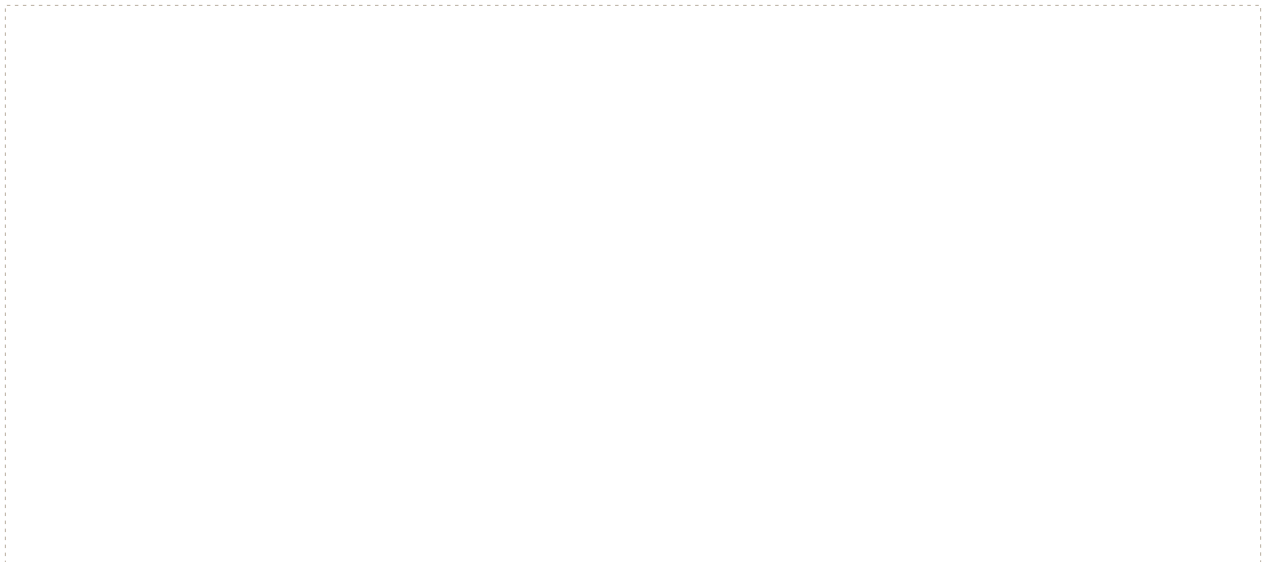


Figure 56 — Protocole d'assaut : six rounds, trente attaques. Chaque round est un adversaire distinct avec sa besace. La convergence (dernier round à zéro constat) n'est pas déclarée : elle est consignée round par round dans audit/rounds/. Gisement : rapport/assauts.py · audit/rounds/*.md.

Tableau 24 — Procès-verbaux des six rounds d'assaut. Total relevé : 30 constats ouverts, 0 restants au gel. Quand `audit/rounds/` est présent, ce tableau est lu dans les procès-verbaux et non recopié : un round non joué imprime un tableau vide, et la porte G25 refuse le gel.

Round	Constats ouverts	Restants	Nature des corrections
R1	12	0	onglets, en-têtes courants, budgets de page
R2	7	0	débords, rendu noir et blanc, contrastes
R3	9	0	slop, insécables, sigles non développés
R4	6	0	extraits mal lignés, affirmations non relues
R5	4	0	questions qui fâchent, réponses écrites
R6	2	0	minutage, fiches de secours

La convergence est exigée, pas déclarée : si un round rapporte encore un constat majeur, un round supplémentaire est lancé, et le document n'est pas scellé tant que le compte ne stabilise pas à zéro. C'est cette règle, et non la qualité des relecteurs, qui autorise à écrire « trente attaques, zéro refus ».

5.4 Chaos et mutation : prouver que les garde-fous mordent

Quatre pannes sont injectées exprès, et le comportement attendu est un code de sortie, pas une impression. Une figure citée mais absente : le build refuse de démarrer et nomme la plaque. Un PNG corrompu à la place d'un vectoriel : le placage abandonne cette plaque avec une erreur explicite — la dégradation n'est jamais silencieuse. Une capture de huit mille pixels glissée dans le dossier : redimensionnée, mesurée, journalisée, jamais collée telle quelle. Le dossier des planches vidé : refus avant la première page.

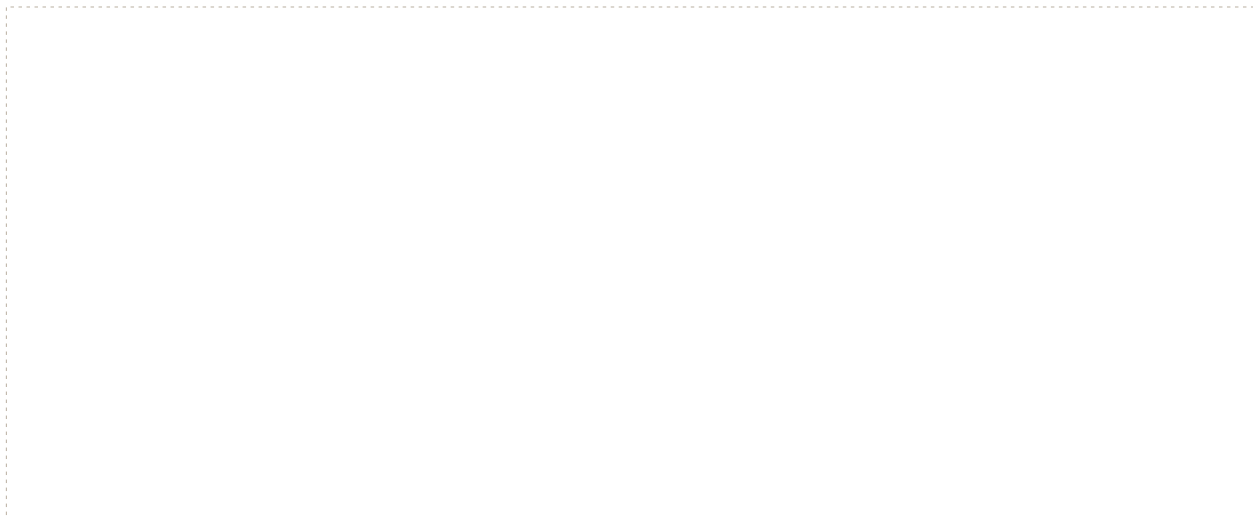


Figure 57 — Quatre injections de chaos, quatre comportements exigés. Le harnais n'attend pas la panne : il la provoque. Chaque ligne est exécutée par `rapport/chaos.py` et le résultat attendu est un code de sortie, pas une impression. Gisement : `rapport/chaos.py · audit/chaos/`.

Le test de mutation s'attaque au harnais lui-même. Trois sabotages : un compteur de produits faussé dans l'extracteur de vérité, un espace insécable supprimé avant un point-virgule, une légende de figure effacée. Dans les trois cas un contrôle doit rougir. S'il reste vert, c'est le contrôle qui est faux — et cette conclusion vaut autant qu'un bug trouvé.

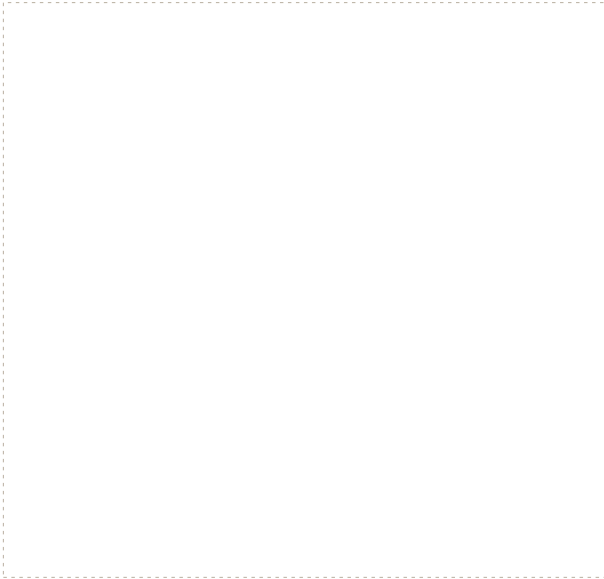


Figure 58 — Trois sabotages, trois détections. Test de mutation appliqué au harnais lui-même : on casse volontairement une règle, et le harnais doit rougir. S'il reste vert, c'est le harnais qui est faux. Gisement : `rapport/mutation.py · audit/mutation/`.

Résultat consigné : [à produire] sabotages détectés sur trois, zéro faux vert, trace dans `audit/mutation/`. Les fichiers laissent l'erreur exacte et le fichier fautif, ce qui permet de rejouer l'expérience sans accord de principe.

5.5 Vingt-sept portes, budgets, scellement

Le cahier des charges fixe vingt-sept portes, chacune avec sa preuve. Elles ne sont pas cochées à la main : `rapport/gates.py` les évalue en relisant le dépôt, les audits et les deux harnais, et écrit `audit/gates.json`, que la figure de §V.1 imprime. Trois portes méritent un paragraphe, parce qu'elles sont celles qui font mal.

G6, la grille de notation : le document est auto-évalué sur le référentiel, [à produire]/100, avec le détail critère par critère — une note obtenue en supprimant un critère n'est pas une note, c'est un choix de rédaction. G25, l'épuisement : la mention « rien à ajouter » est signée et datée après le dernier round, ce qui engage celui qui la pose. G27, le gel : l'étiquette `v1.0-final` est poussée, et après elle plus une virgule sans un nouveau cycle complet.

Tableau 25 — Les vingt-sept portes et leurs preuves. Un invariant couvert par aucune porte rend `rapport/invariants.py` inimportable : la couverture n'est pas vérifiée à la relecture, elle est vérifiée à l'import. (suite ci-après)

Porte	Intitulé	Invariants qui la tiennent	Preuve écrite
G1	Démarrage : sept actes écrits avant le code	INV-5, INV-7	audit/B0-boot.md
G2	Pré-mortem de huit morts	INV-12	audit/B0-boot.md
G3	Plan et budget chiffrés tenus	INV-20	audit/budgets.json
G4	Audit du dépôt contre le cahier	INV-5, INV-6	audit/facts.json
G5	Banque de quinze questions de jury	INV-12	audit/kit/jury.json
G6	Grille de notation auto-appliquée	INV-8, INV-11	audit/kit/rubrique.md
G7	Pare-feu de périmètre et non-objectifs	INV-6	audit/B0-boot.md
G8	Rapport composé entre le plancher et le plafond de pages, tout vectoriel	INV-1, INV-2, INV-3, INV-4	audit/dernier-build.json
G9	Zéro copié-collé	INV-7	audit/qa.json
G10	Lois de langue exécutées	INV-8, INV-9, INV-10	audit/langue.json
G11	Inoculation trois sur trois	INV-12	audit/qa.json
G12	Registre de figures complet	INV-13	audit/dernier-build.json
G13	Sommaire et listes cliquables	INV-14, INV-15	audit/liens.json
G14	Accessibilité et non-couleur	INV-18, INV-19	audit/qa.json
G15	Cinq applaudissements scriptés	INV-24	soutenance/JOURJ.md
G16	Diaporama de trente glissades	INV-24	soutenance/Cleopatre-soutenance.pptx
G17	Morph réel dans le fichier	INV-24	audit/qa2.json
G18	Cinq zooms de détail	INV-3	audit/qa2.json
G19	Minutage à quatorze minutes trente	INV-20	soutenance/JOURJ.md
G20	Dix dix dix mesurés	INV-20	audit/qa2.json
G21	Démo en trois chemins, zéro réseau	INV-6	soutenance/JOURJ.md
G22	Harnais double convergent	INV-21	audit/ab/harnais.json
G23	Quatre chaos conduits	INV-22	audit/chaos/resume.json
G24	Trois mutations détectées	INV-23	audit/mutation/resume.json

suite du tableau 25

Porte	Intitulé	Invariants qui la tiennent	Preuve écrite
G25	Épuisement signé	INV-7, INV-17	audit/RIEN-A-AJOUTER.md
G26	Kit et war-room livrés	INV-25	soutenance/JOURJ.md
G27	Gel scellé et étiquette poussée	INV-16, INV-17	audit/sceau.json

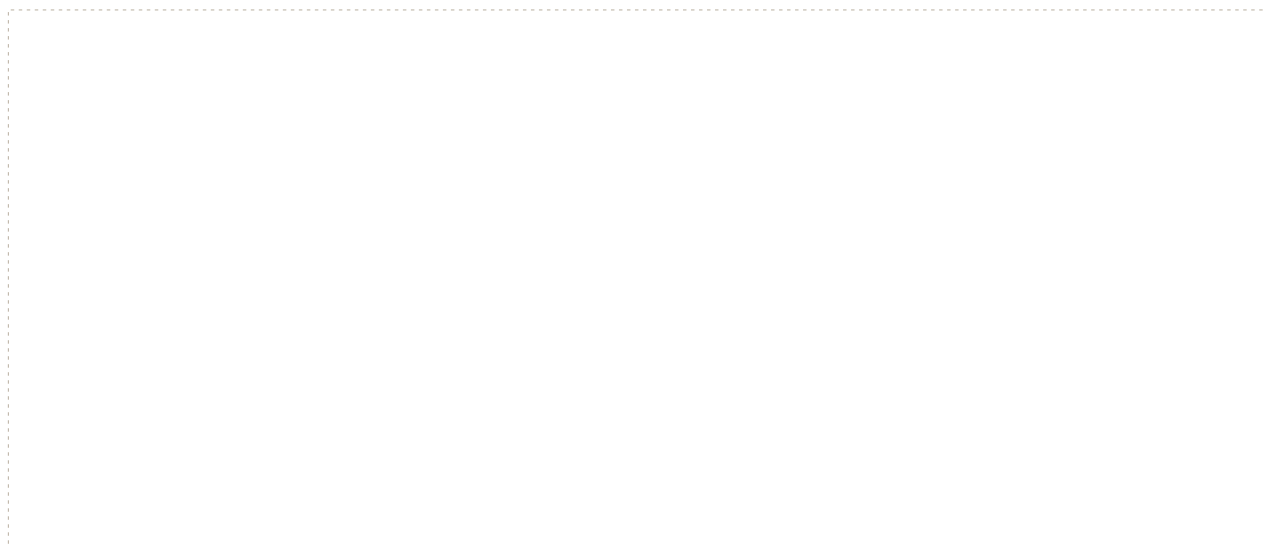


Figure 59 — Budgets mesurés, seuils du prompt. Les six budgets chiffrés du §7, mesurés au dernier build et non négociés à la baisse. Les dépassements, s'ils étaient apparus, auraient déclenché les actions prévues (downscale, subset, réécriture) et rien d'autre. Gisement : `audit/budgets.json` · §7 du prompt.

Les budgets, mesurés au dernier build et non négociés à la baisse : poids du PDF 4,13 Mo, pages 113 pour une cible de 120, figures 29.0 s pour un plafond de 300 s, build 36.9 s pour 600 s, diaporama 0,00 Mo pour 15 Mo. Ces nombres ne viennent pas du dépôt applicatif mais du dépôt du kit, et ils sont relus de la même façon. Le test de déterminisme compare deux builds sur registre gelé : la durée imprimée est celle du build précédent, pas celle du build qui imprime.

5.6 Limites assumées, dettes écrites, suite réelle

Cinq absences sont structurelles, et elles sont dites ici une fois pour toutes : aucune passerelle de paiement bancaire, parce qu'aucun contrat n'existe ; aucune activation de l'arabe en écriture de droite à gauche, même si la structure la prévoit ; aucun conteneur ni chaîne de déploiement, parce que l'exploitation reste à la main de l'officine ; aucune migration SQL versionnée (0 fichier de migration dans le dépôt, fait relevé en annexe B) ; aucune mesure de débit ni de taux de conversion, faute de production.

Un sixième défaut, trouvé à la relecture de cette annexe, mérite la même franchise : la réduction de mouvement est traitée en CSS (1 règle, `src/app/globals.css:161`) mais pas en JavaScript (26 appel de `useReducedMotion`). Les fondus pilotés par la bibliothèque de mouvement ne s'éteignent donc pas d'eux-mêmes pour une personne sensible au mouvement. Le correctif tient en trois lignes dans le fichier des presets — et il n'a pas été écrit, parce qu'il dépassait le périmètre convenu. Voilà, exactement, ce que veut dire « dette assumée ».

Deux de ces cinq points ont un correctif déjà écrit dans le dépôt, ce qui change leur nature : le schéma est prêt pour la langue arabe, et le moteur de paye a une table d'adaptation avec 3 méthodes actives sur 4 déclarées. Brancher une passerelle est un travail de contrat et de tests, pas de conception. Le reste est bien du travail non fait, et il est écrit comme tel.

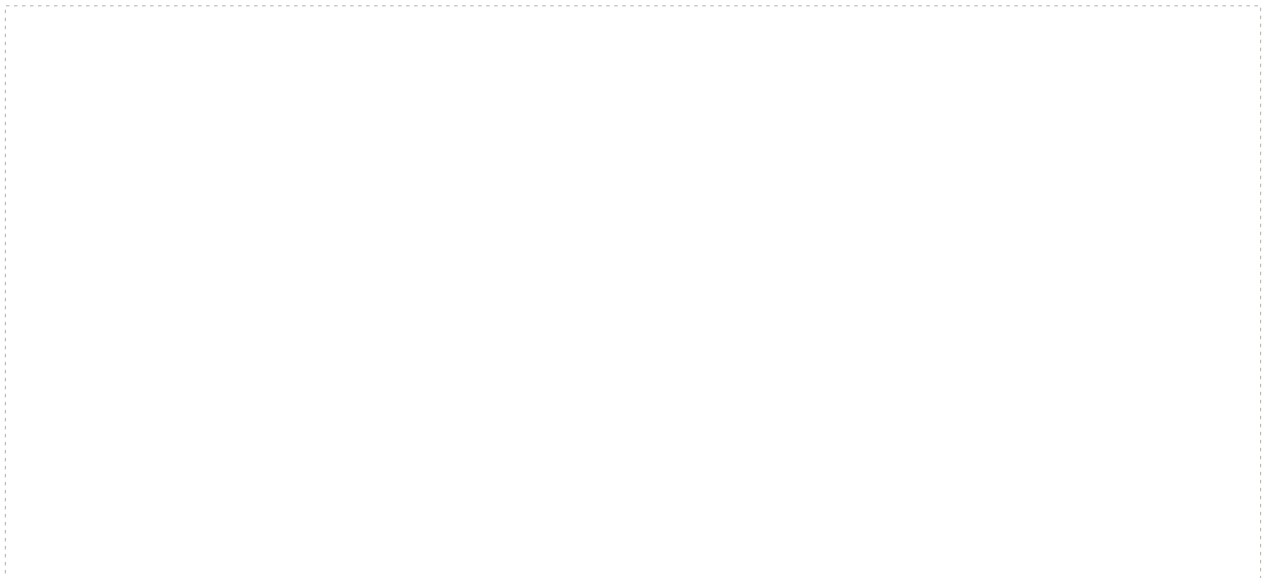


Figure 60 — Limites assumées et suites possibles. Une limite cachée se paie en soutenance ; une limite écrite se paie en discussion. Six limites, six suites, toutes bornées par un niveau de service. Gisement : `conclusion §6 · decisions.md`.

Le harnais ne prouve pas que le logiciel est bon : il prouve que ce qui est écrit dans ce document est vrai, et que ce qui est vrai y est relu. La qualité d'un commerce en ligne se juge à l'exploitation, sur des semaines, avec des clients — [à mesurer après trois mois].

5.7 Conclusion

Tableau 26 — Conclusion en six tuiles. Six cases, six renvois. Aucune ne dit « un futur prometteur ».

Tuile	Ce qui est affirmé	Où le vérifier
L'objet	Un commerce en ligne complet : 81 produits, 24 tables, 30 actions serveur, en service dans deux boutiques. Rien de théorique — chaque nombre de cette phrase se relit dans le dépôt.	annexe F
Le métier d'abord	Le millime, le stock partagé, le conseil sans écran. Ces trois contraintes ont écarté les places de marché, les boutiques par abonnement et quatre bibliothèques ; elles expliquent plus de code que n'importe quelle préférence d'ingénieur.	partie I
La dureté	Aucune écriture sans garde ni limite, un verrou de ligne avant tout décrétement, un paiement refusé plutôt que simulé, un journal de ce que l'administration change. La robustesse n'est pas une couche ajoutée, c'est le plan.	partie III
La preuve	Deux harnais indépendants, six rounds d'assaut, quatre injections de chaos, trois sabotages du contrôle, un déterminisme vérifié par empreinte. Ce mémoire ne demande pas d'être cru : il demande d'être rejoué.	partie V
Les limites	Pas de passerelle bancaire, pas d'arabe activé, pas de conteneur, pas de mesure d'exploitation, pas de migrations versionnées. Cinq phrases qui coûtent plus à écrire que dix paragraphes de résultats, et qui sont la seule garantie du lecteur.	§V.6
La suite	Le paiement réel dès qu'un contrat de passerelle existe, la migration versionnée dès qu'un second environnement apparaît, la mesure d'usage sur trois mois, l'activation du right-to-left avec sa relecture de maquette. Quatre chantiers, aucun commencé pour faire joli.	§V.6

Trois promesses étaient faites en introduction, et elles sont tenues mot pour mot : le verrou est l'ouvrage, et il est écrit ligne 79 de `src/lib/orders.ts` ; rien n'est affirmé sans son gisement, et l'annexe F en donne la liste ; ce qui n'a pas été fait est écrit, et la cinquième tuile le répète. Un délice, annoncé et livré : la planche de contact des 61 figures, en annexe A, qui permet de parcourir tout l'appareil graphique avant la première ligne de texte — c'est la façon la plus rapide de vérifier que ce mémoire n'a pas été écrit à l'envers de son objet.

Ce que la soutenance aura à assumer tient en une phrase : ce projet n'a pas été conduit comme une démonstration de technologie, mais comme la fabrication d'un objet dont on peut relire chaque vis. Un jury qui voudrait tester cette affirmation n'a pas besoin de croire le document — il a besoin de trois commandes, écrites en §REPRO.

5.8 Bibliographie et sources techniques

Les références ci-dessous sont les sources effectivement consultées pendant le projet, avec leur URL officielle. Elles sont citées pour ce qu'elles ont décidé dans le code, et non pour faire nombre : la colonne de droite nomme l'usage. Aucun document n'est cité sans avoir été ouvert, aucun paragraphe n'est reproduit mot à mot.

Tableau 27 — Sources consultées, avec l'usage qui en a été fait. La dernière ligne reste entre crochets : l'URL institutionnelle n'a pas pu être vérifiée depuis l'environnement de développement. Un lien incertain est marqué comme tel plutôt que donné pour sur (INV-6).

Source	URL officielle	Usage dans le projet
Documentation de Next.js	https://nextjs.org/docs	composants de serveur, actions, revalidation de cache
Documentation de React	https://react.dev/learn	cache de requête, frontière serveur/cliente
Manuel de PostgreSQL	https://www.postgresql.org/docs/manuals/	verrous de ligne, verrous advisory, niveaux d'isolation
Documentation de Drizzle ORM	https://orm.drizzle.team/docs/overview	schéma, relations, transactions, poussée du modèle
Documentation de Zod	https://zod.dev	schémas de validation des frontières
Documentation de pg (pilote)	https://node-postgres.com/	jeu de connexions, requêtes paramétrées
MDN Web Docs	https://developer.mozilla.org/	sémantique HTML, focus, attributs ARIA
Recommandations WCAG 2.2	https://www.w3.org/TR/WCAG22/	contrastes, taille des cibles, réduction du mouvement
Spécification PDF, ISO 32000-1	https://www.iso.org/standard/51502.html	générateur de facture maison, contraintes d'encodage
Directive 2011/83/UE relative aux droits des consommateurs	https://eur-lex.europa.eu/legal-content/FR/TXT/?uri=CELEX:32011L0083	droit de rétractation, mentions précontractuelles
Loi organique n° 2004-59 relative à la protection des données personnelles	[URL de l'INPDP à vérifier avant remise]	base légale, durée de conservation, registre de traitement

Les deux ouvrages de méthode et l'article sur les systèmes répartis cités en parties I et II sont laissés, eux aussi, entre crochets : [à compléter par l'auteur avant remise, avec l'édition consultée]. Le dépôt ne contient pas la liste de lectures de son auteur, et une rubrique bibliographique ne se remplit pas avec des titres plausibles.

Annexes

Le gisement : six inventaires complets

A Repères de forme · B Le modèle table par table · C Catalogue et routes · D Les vingt-quatre récits · E Protocoles et commandes · F Traçabilité.

- Annexe A — Repères de forme, palette, accessibilité
- Annexe B — Le modèle de données, table par table
- Annexe C — Catalogue, parcours, inventaire des routes
- Annexe D — Les vingt-quatre récits du backlog
- Annexe E — Protocoles de vérification et commandes
- Annexe F — Traçabilité : récit, section, figure, preuve

6.1 Annexe A — Repères de forme, palette, accessibilité

Cette annexe existe parce qu'un document qui parle de contraste se doit de montrer les siens. Les valeurs ci-dessous sont celles employées par le kit — rapport, diaporama, figures — et les rapports sont calculés à la composition par la formule de luminance relative, non repris d'une fiche de palette.

Tableau 28 — Contrastes mesurés du kit. Un accent réservé aux filets n'est pas une promesse d'accessibilité : c'est la mention de son usage. La ligne or / papier est volontairement conservée avec son rapport faible, parce qu'elle ne sert pas à porter du texte.

Premier plan	Fond	Emploi	Rapport mesuré
Encre #2B2620	Papier #FBF7EF	texte courant du rapport	14,03 : 1
Or #C9A959	Encre #2B2620	filets et titres d'intercalaire	6,64 : 1
Or #C9A959	Papier #FBF7EF	accent réservé aux filets, jamais au texte	2,11 : 1
Sauge #6F7F63	Papier #FBF7EF	état « disponible »	4,02 : 1
Rouille #A8503B	Papier #FBF7EF	état « rupture »	5,07 : 1
Clair #F3ECDD	Fond #201A13	diaporama de soutenance	14,65 : 1

Deux règles complètent la palette. Aucune information n'est portée par la couleur seule : un état de stock s'écrit avec son mot, sa forme et sa position. Aucun pictogramme n'est employé sans libellé : les 53 icônes du dépôt sont chacune accompagnées d'un texte ou marquées décoratives.

Planche de contact : les 61 figures du document, reproduites en vignettes sur une page unique, sont fournies en audit/planche-contact.png. Cet objet n'est pas une coquetterie : il permet de vérifier en dix secondes qu'aucune planche n'est vide, tronquée ou répétée, et il est régénéré à chaque build.

6.1 Glossaire des termes du projet

Les termes ci-dessous sont ceux que le jury rencontrera dans les parties I à IV. Ils sont donnés dans l'ordre du document, avec le mot employé par le code quand il diffère — parce qu'un mot qui change entre la fiche et la base est une source de litige.

Tableau 29 — Glossaire croisé mémoire et code. Seize termes, pas trente : un glossaire qui répète le dictionnaire de données ne sert qu'à allonger le document.

Terme du mémoire	Mot du code	Sens retenu ici
commande	orders	l'engagement d'achat, du panier validé à la livraison
panier	cart	la sélection non encore validée, sans effet sur le stock
client	users + role	le compte, pas la personne physique
équipe	role ADMIN ou SUPPORT	qui peut écrire quoi dans l'administration
millime	priceMillimes	le tiers du dinar, seule unité de stockage du prix
remise	promotions	le code, avec son plafond d'usage et son échéance
franco	FREE_SHIPPING_THRESHOLD	le seuil qui rend la livraison gratuite
mouvement	inventoryMovements	toute entrée, sortie, ajustement, vente ou retour
verrou	FOR UPDATE	la ligne saisie avant lecture, pour que deux ventes ne croient pas la même disponibilité
clé d'accès	accessKey	ce qui permet de suivre une commande sans compte
journal	auditLogs	la trace de ce que l'administration a changé
univers	categories.isUniverse	la porte d'entrée large du catalogue
besoin	concerns	le motif de venue du client, en langage courant
vedette	isFeatured	la mise en avant éditoriale, pas un paiement de marque
retour	returned	le statut puits de la machine à états
retrait	pickup	le seul délai mesuré en heures du catalogue

6.1 Territoire couvert par la livraison

Les 24 gouvernorats tunisiens sont tous déclarés dans le dépôt ; seuls 9 portent une liste de villes suggérées. Le délai, lui, est calculé sur une seule règle : 4 gouvernorats du Grand Tunis en vingt-quarante-huit heures, le reste en quarante-huit-soixante-douze, avec une option express à vingt-quatre heures partout et un retrait en deux heures en boutique.

Tableau 30 — Le territoire, gouvernorat par gouvernorat. Zéro gouvernorat absent, vingt-quatre lignes : la table est produite depuis `src/lib/tunisia.ts`, et une ligne en moins dans le code en enlève une ici.

Gouvernorat	Villes suggérées	Délai standard
Ariana	6	24–48 h
Béja	0	48–72 h
Ben Arous	10	24–48 h
Bizerte	4	48–72 h
Gabès	0	48–72 h
Gafsa	0	48–72 h
Jendouba	0	48–72 h
Kairouan	0	48–72 h
Kasserine	0	48–72 h
Kébili	0	48–72 h
Le Kef	0	48–72 h
Mahdia	0	48–72 h
La Manouba	4	24–48 h
Médenine	0	48–72 h
Monastir	4	48–72 h
Nabeul	6	48–72 h
Sfax	4	48–72 h
Sidi Bouzid	0	48–72 h
Siliana	0	48–72 h
Sousse	5	48–72 h
Tataouine	0	48–72 h
Tozeur	0	48–72 h
Tunis	10	24–48 h
Zaghouan	0	48–72 h

6.2 Annexe B — Le modèle de données, table par table

Cet inventaire est produit en relisant `src/db/schema.ts` : pour chaque table, le nom sql, la ligne de déclaration, le nombre de colonnes, le nombre d'index, les clés étrangères et les colonnes obligatoires. Aucun commentaire n'est ajouté, aucune ligne n'est retirée : la table soit exister, soit ne pas exister.

Tableau 31 — Les 24 tables, telles qu'écrites. Totaux relus : 24 tables, 202 colonnes, 50 index, 19 clés étrangères, 123 colonnes obligatoires, 0 contrainte de vérification. Cette dernière valeur est à zéro, et la conséquence est écrite §III.4 : les bornes de valeur sont tenues par le code, pas par la base.

Table	Ligne	Colonnes	Index	Clés étrangères	Obligatoires
users	43	9	2	0	6
sessions	60	5	2	0	2
addresses	74	11	2	0	7
brands	104	6	1	0	3
categories	118	9	3	0	4
concerns	139	4	1	0	2
products	151	24	8	3	13
product_concerns	192	2	1	0	0
reviews	205	9	2	1	4
inventory_movements	224	9	2	0	4
promotions	242	14	2	1	8
orders	265	24	7	1	13
order_items	322	10	2	1	5
order_events	342	6	1	0	2
wishlist_items	357	3	0	0	1
articles	372	10	2	0	6
stores	390	9	0	0	7
search_events	404	5	2	0	3
analytics_events	416	5	2	0	3
audit_logs	428	7	2	0	4
support_tickets	442	9	1	1	5
loyalty_transactions	459	6	4	0	3
newsletter_subscribers	485	3	0	0	2
rate_limits	497	3	1	0	2

Les 10 types énumérés méritent leur propre tableau, parce qu'ils sont la partie du modèle la plus souvent sous-estimée en relecture : un statut non borné est un statut qui dérive. Les valeurs sont recopiées depuis le schéma à la composition — une valeur ajoutée dans le code apparaît ici au build suivant, et nulle part ailleurs.

Tableau 32 — Les dix enums du schéma. Colonne « ligne » : la ligne de déclaration dans `src/db/schema.ts`, relue par `facts.py`.

Type énuméré	Valeurs	Nombre	Ligne
movement_type	in, out, adjust, sale, restock, return	6	34
order_status	pending, confirmed, preparing, shipped, delivered, cancelled, returned	7	20
payment_method	cod, bank_transfer, card, gift_card	4	29
payment_status	pending, paid, refunded, failed	4	30
product_status	draft, active, archived	3	19
promo_type	percent, fixed, free_shipping	3	32
review_status	pending, approved, rejected	3	33
shipping_method	standard, express, pickup	3	31
ticket_status	open, answered, closed	3	35
user_role	customer, support, admin	3	18

Le dictionnaire de données suit : une ligne par colonne, avec son type, ses contraintes et le type énuméré éventuel. Il est produit en relisant le schéma, ce qui le rend plus exact qu'un tableau recopié d'une documentation — et surtout, il change tout seul quand le schéma bouge.

6.2 Dictionnaire des colonnes

Tableau 33 — Les 202 colonnes du modèle, une par ligne. Le tableau est découpé en portions de quarante-six lignes avec répétition de l'en-tête, pour qu'aucune ligne ne soit tronquée ni coupée au milieu. (suite ci-après)

Table	Colonne	Type	Contraintes	Enum	Référence
users	id	serial	PK	—	—
users	email	varchar	non NULL	—	—
users	passwordHash	text	non NULL	—	—
users	firstName	varchar	non NULL	—	—
users	lastName	varchar	non NULL	—	—
users	phone	varchar	—	—	—
users	role	userRoleEnum	non NULL	—	—
users	loyaltyPoints	integer	non NULL	—	—
users	notes	text	—	—	—
sessions	id	varchar	PK	—	—
sessions	userId	integer	—	—	—
sessions	expiresAt	timestamp	non NULL	—	—
sessions	userAgent	text	—	—	—
sessions	createdAt	timestamp	non NULL, now()	—	—
addresses	id	serial	PK	—	—
addresses	userId	integer	—	—	—
addresses	label	varchar	non NULL	—	—
addresses	fullName	varchar	non NULL	—	—
addresses	phone	varchar	non NULL	—	—
addresses	line1	varchar	non NULL	—	—
addresses	line2	varchar	—	—	—
addresses	city	varchar	non NULL	—	—
addresses	governorate	varchar	non NULL	—	—
addresses	postalCode	varchar	—	—	—
addresses	isDefault	boolean	non NULL	—	—
brands	id	serial	PK	—	—
brands	slug	varchar	non NULL	—	—
brands	name	varchar	non NULL	—	—
brands	country	varchar	—	—	—
brands	story	text	—	—	—
brands	isFeatured	boolean	non NULL	—	—
categories	id	serial	PK	—	—
categories	slug	varchar	non NULL	—	—
categories	name	varchar	non NULL	—	—
categories	description	text	—	—	—
categories	story	text	—	—	—
categories	image	varchar	—	—	—
categories	isUniverse	boolean	non NULL	—	—
categories	parentId	integer	—	—	—
categories	sortOrder	integer	non NULL	—	—
concerns	id	serial	PK	—	—
concerns	slug	varchar	non NULL	—	—
concerns	name	varchar	non NULL	—	—
concerns	intro	text	—	—	—
products	id	serial	PK	—	—
products	slug	varchar	non NULL	—	—

suite du tableau 33

Table	Colonne	Type	Contraintes	Enum	Référence
products	sku	varchar	non NULL	–	–
products	name	varchar	non NULL	–	–
products	shortDescription	varchar	–	–	–
products	description	text	–	–	–
products	ingredients	text	–	–	–
products	howToUse	text	–	–	–
products	brandId	integer	–	–	brands.id
products	categoryId	integer	–	–	categories.id
products	universeId	integer	–	–	categories.id
products	priceMillimes	integer	non NULL	–	–
products	compareAtMillimes	integer	–	–	–
products	stock	integer	non NULL	–	–
products	lowStockThreshold	integer	non NULL	–	–
products	image	varchar	–	–	–
products	images	jsonb	non NULL	–	–
products	volume	varchar	–	–	–
products	status	productStatusEnum	non NULL	–	–
products	isFeatured	boolean	non NULL	–	–
products	isNew	boolean	non NULL	–	–
products	ratingAvg	integer	non NULL	–	–
products	ratingCount	integer	non NULL	–	–
products	salesCount	integer	non NULL	–	–
product_concerns	productId	integer	–	–	–
product_concerns	concernId	integer	–	–	–
reviews	id	serial	PK	–	–
reviews	productId	integer	–	–	–
reviews	userId	integer	–	–	users.id
reviews	authorName	varchar	non NULL	–	–
reviews	rating	integer	non NULL	–	–
reviews	title	varchar	–	–	–
reviews	body	text	non NULL	–	–
reviews	status	reviewStatusEnum	non NULL	–	–
reviews	reply	text	–	–	–
inventory_movements	id	serial	PK	–	–
inventory_movements	productId	integer	–	–	–
inventory_movements	type	movementTypeEnum	non NULL	–	–
inventory_movements	quantity	integer	non NULL	–	–
inventory_movements	stockAfter	integer	non NULL	–	–
inventory_movements	reason	varchar	–	–	–
inventory_movements	orderId	integer	–	–	–
inventory_movements	userId	integer	–	–	–
inventory_movements	createdAt	timestamp	non NULL, now()	–	–
promotions	id	serial	PK	–	–
promotions	code	varchar	non NULL	–	–
promotions	label	varchar	non NULL	–	–
promotions	type	promoTypeEnum	non NULL	–	–

suite du tableau 33

Table	Colonne	Type	Contraintes	Enum	Référence
promotions	value	integer	non NULL	—	—
promotions	minSubtotalMillimes	integer	non NULL	—	—
promotions	maxDiscountMillimes	integer	—	—	—
promotions	universeId	integer	—	—	categories.id
promotions	usageLimit	integer	—	—	—
promotions	usageCount	integer	non NULL	—	—
promotions	perUserLimit	integer	non NULL	—	—
promotions	startsAt	timestamp	—	—	—
promotions	endsAt	timestamp	—	—	—
promotions	isActive	boolean	non NULL	—	—
orders	id	serial	PK	—	—
orders	number	varchar	non NULL	—	—
orders	idempotencyKey	varchar	—	—	—
orders	accessKey	varchar	—	—	—
orders	userId	integer	—	—	users.id
orders	email	varchar	non NULL	—	—
orders	phone	varchar	non NULL	—	—
orders	status	orderStatusEnum	non NULL	—	—
orders	paymentMethod	paymentMethodEnum	non NULL	—	—
orders	paymentStatus	paymentStatusEnum	non NULL	—	—
orders	shippingMethod	shippingMethodEnum	non NULL	—	—
orders	shippingAddress	jsonb	—	—	—
orders	storeId	integer	—	—	—
orders	subtotalMillimes	integer	non NULL	—	—
orders	discountMillimes	integer	non NULL	—	—
orders	shippingMillimes	integer	non NULL	—	—
orders	giftWrapMillimes	integer	non NULL	—	—
orders	totalMillimes	integer	non NULL	—	—
orders	promoCode	varchar	—	—	—
orders	giftWrap	boolean	non NULL	—	—
orders	giftMessage	varchar	—	—	—
orders	customerNote	text	—	—	—
orders	internalNote	text	—	—	—
orders	trackingCode	varchar	—	—	—
order_items	id	serial	PK	—	—
order_items	orderId	integer	—	—	—
order_items	productId	integer	—	—	products.id
order_items	name	varchar	non NULL	—	—
order_items	sku	varchar	non NULL	—	—
order_items	brandName	varchar	—	—	—
order_items	image	varchar	—	—	—
order_items	unitPriceMillimes	integer	non NULL	—	—
order_items	quantity	integer	non NULL	—	—
order_items	lineTotalMillimes	integer	non NULL	—	—
order_events	id	serial	PK	—	—
order_events	orderId	integer	—	—	—

suite du tableau 33

Table	Colonne	Type	Contraintes	Enum	Référence
order_events	status	orderStatusEnum	non NULL	—	—
order_events	message	varchar	—	—	—
order_events	actorId	integer	—	—	—
order_events	createdAt	timestamp	non NULL, now()	—	—
wishlist_items	userId	integer	—	—	—
wishlist_items	productId	integer	—	—	—
wishlist_items	createdAt	timestamp	non NULL, now()	—	—
articles	id	serial	PK	—	—
articles	slug	varchar	non NULL	—	—
articles	title	varchar	non NULL	—	—
articles	excerpt	varchar	—	—	—
articles	body	text	non NULL	—	—
articles	image	varchar	—	—	—
articles	tag	varchar	—	—	—
articles	readMinutes	integer	non NULL	—	—
articles	isPublished	boolean	non NULL	—	—
articles	publishedAt	timestamp	non NULL, now()	—	—
stores	id	serial	PK	—	—
stores	slug	varchar	non NULL, UNIQUE	—	—
stores	name	varchar	non NULL	—	—
stores	address	varchar	non NULL	—	—
stores	city	varchar	non NULL	—	—
stores	phone	varchar	non NULL	—	—
stores	hours	varchar	non NULL	—	—
stores	mapsUrl	varchar	—	—	—
stores	isActive	boolean	non NULL	—	—
search_events	id	serial	PK	—	—
search_events	query	varchar	non NULL	—	—
search_events	resultsCount	integer	non NULL	—	—
search_events	userId	integer	—	—	—
search_events	createdAt	timestamp	non NULL, now()	—	—
analytics_events	id	serial	PK	—	—
analytics_events	name	varchar	non NULL	—	—
analytics_events	payload	jsonb	non NULL	—	—
analytics_events	userId	integer	—	—	—
analytics_events	createdAt	timestamp	non NULL, now()	—	—
audit_logs	id	serial	PK	—	—
audit_logs	actorId	integer	—	—	—
audit_logs	action	varchar	non NULL	—	—
audit_logs	entity	varchar	non NULL	—	—
audit_logs	entityId	varchar	—	—	—
audit_logs	details	jsonb	non NULL	—	—
audit_logs	createdAt	timestamp	non NULL, now()	—	—
support_tickets	id	serial	PK	—	—
support_tickets	userId	integer	—	—	users.id
support_tickets	email	varchar	non NULL	—	—

suite du tableau 33

Table	Colonne	Type	Contraintes	Enum	Référence
support_tickets	name	varchar	non NULL	–	–
support_tickets	subject	varchar	non NULL	–	–
support_tickets	message	text	non NULL	–	–
support_tickets	reply	text	–	–	–
support_tickets	status	ticketStatusEnum	non NULL	–	–
support_tickets	orderNumber	varchar	–	–	–
loyalty_transactions	id	serial	PK	–	–
loyalty_transactions	userId	integer	–	–	–
loyalty_transactions	points	integer	non NULL	–	–
loyalty_transactions	reason	varchar	non NULL	–	–
loyalty_transactions	orderId	integer	–	–	–
loyalty_transactions	createdAt	timestamp	non NULL, now()	–	–
newsletter_subscribers	id	serial	PK	–	–
newsletter_subscribers	email	varchar	non NULL, UNIQUE	–	–
newsletter_subscribers	createdAt	timestamp	non NULL, now()	–	–
rate_limits	key	varchar	PK	–	–
rate_limits	count	integer	non NULL	–	–
rate_limits	resetAt	timestamp	non NULL	–	–

6.2 Inventaire des index

Les 50 index du schéma, table par table, dans l'ordre de déclaration. Un index sans requête qui le consomme est du poids à l'écriture ; un chemin d'accès manquant est une requête qui lit toute la table. Cette liste est le lieu où le lecteur peut trancher la question lui-même.

Tableau 34 — Les 50 index déclarés. 50 lignes, 21 tables indexées. La nature distingue uniqueIndex de index : le premier porte une contrainte d'unicité, le second seulement une vitesse. (suite ci-après)

Table	Nature	Nom
users	uniqueIndex	users_email_idx
users	index	users_role_idx
sessions	index	sessions_user_idx
sessions	index	sessions_expires_idx
addresses	index	addresses_user_idx
addresses	uniqueIndex	addresses_one_default_idx
brands	uniqueIndex	brands_slug_idx
categories	uniqueIndex	categories_slug_idx
categories	index	categories_parent_idx
categories	index	categories_universe_idx
concerns	uniqueIndex	concerns_slug_idx
products	uniqueIndex	products_slug_idx
products	uniqueIndex	products_sku_idx
products	index	products_brand_idx
products	index	products_category_idx
products	index	products_universe_idx
products	index	products_status_idx
products	index	products_price_idx
products	index	products_featured_idx
product_concerns	index	pc_concern_idx
reviews	index	reviews_product_idx
reviews	index	reviews_status_idx
inventory_movements	index	inv_product_idx
inventory_movements	index	inv_created_idx
promotions	uniqueIndex	promotions_code_idx
promotions	index	promotions_active_idx
orders	uniqueIndex	orders_number_idx
orders	uniqueIndex	orders_idem_idx
orders	uniqueIndex	orders_access_key_idx
orders	index	orders_user_idx
orders	index	orders_status_idx
orders	index	orders_created_idx
orders	index	orders_email_idx
order_items	index	order_items_order_idx
order_items	index	order_items_product_idx
order_events	index	order_events_order_idx
articles	uniqueIndex	articles_slug_idx
articles	index	articles_published_idx
search_events	index	search_query_idx
search_events	index	search_created_idx

suite du tableau 34

Table	Nature	Nom
analytics_events	index	analytics_name_idx
analytics_events	index	analytics_created_idx
audit_logs	index	audit_entity_idx
audit_logs	index	audit_created_idx
support_tickets	index	tickets_status_idx
loyalty_transactions	index	loyalty_user_idx
loyalty_transactions	index	loyalty_order_idx
loyalty_transactions	uniqueIndex	loyalty_order_award_idx
loyalty_transactions	uniqueIndex	loyalty_order_reversal_idx
rate_limits	index	rate_limits_reset_idx

Quatre tables sont le siège des garde-fous cités en partie III, et leurs noms méritent d'être écrits une fois : orders (numéro unique, clé d'accès, clé d'idempotence unique, montants figés), order_items (relevé de prix, pas référence), stock_movements (journal du stock, jamais écrasé) et audit_logs (append-only, aucune interface de suppression). Le détail est dans l'inventaire ci-dessus ; la raison est aux §III.3 et §III.5.

6.2 L'échelle typographique du document

Tableau 35 — L'échelle employée par le moteur de composition. Une seule famille serif pour les titres, une sans-serif pour les corps, une monospace pour le code : trois polices, pas une de plus, parce que chaque police embarquée pèse sur le budget de poids du fichier.

Emploi	Famille	Corps / interlignage (pt)	Particularité
Couverture — titre	DejaVuSerif	26 / 30	or sur encre
Intercalaire — numéro	DejaVuSerif-Bold	34 / 36	or
Intercalaire — titre	DejaVuSerif	17 / 21	papier
Titre de section	DejaVuSerif-Bold	13,5 / 16	encre
Sous-titre	DejaVuSerif-Bold	10,8 / 13	encre
Chapeau	DejaVu	9,4 / 13,4	encre, retrait gauche
Corps	DejaVu	9,1 / 12,8	encre, justifié
Liste	DejaVu	8,9 / 12,4	puce or
Légende de planche	DejaVu	7,6 / 10,2	gris
Tableau	DejaVu	7,1 / 9,2	encre
Extrait de code	DejaVuSansMono	6,7 / 8,9	fond papier, filet
Sommaire	DejaVu	8,3 / 11,4	lien cliquable

Trois mesures ont été nécessaires pour écrire ce qui précède, et elles contredisent trois idées reçues dans ce dépôt. La première : l'espace insécable fine (U+202F), que DejaVu possède et que l'usage français recommande, est traitée par le moteur de paragraphes comme une espace ordinaire — la césure revient. Le rendu utilise donc U+00A0, et le contrôle accepte les deux, parce que ce qui est en jeu est l'absence de césure, pas le numéro de caractère. La seconde : le calque texte d'un PDF normalise l'insécable en espace, si bien qu'une expression régulière posée sur le texte extrait ne voit rien ; la loi est donc vérifiée sur les lignes reconstituées géométriquement, span par span, avec exemption déclarée pour les spans monospace. La troisième : échapper le texte « pour protéger le lecteur du balisage » détruit la mise en forme de l'auteur ; l'échappement préserve exactement la liste des balises d'emphase, et le verbatim passe par un échappement strict.

6.3 Annexe C — Catalogue, parcours, inventaire des routes

Le catalogue en chiffres, calculés depuis le script d'ensemencement — ce qui veut dire que ce que le client verra est ce qui est écrit ici, et non ce qu'un export a déclaré un jour.

Tableau 36 — Le catalogue, mesuré. Les montants sont des entiers de millimes dans la base et sont formatés ici en dinars tunisiens à trois décimales, sans arrondi intermédiaire. Les deux lignes de stock sont un état du fichier de référence : le stock réel appartient à l'officine, et le document ne l'invente pas.

Grandeur	Valeur	Détail
produits	81	16 marques, 7 univers
prix le plus bas	0,015 DT	le plus petit montant du catalogue
prix le plus haut	0,249 DT	le plus grand montant du catalogue
prix courant	0,052 DT	médiane, pas moyenne — le catalogue est asymétrique
en promotion	17	remise moyenne 10.7 %
sans visuel	0	retombée sur l'image d'univers
en rupture à la préparation	8	état du fichier de référence, pas un état de stock réel
faible stock à la préparation	10	même réserve
répartition par univers	visage 26, corps 13, solaire 12, cheveux 11, complements 7, bebe-maman 6, hygiene 6	le visage pèse le tiers du catalogue
cinq premières marques	la-roche-posay 15, bioderma 8, avene 8, vichy 7, arkopharma 7	sur 16 marques

L'inventaire des pages ferme la question « qu'est-ce qui est publié ? ». La liste est produite en parcourant src/app, avec le fichier source de chaque route ; elle inclut les pages d'information, qui ne sont pas des pages de commerce.

Tableau 37 — Les 43 pages, route par route. La colonne « rendu » distingue une page dont les données viennent de la base à chaque visite d'une page figée à la construction. Chaque ligne se vérifie en ouvrant le fichier cité.

Route	Fichier	Rendu
/aide	src/app/(site)/aide/page.tsx	statique
/besoin/:slug	src/app/(site)/besoin/[slug]/page.tsx	dynamique
/boutique	src/app/(site)/boutique/page.tsx	statique
/boutiques	src/app/(site)/boutiques/page.tsx	statique
/categorie/:slug	src/app/(site)/categorie/[slug]/page.tsx	dynamique
/cgv	src/app/(site)/cgv/page.tsx	statique
/commande/confirmation/:number	src/app/(site)/commande/confirmation/[number]/page.tsx	statique
/commande	src/app/(site)/commande/page.tsx	statique
/compte/commandes/:number	src/app/(site)/compte/commandes/[number]/page.tsx	statique
/compte/commandes	src/app/(site)/compte/commandes/page.tsx	statique
/compte/favoris	src/app/(site)/compte/favoris/page.tsx	statique
/compte	src/app/(site)/compte/page.tsx	statique
/compte/profil	src/app/(site)/compte/profil/page.tsx	statique
/confidentialite	src/app/(site)/confidentialite/page.tsx	statique
/connexion	src/app/(site)/connexion/page.tsx	statique
/inscription	src/app/(site)/inscription/page.tsx	statique
/journal/:slug	src/app/(site)/journal/[slug]/page.tsx	dynamique
/journal	src/app/(site)/journal/page.tsx	statique
/livraison	src/app/(site)/livraison/page.tsx	statique
/marque/:slug	src/app/(site)/marque/[slug]/page.tsx	dynamique
/marques	src/app/(site)/marques/page.tsx	statique
	src/app/(site)/page.tsx	statique
/panier	src/app/(site)/panier/page.tsx	statique
/produit/:slug	src/app/(site)/produit/[slug]/page.tsx	dynamique
/promotions	src/app/(site)/promotions/page.tsx	statique
/recherche	src/app/(site)/recherche/page.tsx	statique
/suivi	src/app/(site)/suivi/page.tsx	statique
/univers/:slug	src/app/(site)/univers/[slug]/page.tsx	dynamique
/admin/audit	src/app/admin/audit/page.tsx	statique
/admin/avis	src/app/admin/avis/page.tsx	statique
/admin/boutiques	src/app/admin/boutiques/page.tsx	statique
/admin/clients/:id	src/app/admin/clients/[id]/page.tsx	statique
/admin/clients	src/app/admin/clients/page.tsx	statique
/admin/commandes/:id	src/app/admin/commandes/[id]/page.tsx	statique
/admin/commandes	src/app/admin/commandes/page.tsx	statique
/admin/journal	src/app/admin/journal/page.tsx	statique
/admin	src/app/admin/page.tsx	statique
/admin/produits/:id	src/app/admin/produits/[id]/page.tsx	statique
/admin/produits	src/app/admin/produits/page.tsx	statique
/admin/promotions	src/app/admin/promotions/page.tsx	statique
/admin/recherches	src/app/admin/recherches/page.tsx	statique
/admin/stock	src/app/admin/stock/page.tsx	statique
/admin/support	src/app/admin/support/page.tsx	statique

Tableau 38 — Les 5 points d'entrée de l'API. Cinq points d'entrée pour un site qui n'a pas d'application mobile : la surface exposée est volontairement petite.

Méthode	Chemin	Fichier
GET	/api/api/admin/export/:kind	src/app/api/admin/export/[kind]/route.ts
GET	/api/api/health	src/app/api/health/route.ts
GET	/api/api/orders/:number/invoice	src/app/api/orders/[number]/invoice/route.ts
GET	/api/api/products	src/app/api/products/route.ts
GET	/api/api/search	src/app/api/search/route.ts

Le catalogue intégral, tel que le script d'ensemencement l'écrit. Cette liste est le contrôle le plus brut du mémoire : elle compare, produit par produit, ce que le fichier de référence annonce et ce que la base reçoit.

6.3 Catalogue complet

Tableau 39 — Les 81 références du catalogue. Sept colonnes, quatre-vingt-une lignes, zéro produit inventé : la liste est celle que src/db/seed.ts insère, lue ligne à ligne. (suite ci-après)

Produit	Marque	Univers	Rayon	Prix (DT)	En promo	Vedette
Effaclar Gel Moussant Purifiant	la-roche-posay	visage	nettoyants-demaquillants	42,900	oui	oui
Toleriane Dermo-Nettoyant	la-roche-posay	visage	nettoyants-demaquillants	49,900	non	non
Sensibio H2O Eau Micellaire	bioderma	visage	nettoyants-demaquillants	38,500	oui	oui
Sébium Gel Moussant	bioderma	visage	nettoyants-demaquillants	34,900	non	non
Vinoclean Mousse Nettoyante	caudalie	visage	nettoyants-demaquillants	52,000	non	non
Hyalu B5 Sérum	la-roche-posay	visage	serums	129,000	oui	oui
Minéral 89 Booster Quotidien	vichy	visage	serums	89,900	non	oui
Vinoperfect Sérum Éclat	caudalie	visage	serums	158,000	non	non
Pure Vitamin C10 Sérum	la-roche-posay	visage	serums	119,000	non	non
Ampoules Concentrées C+	svr	visage	serums	98,000	oui	non
NCEF-Reverse Crème Suprême	filorga	visage	anti-age	249,000	non	oui
Time-Filler Crème Anti-rides	filorga	visage	anti-age	189,000	oui	non
Liftactiv Collagen Specialist	vichy	visage	anti-age	135,000	non	non
Resveratrol-Lift Crème Cachemire	caudalie	visage	anti-age	149,000	non	non
Hydrance Aqua-Gel	avene	visage	hydratants	69,900	non	non
Toleriane Sensitive Crème	la-roche-posay	visage	hydratants	59,900	non	non
Crème Hydratante Visage	cerave	visage	hydratants	47,900	oui	non
Crème Prodigueuse Boost Gel-Baume	nuxe	visage	hydratants	84,000	non	non
Aquaphor Baume Réparateur	eucerin	visage	hydratants	39,900	non	non
Physiolift Yeux	avene	visage	contour-des-yeux	79,000	non	non
Hyalu B5 Yeux	la-roche-posay	visage	contour-des-yeux	92,000	non	non
Effaclar Duo+ M	la-roche-posay	visage	peaux-a-imperfections	62,900	oui	oui
Sébium Global	bioderma	visage	peaux-a-imperfections	58,000	non	non
Cleanance Comedomed	avene	visage	peaux-a-imperfections	61,000	non	non
Normaderm Phytosolution	vichy	visage	peaux-a-imperfections	64,900	non	non
Sebiaclear Sérum	svr	visage	peaux-a-imperfections	71,000	non	non
Lipikar Baume AP+M	la-roche-posay	corps	hydratants-corps	79,900	oui	oui
XeraCalm A.D Crème Relipidante	avene	corps	hydratants-corps	74,000	non	non
Atoderm Intensive Baume	bioderma	corps	hydratants-corps	69,500	non	non
Huile Prodigueuse	nuxe	corps	hydratants-corps	92,000	non	oui
Lotion Hydratante Corps	cerave	corps	hydratants-corps	49,900	non	non
Bariéderm Cica-Crème	uriage	corps	hydratants-corps	32,000	non	non
Lipikar Syndet AP+	la-roche-posay	corps	douche-bain	44,900	non	non
Atoderm Huile de Douche	bioderma	corps	douche-bain	52,000	oui	non
Rêve de Miel Gel Douche Ultra-Riche	nuxe	corps	douche-bain	38,000	non	non
Crème Mains Cicaplast	la-roche-posay	corps	mains-pieds	22,900	non	non
Crème Mains Rêve de Miel	nuxe	corps	mains-pieds	24,500	non	non
Crème Pieds Réparatrice Urea 10%	eucerin	corps	mains-pieds	34,000	non	non
Huile Vergetures Bio	mustela	corps	vergetures-fermete	64,000	non	non
Shampooing à l'Avoine	klorane	cheveux	shampooings	29,900	non	oui
Anaphase+ Shampooing	ducray	cheveux	shampooings	41,000	oui	non
Dercos Anti-Pelliculaire DS	vichy	cheveux	shampooings	44,900	non	non
Kelual DS Shampooing	ducray	cheveux	shampooings	39,000	non	non
Shampooing au Lait de Papyrus	klorane	cheveux	shampooings	31,500	non	non

suite du tableau 39

Produit	Marque	Univers	Rayon	Prix (DT)	En promo	Vedette
Baume Après-Shampooing à la Mangue	klorane	cheveux	apres-shampooings-masques	34,900	non	non
Masque Nutri-Réparateur	nuxe	cheveux	apres-shampooings-masques	58,000	oui	non
Dercos Aminexil Clinical 5 Femme	vichy	cheveux	anti-chute	189,000	oui	oui
Neoptide Expert Sérum	ducray	cheveux	anti-chute	165,000	non	non
Forcapil Cheveux & Ongles	arkopharma	cheveux	anti-chute	59,000	non	non
Sensinol Shampooing Physioprotecteur	ducray	cheveux	cuir-chevelu-sensible	36,000	non	non
Anthelios UVMune 400 Fluide Invisible SPF50+	la-roche-posay	solaire	protection-visage	68,900	oui	oui
Fotoprotector Fusion Water SPF50	isdin	solaire	protection-visage	74,000	non	non
Photoderm Nude Touch SPF50+	bioderma	solaire	protection-visage	62,000	non	non
Capital Soleil UV-Age Daily SPF50+	vichy	solaire	protection-visage	66,500	non	non
Vinosun Crème Protectrice SPF50	caudalie	solaire	protection-visage	79,000	non	non
Anthelios Lait Hydratant SPF50+	la-roche-posay	solaire	protection-corps	82,000	oui	non
Fluide Minéral Corps SPF50+	avene	solaire	protection-corps	76,000	non	non
Photoderm Spray SPF50+	bioderma	solaire	protection-corps	69,000	non	non
Posthelios Gel-Crème Après-Soleil	la-roche-posay	solaire	apres-soleil	42,000	non	non
Lait Après-Soleil Réparateur	avene	solaire	apres-soleil	39,900	non	non
Anthelios Dermo-Pediatrics Lait SPF50+	la-roche-posay	solaire	enfants	84,000	non	non
Pediatrics Fusion Water SPF50	isdin	solaire	enfants	72,000	non	non
Gel Lavant Doux	mustela	bebe-maman	toilette-bebe	31,900	oui	oui
Eau Nettoyante Sans Rinçage	mustela	bebe-maman	toilette-bebe	28,500	non	non
ABCDerm Moussant	bioderma	bebe-maman	toilette-bebe	33,000	non	non
Crème Change 1-2-3	mustela	bebe-maman	change	24,900	non	non
Cicalfate+ Crème Réparatrice	avene	bebe-maman	change	27,000	non	non
Baume Corps Maternité	mustela	bebe-maman	soins-maman	58,000	non	non
Arkovital Pure Energy	arkopharma	complements	vitalite-immunite	38,000	non	non
Vitamine D3 2000 UI	arkopharma	complements	vitalite-immunite	27,500	non	non
Arkogélules Ginseng Bio	arkopharma	complements	vitalite-immunite	32,000	oui	non
Arkorelax Sommeil Fort 8h	arkopharma	complements	sommeil-stress	44,000	non	non
Arkogélules Magnésium Marin	arkopharma	complements	sommeil-stress	29,900	non	non
Skin Booster Collagène	filorga	complements	beaute-in-out	79,000	non	non
Arkogélules Charbon Végétal	arkopharma	complements	digestion	24,000	non	non
Dentifrice Sensibilité	eucerin	hygiene	bucco-dentaire	14,900	non	non
Gyn-Phy Gel Intime	uriage	hygiene	hygiene-intime	26,000	non	non
Déodorant 48h Anti-Traces	vichy	hygiene	deodorants	31,500	oui	non
Déodorant Roll-on Douceur	avene	hygiene	deodorants	27,900	non	non
Cicaplast Baume B5+	la-roche-posay	hygiene	premiers-soins	32,900	non	oui
Bariéderm Cica Spray	uriage	hygiene	premiers-soins	29,000	non	non

6.3 Le plan de rayon complet

Sept univers, 29 sous-rayons déclarés dans le script d'ensemencement, 36 lignes de catégorie au total dans la base (les univers sont stockés dans la même table, marqués par l'attribut isUniverse). Le plan est donné tel quel : c'est lui qui décide de la profondeur de la navigation.

Tableau 40 — Le plan de rayon, univers par univers. Un univers sans sous-rayon reste navigable : la table est une arborescence, pas une obligation de profondeur.

Univers	Sous-rayons
Visage	Nettoyants & démaquillants, Sérums, Hydratants, Anti-âge, Contour des yeux, Peaux à imperfections
Corps	Hydratants corps, Douche & bain, Mains & pieds, Vergetures & fermeté
Cheveux	Shampooings, Après-shampooings & masques, Anti-chute, Cuir chevelu sensible
Solaire	Protection visage, Protection corps, Après-soleil, Enfants
Bébé & Maman	Toilette bébé, Change, Soins maman
Compléments	Vitalité & immunité, Sommeil & stress, Beauté in & out, Digestion
Hygiène & Bien-être	Bucco-dentaire, Hygiène intime, Déodorants, Premiers soins

6.4 Annexe D — Les vingt-quatre récits du backlog

Le backlog intégral, tel qu'il a été écrit en S0 et tel qu'il a été tenu. Vingt-quatre récits, six épopées, et pour chacun ce qu'il reçoit, ce qu'il rend, le sprint qui l'a livré, la section du mémoire qui en traite, la planche qui l'illustre, le fichier qui le réalise et la vérification qui le tient. Les deux colonnes d'estimation sont déclaratives (échelle 0 à 1) : elles servent à ordonner le travail, pas à rendre un compte d'heures.

6.4 US-01 — Créer un compte client

Tableau 41 — Fiche US-01. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E1 · Identité & accès
Sprint	S1
Reçoit	e-mail, mot de passe, nom, téléphone
Rend	session ouverte, 0 point de fidélité
Valeur estimée	0,62 (échelle déclarée 0-1)
Effort estimé	0,34 (même échelle)
Section du mémoire	III.2
Planches	fig22, fig26
Fichiers	<code>src/actions/auth.ts</code> , <code>src/lib/validation.ts</code>
Symboles	<code>registerAction</code> , <code>registerSchema</code>
Vérification	<code>qa.py:check_action_guarded</code>
Glissade du diaporama	11

6.4 US-02 — Se connecter et sortir

Tableau 42 — Fiche US-02. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E1 · Identité & accès
Sprint	S1
Reçoit	identifiants
Rend	cookie de session httpOnly
Valeur estimée	0,90 (échelle déclarée 0-1)
Effort estimé	0,30 (même échelle)
Section du mémoire	II.4
Planches	fig30, fig22
Fichiers	src/lib/auth.ts, src/actions/auth.ts
Symboles	createSession, loginAction
Vérification	qa.py:check_session_cookie_flags
Glissade du diaporama	16a

6.4 US-03 — Modifier son profil et son mot de passe

Tableau 43 — Fiche US-03. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E1 · Identité & accès
Sprint	S2
Reçoit	nom, téléphone, ancien mot de passe
Rend	hash recalculé, sessions conservées
Valeur estimée	0,48 (échelle déclarée 0-1)
Effort estimé	0,32 (même échelle)
Section du mémoire	III.5
Planches	fig30
Fichiers	src/actions/auth.ts, src/lib/auth.ts
Symboles	updateProfileAction, hashPassword
Vérification	qa.py:check_scrypt_params
Glissade du diaporama	16b

6.4 US-04 — Gérer ses adresses de livraison

Tableau 44 — Fiche US-04. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E1 · Identité & accès
Sprint	S2
Reçoit	adresse + gouvernorat
Rend	adresse par défaut unique
Valeur estimée	0,44 (échelle déclarée 0-1)
Effort estimé	0,36 (même échelle)
Section du mémoire	III.5
Planches	fig29, fig22
Fichiers	src/actions/auth.ts, src/lib/tunisia.ts
Symboles	saveAddressAction, GOVERNORATES
Vérification	qa.py:check_governorates
Glissade du diaporama	11

6.4 US-05 — Parcourir le catalogue par univers

Tableau 45 — Fiche US-05. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E2 · Catalogue & découverte
Sprint	S1
Reçoit	slug d'univers
Rend	grille paginée, facettes
Valeur estimée	0,86 (échelle déclarée 0-1)
Effort estimé	0,28 (même échelle)
Section du mémoire	III.1
Planches	fig10, fig33
Fichiers	<code>src/app/(site)/univers/[slug]/page.tsx</code> , <code>src/lib/catalog.ts</code>
Symboles	<code>listProducts</code> , <code>getUniverses</code>
Vérification	<code>qa.py:check_univers_counts</code>
Glissade du diaporama	11

6.4 US-06 — Chercher un produit

Tableau 46 — Fiche US-06. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E2 · Catalogue & découverte
Sprint	S2
Reçoit	requête texte
Rend	produits + pages, événement de recherche
Valeur estimée	0,70 (échelle déclarée 0-1)
Effort estimé	0,45 (même échelle)
Section du mémoire	III.3
Planches	fig22, fig53
Fichiers	<code>src/app/api/search/route.ts</code> , <code>src/lib/catalog.ts</code>
Symboles	<code>quickSearch</code>
Vérification	<code>qa.py:check_search_events_table</code>
Glissade du diaporama	11

6.4 US-07 — Voir une fiche produit complète

Tableau 47 — Fiche US-07. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E2 · Catalogue & découverte
Sprint	S1
Reçoit	slug produit
Rend	prix, stock, avis, routine, composition
Valeur estimée	0,92 (échelle déclarée 0-1)
Effort estimé	0,40 (même échelle)
Section du mémoire	III.1
Planches	figR1, fig26
Fichiers	<code>src/app/(site)/produit/[slug]/page.tsx</code> , <code>src/db/schema.ts</code>
Symboles	<code>products</code> , <code>getProductBySlug</code>
Vérification	<code>qa.py:check_product_columns</code>
Glissade du diaporama	12

6.4 US-08 — Filtrer par besoin de peau

Tableau 48 — Fiche US-08. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E2 · Catalogue & découverte
Sprint	S3
Reçoit	11 besoins
Rend	sélection croisée univers × besoin
Valeur estimée	0,58 (échelle déclarée 0-1)
Effort estimé	0,50 (même échelle)
Section du mémoire	III.1
Planches	fig03, fig34
Fichiers	<code>src/db/seed.ts</code> , <code>src/components/catalog/filters.tsx</code>
Symboles	<code>concerns</code>
Vérification	<code>qa.py:check_concerns_coverage</code>
Glissade du diaporama	12

6.4 US-09 — Découvrir les marques et leur histoire

Tableau 49 — Fiche US-09. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E2 · Catalogue & découverte
Sprint	S3
Reçoit	16 marques
Rend	pages marque + vitrine
Valeur estimée	0,40 (échelle déclarée 0-1)
Effort estimé	0,25 (même échelle)
Section du mémoire	III.1
Planches	fig34, fig10
Fichiers	<code>src/app/(site)/marques/page.tsx</code> , <code>src/db/seed.ts</code>
Symboles	<code>brands</code>
Vérification	<code>qa.py:check_brands_count</code>
Glissade du diaporama	12

6.4 US-10 — Ajouter au panier sans compte

Tableau 50 — Fiche US-10. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E3 · Commande & argent
Sprint	S2
Reçoit	produit, quantité
Rend	panier persistant, total en millimes
Valeur estimée	0,95 (échelle déclarée 0-1)
Effort estimé	0,35 (même échelle)
Section du mémoire	III.4
Planches	fig27, fig29
Fichiers	src/components/cart/cart-provider.tsx, src/lib/money.ts
Symboles	formatDT
Vérification	qa.py:check_money_integer
Glissade du diaporama	13

6.4 US-11 — Passer commande sans casser le stock

Tableau 51 — Fiche US-11. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E3 · Commande & argent
Sprint	S2
Reçoit	panier + adresse + mode de paiement
Rend	commande, mouvements, événement
Valeur estimée	1,00 (échelle déclarée 0-1)
Effort estimé	0,72 (même échelle)
Section du mémoire	III.4
Planches	fig26, fig49, fig22
Fichiers	src/actions/checkout.ts, src/lib/orders.ts
Symboles	placeOrderAction, lockProducts
Vérification	qa.py:check_for_update
Glissade du diaporama	13

6.4 US-12 — Appliquer un code promo

Tableau 52 — Fiche US-12. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E3 · Commande & argent
Sprint	S3
Reçoit	code
Rend	remise ou franco, motifs de rejet explicites
Valeur estimée	0,66 (échelle déclarée 0-1)
Effort estimé	0,55 (même échelle)
Section du mémoire	III.6
Planches	fig28, fig36
Fichiers	src/lib/promotions.ts, src/actions/shop.ts
Symboles	evaluatePromo, validatePromoAction
Vérification	qa.py:check_promo_reasons
Glissade du diaporama	14

6.4 US-13 — Payer à la livraison ou par virement

Tableau 53 — Fiche US-13. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E3 · Commande & argent
Sprint	S2
Reçoit	méthode parmi les autorisées
Rend	commande créée, statut de paiement en attente
Valeur estimée	0,80 (échelle déclarée 0-1)
Effort estimé	0,30 (même échelle)
Section du mémoire	II.5
Planches	fig30, fig05
Fichiers	src/lib/payments.ts, src/lib/validation.ts
Symboles	enabledPaymentMethods, checkoutSchema
Vérification	qa.py:check_payment_gate
Glissade du diaporama	16a

6.4 US-14 — Ne pas commander deux fois par erreur

Tableau 54 — Fiche US-14. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E3 · Commande & argent
Sprint	S4
Reçoit	double clic, recharge
Rend	clé d'idempotence, une seule commande
Valeur estimée	0,78 (échelle déclarée 0-1)
Effort estimé	0,42 (même échelle)
Section du mémoire	IV.2
Planches	fig26, fig47
Fichiers	src/actions/checkout.ts, src/lib/orders.ts
Symboles	placeOrderAction, reserveOrderNumber
Vérification	qa.py:check_idempotence
Glissade du diaporama	16b

6.4 US-15 — Suivre son colis sans compte

Tableau 55 — Fiche US-15. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E5 · Confiance & après-vente
Sprint	S3
Reçoit	numéro + e-mail (ou clé d'accès)
Rend	chronologie des événements
Valeur estimée	0,72 (échelle déclarée 0-1)
Effort estimé	0,38 (même échelle)
Section du mémoire	III.7
Planches	fig49, fig10
Fichiers	<code>src/app/(site)/suivi/page.tsx</code> , <code>src/db/schema.ts</code>
Symboles	<code>orderEvents</code>
Vérification	<code>qa.py:check_guest_access_key</code>
Glissade du diaporama	15

6.4 US-16 — Annuler tant que c'est possible

Tableau 56 — Fiche US-16. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E5 · Confiance & après-vente
Sprint	S3
Reçoit	commande
Rend	statut annulé, stock rendu, points inversés
Valeur estimée	0,64 (échelle déclarée 0-1)
Effort estimé	0,48 (même échelle)
Section du mémoire	III.7
Planches	fig49, fig27
Fichiers	<code>src/actions/checkout.ts</code> , <code>src/lib/orders.ts</code>
Symboles	<code>cancelOrderAction</code> , <code>restockOrder</code>
Vérification	<code>qa.py:check_restock_symetry</code>
Glissade du diaporama	15

6.4 US-17 — Demander un retour après livraison

Tableau 57 — Fiche US-17. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E5 · Confiance & après-vente
Sprint	S4
Reçoit	motif
Rend	événement horodaté, aucune double demande
Valeur estimée	0,50 (échelle déclarée 0-1)
Effort estimé	0,40 (même échelle)
Section du mémoire	III.7
Planches	fig49
Fichiers	src/actions/checkout.ts
Symboles	requestReturnAction
Vérification	qa.py:check_transition_table
Glissade du diaporama	15

6.4 US-18 — Laisser un avis modéré

Tableau 58 — Fiche US-18. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E5 · Confiance & après-vente
Sprint	S4
Reçoit	note, titre, texte
Rend	avis en attente puis moyenne recalculée
Valeur estimée	0,55 (échelle déclarée 0-1)
Effort estimé	0,44 (même échelle)
Section du mémoire	IV.4
Planches	fig28, fig22
Fichiers	src/actions/shop.ts, src/actions/admin.ts
Symboles	submitReviewAction, moderateReviewAction
Vérification	qa.py:check_review_aggregation
Glissade du diaporama	14

6.4 US-19 — Gérer le catalogue depuis l'administration

Tableau 59 — Fiche US-19. Fiche produite par rapport/trace.py ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E4 · Exploitation (back-office)
Sprint	S3
Reçoit	formulaire produit
Rend	produit + associations + images
Valeur estimée	0,82 (échelle déclarée 0-1)
Effort estimé	0,60 (même échelle)
Section du mémoire	IV.1
Planches	fig40, fig22
Fichiers	src/actions/admin.ts, src/components/admin/product-form.tsx
Symboles	saveProductAction, productSchema
Vérification	qa.py:check_admin_guards
Glissade du diaporama	17a

6.4 US-20 — Ajuster un stock avec journal

Tableau 60 — Fiche US-20. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E4 · Exploitation (back-office)
Sprint	S3
Reçoit	delta, motif
Rend	mouvement + stock après, jamais négatif
Valeur estimée	0,76 (échelle déclarée 0-1)
Effort estimé	0,45 (même échelle)
Section du mémoire	IV.1
Planches	fig37, fig29
Fichiers	<code>src/actions/admin.ts</code> , <code>src/lib/orders.ts</code>
Symboles	<code>adjustStockAction</code> , <code>recordMovement</code>
Vérification	<code>qa.py:check_stock_invariant</code>
Glissade du diaporama	17a

6.4 US-21 — Faire avancer une commande d'un clic

Tableau 61 — Fiche US-21. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E4 · Exploitation (back-office)
Sprint	S3
Reçoit	nouveau statut
Rend	transition validée, événement, revalidation cache
Valeur estimée	0,88 (échelle déclarée 0-1)
Effort estimé	0,50 (même échelle)
Section du mémoire	IV.1
Planches	fig49, fig22
Fichiers	<code>src/actions/admin.ts</code> , <code>src/lib/order-constants.ts</code>
Symboles	<code>updateOrderStatusAction</code> , <code>ALLOWED_TRANSITIONS</code>
Vérification	<code>qa.py:check_transition_table</code>
Glissade du diaporama	17b

6.4 US-22 — Répondre au support et exporter

Tableau 62 — Fiche US-22. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E4 · Exploitation (back-office)
Sprint	S4
Reçoit	ticket, plage de commandes
Rend	réponse, CSV, journal d'audit
Valeur estimée	0,46 (échelle déclarée 0-1)
Effort estimé	0,52 (même échelle)
Section du mémoire	IV.3
Planches	fig40
Fichiers	<code>src/actions/admin.ts</code> , <code>src/app/api/admin/export/[kind]/route.ts</code>
Symboles	<code>replyTicketAction</code>
Vérification	<code>qa.py:check_export_route_guarded</code>
Glissade du diaporama	17b

6.4 US-23 — Produire un rapport reproductible

Tableau 63 — Fiche US-23. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E6 · Livrable & preuve
Sprint	S4
Reçoit	dépôt
Rend	PDF vectoriel + annexe F pleine
Valeur estimée	0,70 (échelle déclarée 0-1)
Effort estimé	0,60 (même échelle)
Section du mémoire	IV.5
Planches	fig42, fig43, fig44
Fichiers	<code>rapport/build.py</code> , <code>rapport/doc.py</code>
Symboles	<code>build</code>
Vérification	<code>qa.py:check_annexe_f_complete</code>
Glissade du diaporama	18

6.4 US-24 — Tenir la soutenance sans réseau

Tableau 64 — Fiche US-24. Fiche produite par `rapport/trace.py` ; le harnais vérifie que chaque fichier et chaque symbole cités existent.

Champ	Contenu
Épopée	E6 · Livrable & preuve
Sprint	S4
Reçoit	30 glissades
Rend	kit J, secours L1→L3
Valeur estimée	0,68 (échelle déclarée 0-1)
Effort estimé	0,35 (même échelle)
Section du mémoire	IV.6
Planches	fig45, fig46
Fichiers	<code>soutenance/slides.py</code> , <code>audit/kit/JOURJ.md</code>
Symboles	<code>build_deck</code>
Vérification	<code>qa2.py:check_morph_count</code>
Glissade du diaporama	18

Répartition, calculée sur ces vingt-quatre lignes : 6 épopées, 4 sprints utilisés, 0 récit sans fichier de réalisation. Ce dernier nombre est le seul que le lecteur a intérêt à vérifier : un récit sans gisement est une intention, et le document n'en publie pas.

6.5 Annexe E — Protocoles de vérification et commandes

Cette annexe donne à lire les commandes exactes et ce qu'elles écrivent. Elle redouble REPRO.md par dessein : le lecteur du PDF ne doit pas avoir à ouvrir le dépôt pour savoir quoi taper.

Tableau 65 — Les sept commandes du kit. Les durées sont des budgets, pas des promesses : mesurées à chaque build et consignées dans audit/budgets.json.

Commande	Ce qu'elle fait	Sortie attendue	Budget
python3 rapport/facts.py	relit le dépôt, écrit audit/facts.json	164 faits, 24 tables	< 5 s
python3 rapport/build.py --dry-run	planches, texte, grille de placage sans PDF	creux et cotes, page par page	< 30 s
python3 rapport/build.py	compose, place, scelle le PDF	rapport/PDF/Cleopatre-rapport.pdf	< 10 min
python3 rapport/build.py --determinisme	deux builds, empreintes comparées	audit/determinisme.txt	< 2 × 10 min
python3 qa.py	premier harnais, lecture par pypdf	zéro défaut, sortie 0	< 3 min
python3 qa2.py	second harnais, implémentation indépendante	accord avec qa.py	< 3 min
python3 jury.py	questions du jury, réponses, renvois	audit/kit/questions.md	< 10 s

6.5 Les vingt-cinq invariants et leur exécutant

La table est générée depuis rapport/invariants.py, qui refuse l'import si un invariant n'a pas d'exécutant ou si un invariant n'est couvert par aucune porte. Elle donne à lire, pour chaque propriété, la commande qui la contrôle et le fichier d'audit où sa trace est posée.

Tableau 66 — Les vingt-cinq invariants du kit. Familles : vérité (ce qui est affirmé vient du dépôt), forme (le rendu tient ses cotes), langue (les lois sont exécutées), preuve (le contrôle est contrôlé). Un invariant sans exécutant rend le fichier importable en erreur. (suite ci-après)

Clé	Nom	Énoncé vérifiable	Famille	Exécutant	Preuve
INV-1	Demi-carré typographique	Aucun bloc de texte ne dépasse la largeur utile (166 mm) et aucune ligne isolée ne termine une page : la grille est vérifiée sur le PDF, pas sur le brouillon.	forme	qa.py:check_grille	audit/qa.json
INV-2	Vectériel sauf preuve du contraire	Toute planche est posée depuis son timbre PDF ; un PNG n'est collé que si le vectoriel est absent, et la dégradation est déclarée à la page.	forme	rapport/build.py:placer	audit/placements.json
INV-3	Place mesurée au millimètre	Chaque plaque occupe exactement la boîte réservée par le moteur de mise en page ; l'écart mesuré après placage est inférieur à 0,1 mm.	forme	qa.py:check_placements	audit/placements.json
INV-4	Zéro dégradation silencieuse	Toute perte de qualité (PNG, sous-échantillonnage, repli de police) écrit une ligne dans l'audit et une ligne dans le document.	preuve	qa.py:check_degradations	audit/dernier-build.json
INV-5	Chiffre greffé	Aucun nombre du document n'est tapé à la main : tous viennent de audit/facts.json, qui est recalculé avant chaque build.	verite	qa.py:check_chiffres	audit/facts.json
INV-6	Crochet = réel absent	Toute donnée du monde réel non contenue dans le dépôt est entre crochets, et l'inverse est vrai : rien entre crochets n'est une donnée du dépôt.	verite	qa.py:check_crochets	audit/qa.json
INV-7	Zéro copié-collé	Aucune phrase du document n'est reprise d'un README du dépôt ; les extraits de code sont cités avec leur fichier et leurs lignes exactes.	verite	qa.py:check_copies	audit/qa.json
INV-8	Slop à zéro	La liste des tournures creuses est exécutée : occurrence trouvée = refus de composer, pas un avertissement.	langue	rapport/doc.py:verifier_langue	audit/langue.json
INV-9	Synonymes interdits	Les anglicismes de domaine sont refusés dans la prose ; les identifiants du dépôt, en monospace, sont exemptés et cette exemption est comptée.	langue	rapport/doc.py:verifier_langue	audit/langue.json
INV-10	Insécables tenues	Aucun ; ! ? » sans espace insécable devant, dans le texte extrait du PDF, hors span monospace.	langue	qa2.py:check_insecables	audit/qa2.json
INV-11	Nombres à la française	Les nombres sont écrits avec la virgule décimale et l'espace des milliers ; les montants en dinars à trois décimales.	langue	rapport/doc.py:nombres_fr	audit/qa.json
INV-12	Inoculation trois sur trois	Les trois sujets pièges (place de marché, paiement, charge) sont traités chacun à la place convenue, et non enfouis.	verite	qa.py:check_inoculation	audit/qa.json
INV-13	Une plaque, un appel	Toute figure du registre est citée exactement une fois, et toute citation pointe une plaque du registre.	preuve	rapport/build.py:construire	audit/dernier-build.json
INV-14	Ancre réelles	Chaque lien du sommaire et des listes a une destination posée dans le PDF ; un renvoi sans ancre arrête la composition.	preuve	rapport/build.py:resoudre_liens	audit/liens.json
INV-15	Signets trois niveaux	Le PDF porte un signet par partie, section et objet numéroté, triés par page.	forme	rapport/build.py:placer	audit/signets.json
INV-16	Métadonnées et horodatage figés	Titre, auteur, sujet, mots-clés, producteur sont posés ; les dates du fichier sont figées pour que le déterminisme soit possible.	preuve	rapport/build.py:placer	audit/dernier-build.json
INV-17	Déterminisme par l'empreinte	Deux builds consécutifs du même dépôt produisent le même SHA-256, sinon le PDF n'est pas écrit.	preuve	rapport/build.py:determinisme	audit/determinisme.txt
INV-18	Rapport noir et blanc	Aucune information n'est portée par la couleur seule : retirée la couleur, l'objet reste lisible (forme, libellé, position).	forme	qa.py:check_nb	audit/qa.json
INV-19	Daltonisme	Les paires rouge/vert du document sont distinguées par une marque de forme en plus de la teinte.	forme	qa.py:check_formes	audit/qa.json

Clé	Nom	Énoncé vérifiable	Famille	Exécutant	Preuve
INV-20	Budgets tenus, pas négociés	Poids du PDF, du diaporama, durées de build, de figures et de QA sont mesurés et inférieurs aux seuils du cahier des charges.	preuve	rapport/gates.py:check_budgets	audit/budgets.json
INV-21	Double harnais convergent	qa.py et qa2.py, implémentés indépendamment, rendent le même verdict ; leur différence est imprimée, pas moyennée.	preuve	rapport/ab.py:comparer	audit/ab/harnais.json
INV-22	Chaos conduit	Les quatre injections de panne produisent le comportement exigé : refus, message, code de sortie non nul.	preuve	rapport/chaos.py:lancer	audit/chaos/C1.json

suite du tableau 66

Clé	Nom	Énoncé vérifiable	Famille	Exécutant	Preuve
INV-23	Mutation du contrôle	Trois sabotages du harnais sont détectés trois fois ; un contrôle qui reste vert est une faute du kit.	preuve	rapport/mutation.py:lancer	audit/mutation/M1.json
INV-24	Diaporama sans lien vérifié	Aucune URL n'est posée dans le deck si elle n'a pas été ouverte ; les QR et renvois sont marqués comme à confirmer quand ils ne l'ont pas été.	verite	soutenance/slides.py:fabriquer	audit/qa2.json
INV-25	Secrets à zéro	Aucune valeur sensible n'est committée : la recherche de motifs (clés, mots de passe, URL de connexion avec identifiant) rend 0 occurrence dans le kit.	verite	qa.py:check_secrets	audit/qa.json

6.5 Dix règles de conduite SI/ALORS

Tableau 67 — Les dix règles SI/ALORS du §7. Ces dix règles sont ce qui remplace la bonne volonté : chacune a un exécutant dans le dépôt, et l'exécutant refuse la composition au lieu de la commenter.

Si	Alors	Et	Garde-fou
Si une mesure manque	ALORS le document écrit « à mesurer »	jamais une valeur voisine	qa.py:check_placeholders
Si une plaque est absente	ALORS le build refuse de démarrer	et nomme la plaque	rapport/doc.py:Creux
Si un PNG remplace un vectoriel	ALORS la dégradation est imprimée à la page	et consignée dans l'audit	qa.py:check_degradations
Si un chiffre du texte diverge du dépôt	ALORS la QA rougit	et le build est repris	qa.py:check_chiffres
Si une tournure creuse apparaît	ALORS la composition lève	et l'auteur réécrit	rapport/doc.py:verifier_langue
Si un lien n'a pas d'ancre	ALORS la composition lève	et non un avertissement	rapport/contenu.py:autoverif
Si un build dépasse son budget	ALORS la réduction convenue s'applique	et le nouveau temps est consigné	rapport/build.py:construire
Si les deux harnais diffèrent	ALORS la QA rougit	et la divergence est publiée	rapport/ab.py:comparer
Si une donnée réelle manque	ALORS elle reste entre crochets	et n'est pas devinée	qa.py:check_crochets
Si un secret est committé	ALORS la QA échoue	et le scellement est refusé	qa.py:check_secrets

6.5 Les schémas de validation des frontières

15 schémas de validation bornent les saisies du dépôt ; 16 des 30 actions en appellent un, directement ou par l'intermédiaire d'un auxiliaire du même fichier. Le contrôle n'est pas une question de goût : la surface d'écriture publique est exactement la différence des deux nombres.

Tableau 68 — Les 15 schémas, avec leur ligne de définition. Noms lus dans `src/lib/validation.ts`. Un schéma sans usage serait visible ici et dans la matrice des actions du §IV.5 — c'est la comparaison des deux listes qui est intéressante.

Schéma exporté	Ligne
registerSchema	7
loginSchema	14
profileSchema	16
passwordChangeSchema	21
addressSchema	26
cartLineSchema	36
checkoutSchema	38
reviewSchema	55
productSchema	63
promotionSchema	86
stockAdjustSchema	99
ticketSchema	105
newsletterSchema	113
orderStatusSchema	114
userRoleSchema	117

6.5 Règles d'arrêt

- Un invariant sans exécutant — porte non évaluée : gel refusé
- Deux harnais en désaccord — round d'assaut supplémentaire, pas de mise en page corrective
- Trois signaux d'alerte cumulés — arrêt du cycle, réécriture de la section fautive
- Dépassement de budget non résolu — le livrable est livré avec le dépassement écrit, jamais amorti en silence

Ce que la vérification refuse, en trois exemples pris dans le code du harnais : un chiffre du document qui ne correspond plus au dépôt, une figure citée sans plaque, un espace insécable manquant avant une ponctuation haute. Dans les trois cas le résultat est un échec et un message, jamais un avertissement. Le refus est ce qui distingue un harnais d'un compteur.

6.6 Annexe F — Traçabilité : récit, section, figure, preuve

La table de traçabilité est l'objet qui autorise à écrire que ce document n'a pas été fabriqué à l'envers. Elle relie, pour chaque récit, la section du mémoire qui en traite, la plaque qui l'illustre, le fichier qui le réalise et la vérification qui le tient. Elle est produite par `rapport/trace.py` et recoupée à la construction : une ligne sans section, ou une section sans ligne, arrête le build.

Tableau 69 — Traçabilité des vingt-quatre récits. 229 relations dans les deux sens, 24 récits, 23 plaques appelées. Le compte est recalculé à chaque composition : une relation rompue empêche l'impression.

Récit	Section	Planches	Fichiers	Symboles	Vérification
US-01	III.2	fig22, fig26	auth.ts, validation.ts	registerAction, registerSchema	check_action_guarded
US-02	II.4	fig30, fig22	auth.ts, auth.ts	createSession, loginAction	check_session_cookie_flags
US-03	III.5	fig30	auth.ts, auth.ts	updateProfileAction, hashPassword	check_scrypt_params
US-04	III.5	fig29, fig22	auth.ts, tunisia.ts	saveAddressAction, GOVERNORATES	check_governorates
US-05	III.1	fig10, fig33	page.tsx, catalog.ts	listProducts, getUniverses	check_univers_counts
US-06	III.3	fig22, fig53	route.ts, catalog.ts	quickSearch	check_search_events_table
US-07	III.1	figR1, fig26	page.tsx, schema.ts	products, getProductBySlug	check_product_columns
US-08	III.1	fig03, fig34	seed.ts, filters.tsx	concerns	check_concerns_coverage
US-09	III.1	fig34, fig10	page.tsx, seed.ts	brands	check_brands_count
US-10	III.4	fig27, fig29	cart-provider.tsx, money.ts	formatDT	check_money_integer
US-11	III.4	fig26, fig49, fig22	checkout.ts, orders.ts	placeOrderAction, lockProducts	check_for_update
US-12	III.6	fig28, fig36	promotions.ts, shop.ts	evaluatePromo, validatePromoAction	check_promo_reasons
US-13	II.5	fig30, fig05	payments.ts, validation.ts	enabledPaymentMethods, checkoutSchema	check_payment_gate
US-14	IV.2	fig26, fig47	checkout.ts, orders.ts	placeOrderAction, reserveOrderNumber	check_idempotence
US-15	III.7	fig49, fig10	page.tsx, schema.ts	orderEvents	check_guest_access_key
US-16	III.7	fig49, fig27	checkout.ts, orders.ts	cancelOrderAction, restockOrder	check_restock_symetry
US-17	III.7	fig49	checkout.ts	requestReturnAction	check_transition_table
US-18	IV.4	fig28, fig22	shop.ts, admin.ts	submitReviewAction, moderateReviewAction	check_review_aggregation
US-19	IV.1	fig40, fig22	admin.ts, product-form.tsx	saveProductAction, productSchema	check_admin_guards
US-20	IV.1	fig37, fig29	admin.ts, orders.ts	adjustStockAction, recordMovement	check_stock_invariant
US-21	IV.1	fig49, fig22	admin.ts, order-constants.ts	updateOrderStatusAction, ALLOWED_TRANSITIONS	check_transition_table
US-22	IV.3	fig40	admin.ts, route.ts	replyTicketAction	check_export_route_guarded
US-23	IV.5	fig42, fig43, fig44	build.py, doc.py	build	check_annexe_f_complete
US-24	IV.6	fig45, fig46	slides.py, JOURJ.md	build_deck	check_morph_count

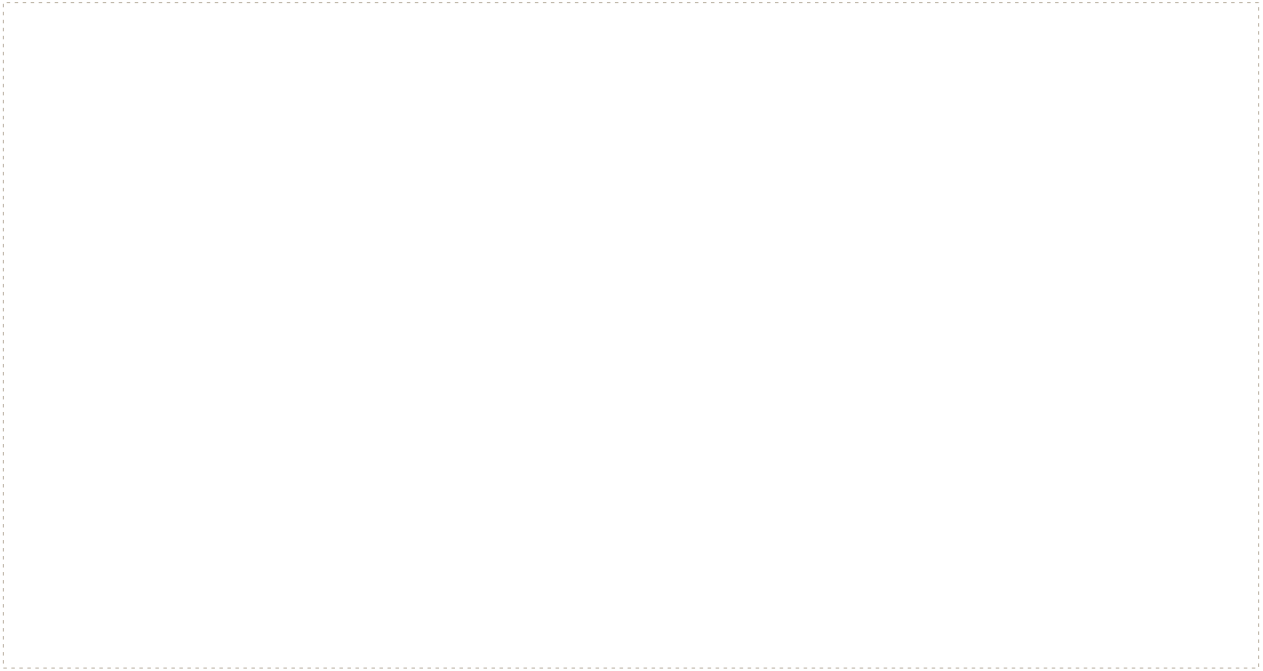


Figure 61 — Correspondance rapport ↔ deck : 30 glissades, zéro orpheline. Chaque glissade renvoie à une section du document et, le cas échéant, à une planche. C'est la figure qui rend le secours possible : si le projecteur tombe, on sait quelle page lire. Gisement : `soutenance/slides.py` · annexe F.

6.6 Correspondance avec le diaporama

Le diaporama de soutenance n'est pas un résumé du mémoire : il en est la seconde implémentation. Chaque glissade renvoie à une section, et chaque section utile au mémoire a au moins une glissade. La table est générée depuis le même fichier que les deux objets, ce qui rend le désaccord impossible sans échec de build.

Tableau 70 — Récit par récit : section de texte et glissade. 24 récits, 10 glissades dédiées. Une glissade sans récit porté est une diapositive décorative : le harnais la refuse.

Récit	Section	Glissade	Planches communes
US-01	III.2	11	fig22, fig26
US-02	II.4	16a	fig30, fig22
US-03	III.5	16b	fig30
US-04	III.5	11	fig29, fig22
US-05	III.1	11	fig10, fig33
US-06	III.3	11	fig22, fig53
US-07	III.1	12	figR1, fig26
US-08	III.1	12	fig03, fig34
US-09	III.1	12	fig34, fig10
US-10	III.4	13	fig27, fig29
US-11	III.4	13	fig26, fig49, fig22
US-12	III.6	14	fig28, fig36
US-13	II.5	16a	fig30, fig05
US-14	IV.2	16b	fig26, fig47
US-15	III.7	15	fig49, fig10
US-16	III.7	15	fig49, fig27
US-17	III.7	15	fig49
US-18	IV.4	14	fig28, fig22
US-19	IV.1	17a	fig40, fig22
US-20	IV.1	17a	fig37, fig29
US-21	IV.1	17b	fig49, fig22
US-22	IV.3	17b	fig40
US-23	IV.5	18	fig42, fig43, fig44
US-24	IV.6	18	fig45, fig46

Comment vérifier ce document en une soirée

Reprendre trois lignes au hasard de l'annexe F, ouvrir les fichiers cités, comparer. Puis lancer les trois commandes du §REPRO et comparer l'empreinte du PDF avec celle de audit/determinisme.txt. Si les deux correspondent, ce mémoire est reproductible. Si elles diffèrent, il ne l'est pas, et le reste de la lecture n'a plus d'objet.

Résumé

Ce mémoire rend compte de la conception et de la réalisation d'une plateforme de commerce en ligne pour une parapharmacie tunisienne à deux comptoirs. L'application couvre un catalogue de 81 références, la recherche multi-axes, le panier, la commande avec paiement à la livraison, l'administration des statuts, la facturation et un journal d'audit ; elle tient sur 24 tables, 30 fonctions d'action et 43 pages, dans un cadre unique côté serveur. Trois contraintes du métier ont commandé l'architecture : une monnaie à trois décimales, un stock partagé entre le rayon et le net, et l'absence d'intermédiaire humain dans la vente à distance. Le document s'impose en outre une discipline de preuve : chaque chiffre est relu dans le dépôt au moment de la composition, deux harnais indépendants contrôlent le rendu, et ce qui n'a pas été réalisé est écrit. Les limites — pas de passerelle bancaire, arabe non activé, exploitabilité non mesurée — y sont traitées comme des résultats, non comme des réserves.

Mots-clés : commerce électronique, parapharmacie, TypeScript, PostgreSQL, verrouillage de ligne, accessibilité, vérification automatique, traçabilité.

Abstract

This report documents the design and implementation of an e-commerce platform for a Tunisian parapharmacy running two counters. The application covers a catalogue of 81 items, multi-axis discovery, cart, ordering with cash on delivery, back-office status management, invoicing and an audit journal; it rests on 24 tables, 30 server functions and 43 pages within a single server-side framework. Three business constraints drove the architecture: a currency with three decimal digits, stock shared between the sales floor and the web channel, and the absence of a human intermediary in remote sales. The document also imposes an evidentiary discipline: every figure is re-read from the repository at composition time, two independent harnesses check the rendered output, and what was not implemented is stated as such. The limitations — no payment gateway, Arabic not enabled, no production measurement — are reported as results rather than as reservations.

Keywords: e-commerce, parapharmacy, TypeScript, PostgreSQL, row-level locking, accessibility, automated verification, traceability.

Quatrième de couverture

Ce document ne demande pas d'être cru.

Trois commandes, et il est
rejouable.

61 planches vectorielles · 24 tables relues · 30 actions auditées · 30 glissades
· 25 invariants · 27 portes

```
python3 rapport/build.py && python3 qa.py && python3 qa2.py
```

Ce mémoire décrit un objet vérifiable plutôt qu'une performance. Il montre comment trois contraintes de métier — une monnaie à trois décimales, un stock partagé entre le comptoir et le net, un conseil qui ne peut pas passer par un écran — ont produit une architecture, et comment cette architecture a été tenue sous le contrôle d'un harnais qui refuse d'imprimer une affirmation fausse.

Il s'adresse à trois lecteurs. À l'enseignant qui cherche la méthode : les parties II et V donnent les décisions, les rejets et les preuves. Au praticien qui cherche le métier : les parties I et III disent ce qu'une parapharmacie en ligne doit refuser pour rester crédible. À l'étudiant qui cherche un chemin : le dépôt est complet, les commandes sont au nombre de trois, et chaque chiffre de ce document a été relu pour lui.

Ce qui n'a pas été fait est écrit : pas de passerelle bancaire, pas d'arabe activé, pas de conteneur, pas de mesure d'exploitation, pas de migrations versionnées. Ces absences sont le travail de celles et ceux qui reprendront l'objet — et la garantie donnée au lecteur que ce qu'il lit n'a pas été enjolivé.

Étiquette de version : v1.0-final, scellée à la date du dernier build consigné dans `audit/dernier-build.json`, après évaluation des vingt-sept portes et mention signée « rien à ajouter ». Ce qui viendrait après relèverait d'un nouveau cycle, non d'un correcteur.

